

파싱 전환 — 내부망 적용 지시서 v2 (스키마 무변경)

DB 컬럼을 하나도 늘리지 않고 같은 기능을 하도록 고친 판. v1(컬럼 7개판)을 대체한다

작성일 2026-09-23

목표 상태 9c42b28 — GGReportAgent/uploader

2장(델타) 기준 v1 지시서를 이미 적용한 상태 (1eb7757)

부록(전체) 기준 파싱 전환 이전 상태 (2ebfcda)

표기 초록 + = 넣을 줄 · 붉은 - = 지울 줄 · 나머지는 위치를 잡는 주변 줄

DDL 이 필요 없어졌다. 2026-09-23_TSKGIAF01_파싱컬럼추가-*.sql 은 적용하지 않는다. 이미 적용했다면 그대로 뒤도 된다 — 앱이 그 컬럼을 읽지도 쓰지도 않는다(지우고 싶으면 그 파일의 롤백 문단을 쓴다). DBA 제출도 필요 없다.

1. 무변경안이 어떻게 돌아가는가

컬럼 7개에 담던 값을 파일명 규칙에서 읽는다. 규칙 정보는 code/DocumentType 이고, Mapper XML 의 <sql> 조각이 같은 규칙의 SQL 판이다.

v1 에서 쓰던 컬럼	무변경안에서 읽는 곳
문서종류구분	원본파일명 확장자 — .ppt·.pptx 제안서 / .pdf·.hwp·.hwpX RFP
파싱상태구분	저장경로내용 확장자 — .json·.md 면 성공(산출물이 생겼다), 아니면 실패
파싱산출물경로내용	저장경로내용 에 그대로 넣는다. 원본은 userdata + 원본파일명 으로 재구성한다
문서일자	산출물 파일명 앞 토큰 — 20260801_공공기관_서울에너지공사_20260923.md
파싱일시	분류일시 — 이 흐름에서 둘은 같은 시각이다
문서페이지수	저장하지 않는다 (제안서 JSON 안에 total_slides 가 있다)
파싱메시지내용	저장할 자리가 없다 — 아래 참조

없는 것은 하나다 — 파싱 실패 사유. 넣을 컬럼이 없어 화면에는 "사유는 서버 로그 확인" 으로만 나온다. 대신 DocumentParseService 가 스택까지 로그에 남긴다. 실패가 찾아지면 그때 파싱상태구분 + 파싱메시지내용 2개만 추가하는 절충안으로 옮기면 된다 (TODO_modify.md 의 "내부망 스키마 변경 폭 줄이기" 참조).

규칙이 두 곳에 있다. code/DocumentType (자바)과 Mapper XML 의 sql 조각(SQL)이 같은 판별을 각각 표현한다. 한쪽만 고치면 화면 숫자와 목록이 따로 논다. 확장자를 늘리거나 산출물 이름 규칙을 바꿀 때는 반드시 둘을 같이 고칠 것.

그 밖에 달라진 동작 두 가지

- 원본파일명에 실제로 저장된 이름을 넣는다. 같은 이름이 이미 있으면 _yyyyMMddHHmmss 가 붙는데, 파싱에 성공하면 저장경로가 산출물로 바뀌므로 원본을 다시 찾으려면 이 이름이 정확해야 한다(재파싱이 여기 걸린다).
- 삭제(04)·반려(03) 건은 파싱 집계에서 뺀다. 사람이 처리를 끝낸 행이라 섞이면 KPI 가 틀어진다.

2. 델타 — v1 을 이미 적용했다면 여기만

v1 지시서대로 이미 고쳐 두었다면 **이 장만 적용하면** 무변경안이 된다. 파일 15개다. `parse` 패키지와 `agent-test.js` 는 그대로 두면 된다 — 바뀌지 않았다.

이 장을 적용하고 나면 DB 컬럼 7개는 쓰이지 않는다. 이미 추가했다면 그대로 뒀도 되고, 지우려면 마이그레이션 DDL 의 롤백 문단을 쓴다.

수정 `code/DocumentType.java`

```
@@ -71,6 +71,40 @@ public final class DocumentType {

    int dot = filename.lastIndexOf('.');
    return dot < 0 ? "" : filename.substring(dot + 1).toLowerCase();
}

+ // — 스키마 무변경안(2026-09-23)의 판별 규칙 —————
+ // 컬럼을 늘리지 않으므로 문서종류·파싱 여부를 **파일명에서** 읽는다.
+ // SQL(UploadedFileMapper.xml)도 같은 규칙을 LIKE 로 표현한다 — 한쪽을 바꾸면 다른 쪽도 바꿀 것.
+
+ /** 원본 확장자로 문서종류를 판별한다. 규칙 밖(md·xlsx 등 옛 업로드)이면 null. */
+ public static String ofFileName(String originalName) {
+     String ext = extensionOf(originalName);
+     for (Map.Entry<String, Set<String>> e : EXTENSIONS.entrySet()) {
+         if (e.getValue().contains(ext)) return e.getKey();
+     }
+     return null;
+ }
+
+ /** 산출물 확장자. 제안서는 JSON, RFP 요약은 Markdown 이다. */
+ public static boolean isOutputPath(String path) {
+     String ext = extensionOf(path);
+     return "json".equals(ext) || "md".equals(ext);
+ }
+
+ /**
+  * 산출물 파일명 앞의 날짜 토큰. {@code 20240315_지자체_...} → {@code 20240315},
+  * {@code 2025_대학교_...} → {@code 2025}. 형식이 아니면 null.
+  */
+ }
```

```

+ public static String leadingDate(String outputFileName) {
+     if (outputFileName == null) return null;
+     int bar = outputFileName.indexOf('_');
+     if (bar != 4 && bar != 8) return null;
+     String head = outputFileName.substring(0, bar);
+     for (int i = 0; i < head.length(); i++) {
+         if (!Character.isDigit(head.charAt(i))) return null;
+     }
+     return head;
+ }
+
private DocumentType() {}
}

```

수정 controller/KGI10000\$DashboardView.java

```

@@ -27,18 +27,18 @@ public class KGI10000$DashboardView {

    Model model) {

        // — 2026-09-23 파싱 전환: KPI 측이 분류 → 파싱으로 바뀌었다 —
        long total = fileMapper.countAll();
-       long parseSuccess = fileMapper.countByParseStatus("SUCCESS");
-       long parseFailed = fileMapper.countByParseStatus("FAILED");
+       long parseSuccess = fileMapper.countParseSuccess();
+       long parseFailed = fileMapper.countParseFailed();

        long unclassified = fileMapper.countParsedUnclassified();

        model.addAttribute("total", total);
        model.addAttribute("parseSuccess", parseSuccess);
        model.addAttribute("parseFailed", parseFailed);
        model.addAttribute("unclassified", unclassified);
-       model.addAttribute("proposalCount", fileMapper.countByDocType("BID_PROPOSAL"));
-       model.addAttribute("rfpCount", fileMapper.countByDocType("RFP"));
+       model.addAttribute("proposalCount", fileMapper.countProposalDocs());
+       model.addAttribute("rfpCount", fileMapper.countRfpDocs());

        // 좌측 네비 배치 - 미분류 건수에서 파싱 실패 건수로 바뀌었다
        model.addAttribute("parseFailedCount", parseFailed);

        List<UploadedFile> recent = fileMapper.findRecent(10);

```

수정 controller/KGI11100\$UploadAction.java

```

@@ -45,9 +45,9 @@ public class KGI11100$UploadAction {

    return handle(DocumentType.RFP, files, model);
}

```

```

private String handle(String docType, List<MultipartFile> files, Model model) {
-   model.addAttribute("parseFailedCount", fileMapper.countByParseStatus("FAILED"));

+   model.addAttribute("parseFailedCount", fileMapper.countParseFailed());

    List<MultipartFile> validFiles = files.stream()
        .filter(f -> !f.isEmpty())
        .collect(Collectors.toList());

```

수정 controller/KGI12000\$ParseStatusList.java

```

@@ -41,10 +41,10 @@ public class KGI12000$ParseStatusList {

    model.addAttribute("docTypes", DocumentType.nameToCode().keySet());
    model.addAttribute("docType", docType);
    model.addAttribute("parseStatus", parseStatus);
    model.addAttribute("category", category);
-   model.addAttribute("successCount", fileMapper.countByParseStatus("SUCCESS"));
-   model.addAttribute("parseFailedCount", fileMapper.countByParseStatus("FAILED"));
+   model.addAttribute("successCount", fileMapper.countParseSuccess());
+   model.addAttribute("parseFailedCount", fileMapper.countParseFailed());

    model.addAttribute("unclassifiedCount", fileMapper.countParsedUnclassified());
    return "parse-status";
}

```

수정 controller/KGI12100\$FileStatusClassify.java

```

@@ -62,14 +62,15 @@ public class KGI12100$FileStatusClassify {

    try {
        institutionService.save(name, category.trim());
        Optional<UploadedFile> result = parseService.reparse(id);
-       if (result.isPresent() && "SUCCESS".equals(result.get().getParseStatus())) {
+       if (result.isPresent() && result.get().isParsed()) {

            ra.addFlashAttribute("message",
                "기관 등록 후 재파싱 완료: " + name + " → " + result.get().getOutputFileName());
        } else {
-           ra.addFlashAttribute("error", "기관은 등록했으나 재파싱에 실패했습니다"

+           + (result.isPresent() ? " - " + result.get().getParseMessage() : ""));

+           // ⚠ 실패 사유는 컬럼이 없어 저장하지 않는다(스키마 무변경안) - 로그를 봐야 한다.

+           ra.addFlashAttribute("error",

+               "기관은 등록했으나 재파싱에 실패했습니다 - 사유는 서버 로그를 확인하세요");

        }
    } catch (Exception e) {
        log.warn("기관 지정 재파싱 실패: id={}", id, e);
        ra.addFlashAttribute("error", "처리 실패: " + e.getMessage());
    }

```

수정 controller/KGI12110\$ParseReparse.java

```
@@ -30,14 +30,15 @@ public class KGI12110$ParseReparse {  
  
    public String execute(@PathVariable Long id, RedirectAttributes ra) {  
        Optional<UploadedFile> result = parseService.reparse(id);  
        if (!result.isPresent()) {  
            ra.addFlashAttribute("error", "파일을 찾을 수 없습니다 (id=" + id + ")");  
-        } else if ("SUCCESS".equals(result.get().getParseStatus())) {  
+        } else if (result.get().isParsed()) {  
  
            ra.addFlashAttribute("message", "재파싱 완료: " + result.get().getOriginalName()  
                + " → " + result.get().getOutputFileName());  
        } else {  
+        // ⚠ 실패 사유는 저장하지 않는다(스키마 무변경안). 로그를 볼 것.  
  
            ra.addFlashAttribute("error", "재파싱 실패: " + result.get().getOriginalName()  
-                + " - " + result.get().getParseMessage());  
+                + " - 사유는 서버 로그를 확인하세요");  
  
        }  
        return "redirect:/parse-status";  
    }  
}
```

수정 controller/KGI12400\$ClassifiedFileList.java

```
@@ -33,8 +33,8 @@ public class KGI12400$ClassifiedFileList {  
  
    List<UploadedFile> classifiedFiles =  
        fileMapper.searchParseStatus(null, "SUCCESS", Boolean.TRUE);  
    model.addAttribute("classifiedFiles", classifiedFiles);  
    model.addAttribute("categories", InstitutionCategory.names());  
-    model.addAttribute("parseFailedCount", fileMapper.countByParseStatus("FAILED"));  
+    model.addAttribute("parseFailedCount", fileMapper.countParseFailed());  
  
    return "classified-rework";  
}  
}
```

수정 controller/ProposalApiController.java

```
@@ -76,11 +76,10 @@ public class ProposalApiController {  
  
    if (!found.isPresent() || !DocumentType.BID_PROPOSAL.equals(found.get().getDocType())) {  
        return error(HttpStatus.NOT_FOUND, "입찰제안서를 찾을 수 없습니다 (id=" + id + ")");  
    }  
    UploadedFile f = found.get();  
-    if (!"SUCCESS".equals(f.getParseStatus()) || f.getOutputPath() == null) {  
-  
-        return error(HttpStatus.NOT_FOUND, "파싱에 성공하지 않은 제안서입니다 (id=" + id + ", 상태="  
-            + f.getParseStatus() + ")");  
+    if (!f.isParsed()) {  
+        return error(HttpStatus.NOT_FOUND, "파싱에 성공하지 않은 제안서입니다 (id=" + id + ")");  
    }  
    Path json = Paths.get(f.getOutputPath());  
    if (!Files.exists(json)) {  
        return error(HttpStatus.NOT_FOUND, "파싱 JSON 파일이 없습니다: " + json.getFileName());  
    }  
}
```

```

@@ -102,9 +101,8 @@ public class ProposalApiController {

    m.put("fileName", f.getOriginalName());
    m.put("institutionName", f.getInstitutionName());
    m.put("institutionCategory", f.getCategoryLabel());
    m.put("year", f.getDocDate());
-   m.put("totalSlides", f.getPageCount());

    m.put("jsonFileName", f.getOutputFileName());
    m.put("uploadedAt", f.getUploadedAt() != null ? f.getUploadedAt().format(TS) : null);
    m.put("parsedAt", f.getParsedAt() != null ? f.getParsedAt().format(TS) : null);
    return m;

```

수정 controller/RfpApiController.java

```

@@ -86,11 +86,10 @@ public class RfpApiController {

    if (!found.isPresent() || !DocumentType.RFP.equals(found.get().getDocType())) {
        return error(HttpStatus.NOT_FOUND, "입찰공고문(RFP)을 찾을 수 없습니다 (id=" + id + ")");
    }
    UploadedFile f = found.get();
-   if (!"SUCCESS".equals(f.getParseStatus()) || f.getOutputPath() == null) {
-
-       return error(HttpStatus.NOT_FOUND, "파싱에 성공하지 않은 공고문입니다 (id=" + id + ", 상태="
-
-           + f.getParseStatus() + ")");
+   if (!f.isParsed()) {
+       return error(HttpStatus.NOT_FOUND, "파싱에 성공하지 않은 공고문입니다 (id=" + id + ")");
    }
    Path md = Paths.get(f.getOutputPath());
    if (!Files.exists(md)) {
        return error(HttpStatus.NOT_FOUND, "요약 MD 파일이 없습니다: " + md.getFileName());
@@ -123,9 +122,8 @@ public class RfpApiController {

    m.put("fileName", f.getOriginalName());
    m.put("institutionName", f.getInstitutionName());
    m.put("institutionCategory", f.getCategoryLabel());
    m.put("noticeDate", f.getDocDate());
-   m.put("pageCount", f.getPageCount());

    m.put("mdFileName", f.getOutputFileName());
    m.put("uploadedAt", f.getUploadedAt() != null ? f.getUploadedAt().format(TS) : null);
    m.put("parsedAt", f.getParsedAt() != null ? f.getParsedAt().format(TS) : null);
    return m;

```

수정 domain/UploadedFile.java

```

@@ -26,98 +26,122 @@ public class UploadedFile {

    private String status;
    private LocalDateTime uploadedAt;
    private LocalDateTime classifiedAt;

-   // — 2026-09-23 파싱 전환으로 추가한 7개 —————
-
-   /** 문서종류구분. code/DocumentType 의 이름값(BID_PROPOSAL·RFP). DB 에는 코드(01·02)로 들어간다. */
-
-   private String docType;
-
-   /** 파싱상태구분. code/ParseStatus 의 이름값(PENDING·SUCCESS·FAILED). DB 에는 코드(01·02·03). */
-
-   private String parseStatus;

```

```

- /** 파싱 실패 사유, 화면의 실패 배지 옆에 그대로 보여 준다. */
- private String parseMessage;
- /** 산출물(제안서 JSON·RFP 요약 MD) 경로. ⚠ 원본 경로인 storedPath 와 다르다. */
- private String outputPath;
- /** 제안서 슬라이드 수 / RFP 페이지 수. */
- private Integer pageCount;
- private LocalDateTime parsedAt;
- /**
-  * 제안서는 연도(yyyy), RFP 는 공고일(yyyyMMdd).
-  * ⚠ 기존 {@code year}(문서년 VARCHAR2(4))는 8자리를 담지 못해 새 컬럼을 쓴다.
-  * 옛 행이 문서년을 쓰고 있으므로 그 필드는 남겨 두고 <b>새 코드는 이 값만</b> 쓴다.
-  */
- private String docDate;
-
- /** 기관 테이블에 없을 때의 기관분류 값. ⚠ InstitutionCategory 코드에는 없는 값이다. */
- public static final String UNKNOWN_CATEGORY = "미분류";
-
- public UploadedFile() {}
-
- /** 파싱 전환 후의 업로드. 기관·연도는 파싱해서 채우므로 이 시점에는 모른다. */
- public UploadedFile(String docType, String originalName, String storedPath) {
-     this.docType = docType;
-
- + /**
- +  * 파싱 전환 후의 업로드 (2026-09-23).
- +  *
- +  * <p>⚠ 문서종류·파싱상태를 <b>컬럼으로 두지 않는다</b>(스키마 무변경안). 문서종류는
- +  * 원본 확장자로, 파싱 성공 여부는 저장경로의 확장자로 판별한다 – getDocType().isParsed() 참조.
- +  */
- + public UploadedFile(String originalName, String storedPath) {
-
-         this.originalName = originalName;
-         this.storedPath = storedPath;
-         this.status = "UNCLASSIFIED";
- -         this.parseStatus = "PENDING";
-
-         this.uploadedAt = LocalDateTime.now();
-     }
-
- /**
- -  * 파싱 성공 기록.
- +  * 파싱 성공 기록 (스키마 무변경안).
-
-  *
- -  * <p>⚠ 파싱상태와 분류상태는 <b>다른 축</b>이다. 파싱은 됐는데 기관 테이블에 없어
-
-  * 분류가 안 되는 건이 정상적으로 존재한다 – 그 경우 분류상태는 UNCLASSIFIED 로 남는다.

```

```

+ * <p>산출물 경로를 {@code 저장경로내용} 에 넣는다. 원본 경로는
+ * {@code userdata + 원본파일명} 으로 언제나 재구성되므로 따로 보관하지 않는다.
+ * 파싱 시각은 {@code 분류일시} 에 넣는다 - 이 흐름에서 둘은 같은 시각이다.
+ *
+ * <p>⚠ 기관을 못 찾으면 분류상태는 UNCLASSIFIED 로 남는다. 그래도 산출물은 만들어졌으므로
+ * 화면에서는 "파싱 성공 + 기관 미분류" 로 보인다(판별은 저장경로 확장자로 한다).

```

```

*/
public void markParsed(String institutionName, String category, String docDate,
- String outputPath, Integer pageCount, String message) {
+ String outputPath, LocalDateTime parsedAt) {

    this.institutionName = institutionName;
    this.category = category;
- this.docDate = docDate;

- this.outputPath = outputPath;
- this.pageCount = pageCount;
- this.parseMessage = truncate(message);
- this.parseStatus = "SUCCESS";
- this.parsedAt = LocalDateTime.now();

- if (category != null && !UNKNOWN_CATEGORY.equals(category)) {
-     this.status = "CLASSIFIED";
-     this.classifiedAt = this.parsedAt;
- } else {
-     this.status = "UNCLASSIFIED";
-     this.classifiedAt = null;
+ this.storedPath = outputPath;
+ this.classifiedAt = parsedAt;
+ if (docDate != null && docDate.length() == 4) {
+     this.year = docDate;        // 제안서 연도는 기존 문서년(4) 에 그대로 들어간다
    }
+ this.status = (category != null && !UNKNOWN_CATEGORY.equals(category))
+     ? "CLASSIFIED" : "UNCLASSIFIED";

}

- public void markParseFailed(String message) {
-     this.parseStatus = "FAILED";
-     this.parseMessage = truncate(message);
-     this.parsedAt = LocalDateTime.now();
+ /**

```

```

+   * 파싱 실패 기록.
+
+   *
+   * <p>⚠ <b>실패 사유는 저장하지 않는다</b></p>(널을 컬럼이 없다 - 스키마 무변경안의 유일한 손실).
+   * 사유는 서버 로그에만 남는다. 저장경로는 원본 그대로 두며, 그 사실이 곧 "파싱 실패" 표시다.
+   */
+   public void markParseFailed(String originalPath) {
+       this.storedPath = originalPath;
+       this.classifiedAt = null;
+       this.status = "UNCLASSIFIED";
+   }
-   /** 파싱메시지내용 은 1000자다. 넘치면 잘라 넣는다(길이 초과로 INSERT 가 죽는 쪽이 더 나쁘다). */
-   private static String truncate(String message) {
-       if (message == null) return null;
-       return message.length() <= 1000 ? message : message.substring(0, 997) + "...";
-   }
-
-   /**
+   * 화면에 보여 줄 기관분류. DB 에는 "미분류" 코드가 없어 NULL 로 저장하므로
+   * 읽을 때 되돌린다(Mapper XML 의 {@code 기관분류값} 조각 참조).
+   */
+   public String getCategoryLabel() {
+       return category == null ? UNKNOWN_CATEGORY : category;
+   }
-   /** 문서종류 표시명. 옛 행은 null 이라 "이전 방식" 으로 보여 준다. */
+   // — 파생값 (스키마 무변경안) —————
+   // 컬럼을 늘리지 않으려고 아래 값들은 **저장하지 않고 그때그때 계산**한다.
+   // 규칙이 한곳(DocumentType)에 모여 있어 SQL 과 화면이 같은 기준을 쓴다.
+
+   /** 문서종류. 원본 확장자로 판별한다. 규칙 밖이면 null(옛 방식 업로드). */
+   public String getDocType() {
+       return com.kb.uploader.code.DocumentType.ofFileName(originalName);
+   }
+
+   public String getDocTypeLabel() {
-       return docType == null ? "이전 방식" : com.kb.uploader.code.DocumentType.label(docType);
+       String type = getDocType();
+       return type == null ? "이전 방식" : com.kb.uploader.code.DocumentType.label(type);
+   }
+
+

```

```

+  /**
+   * 파싱 성공 여부. <b>저장경로가 산출물(.json.md)이면 성공</b>이다.
+   * 원본 확장자 그대로면 아직 산출물이 없다는 뜻이라 실패로 본다.
+   */
+   public boolean isParsed() {
+       return com.kb.uploader.code.DocumentType.isOutputPath(storedPath);
+   }
+
+   /** 화면-API 가 쓰는 파싱상태 문자열. 컬럼이 아니라 계산값이다. */
+   public String getParseStatus() {
+       if (getDocType() == null) return null;    // 옛 방식 행
+       return isParsed() ? "SUCCESS" : "FAILED";
+   }
+
+   /** 산출물 경로. 파싱 전/실패면 null. */
+   public String getOutputPath() {
+       return isParsed() ? storedPath : null;
+   }
+
+   /** 파싱 시각. 분류일시를 그대로 쓴다. */
+   public LocalDateTime getParsedAt() {
+       return isParsed() ? classifiedAt : null;
+   }
+
+   /**
+   * 제안서 연도(4) 또는 RFP 공고일(8). 산출물 파일명의 앞 토큰이 그 값이다
+   * ({@code 20240315_지자체_서울특별시_20260923.md}). 없으면 기존 문서년으로 떨어진다.
+   */
+   public String getDocDate() {
+       String head = com.kb.uploader.code.DocumentType.leadingDate(getOutputFileName());
+       return head != null ? head : year;
+   }
+
+   /** 산출물 파일명만. 화면 표시용. */
+   public String getOutputFileName() {
+       if (outputPath == null) return null;
+
+       int slash = Math.max(outputPath.lastIndexOf('/'), outputPath.lastIndexOf('\\'));

```

```

-     return slash < 0 ? outputPath : outputPath.substring(slash + 1);
+     String path = getOutputPath();
+     if (path == null) return null;
+     int slash = Math.max(path.lastIndexOf('/'), path.lastIndexOf('\\'));
+     return slash < 0 ? path : path.substring(slash + 1);
}

public UploadedFile(String originalName, String storedPath,
                    String year, String institutionName) {
@@ -163,19 +187,5 @@ public class UploadedFile {

    public void setSystemUserNo(String systemUserNo) { this.systemUserNo = systemUserNo; }
    public LocalDateTime getSystemUsedAt() { return systemUsedAt; }
    public void setSystemUsedAt(LocalDateTime systemUsedAt) { this.systemUsedAt = systemUsedAt; }

-    public String getDocType() { return docType; }
-    public void setDocType(String docType) { this.docType = docType; }
-    public String getParseStatus() { return parseStatus; }
-    public void setParseStatus(String parseStatus) { this.parseStatus = parseStatus; }
-    public String getParseMessage() { return parseMessage; }
-    public void setParseMessage(String parseMessage) { this.parseMessage = parseMessage; }
-    public String getOutputPath() { return outputPath; }
-    public void setOutputPath(String outputPath) { this.outputPath = outputPath; }
-    public Integer getPageCount() { return pageCount; }
-    public void setPageCount(Integer pageCount) { this.pageCount = pageCount; }
-    public LocalDateTime getParsedAt() { return parsedAt; }
-    public void setParsedAt(LocalDateTime parsedAt) { this.parsedAt = parsedAt; }
-    public String getDocDate() { return docDate; }
-    public void setDocDate(String docDate) { this.docDate = docDate; }

}

```

수정 mapper/UploadedFileMapper.java

```

@@ -45,20 +45,27 @@ public interface UploadedFileMapper {

    List<UploadedFile> search(@Param("institution") String institution,
                            @Param("year") String year,
                            @Param("keyword") String keyword);

-    // — 2026-09-23 파싱 전환 —————
-    /** 파싱 결과 기록. JPA 의 더티체킹 대신 쓰는 UPDATE 다. */
+    // — 2026-09-23 파싱 전환 (스키마 무변경안) —————
+    // ⚠ 컬럼을 늘리지 않았다. 문서종류·파싱 성공 여부는 XML 이 **파일명 확장자**로 판별한다.
+    // 판별 규칙 정보는 code/DocumentType 이고 XML 의 sql 조각이 그 SQL 판이다.

```

```

+
+ /** 파싱 결과 기록. 저장경로내용(산출물/원본)·분류일시(파싱시각)·문서년에 태운다. */
+
+ void updateParseResult(UploadedFile file);
- long countByParseStatus(@Param("parseStatus") String parseStatus);
+ long countParseSuccess();
+
+ long countParseFailed();
+
+ long countProposalDocs();
- long countByDocType(@Param("docType") String docType);
+ long countRfpDocs();

+ /** 파싱은 성공했는데 기관을 못 찾은 건(기관분류 NULL). */
+ long countParsedUnclassified();
- /** 파싱 현황 화면. 조건은 모두 선택이며 null 이면 걸지 않는다. */
+ /** 파싱 현황 화면. docType 은 "BID_PROPOSAL"·"RFP", parseStatus 는 "SUCCESS"·"FAILED". */

+ List<UploadedFile> searchParseStatus(@Param("docType") String docType,
+                                     @Param("parseStatus") String parseStatus,
+                                     @Param("unclassifiedOnly") Boolean unclassifiedOnly);

```

수정 service/DocumentParseService.java

```

@@ -79,19 +79,22 @@ public class DocumentParseService {

+ /** 파싱하고 결과(성공/실패)를 DB 에 기록한다. 예외를 밖으로 던지지 않는다. */
+ public UploadedFile parse(UploadedFile file) {
+     try {
-         if (DocumentType.BID_PROPOSAL.equals(file.getDocType())) {
+         String docType = file.getDocType(); // 원본 확장자로 판별한다(스키마 무변경안)
+         if (DocumentType.BID_PROPOSAL.equals(docType)) {
+             parseProposal(file);
-         } else if (DocumentType.RFP.equals(file.getDocType())) {
+         } else if (DocumentType.RFP.equals(docType)) {
+             parseRfp(file);
+         } else {
+             throw new IllegalStateException("문서 종류가 없는 이전 방식 업로드입니다");
+         }
+     } catch (Exception e) {
-         log.warn("파싱 실패: {}", file.getOriginalName(), e);
-         String msg = e.getMessage() != null ? e.getMessage() : e.getClass().getSimpleName();
-         file.markParseFailed("파싱 실패: " + msg);
+         // ⚠ 실패 사유를 담은 컬럼이 없다(스키마 무변경안). 로그가 유일한 기록이므로

```

```

+ // 스택까지 남긴다 - 화면에는 "실패" 표시만 나간다.
+ log.warn("파싱 실패: {} - {}", file.getOriginalName(),
+ e.getMessage() != null ? e.getMessage() : e.getClass().getSimpleName(), e);
+ file.markParseFailed(originalPath(file));

    }
    stampAudit(file);
    fileMapper.updateParseResult(file);
    return file;
@@ -102,18 +105,18 @@ public class DocumentParseService {

    Optional<UploadedFile> found = fileMapper.findById(id);
    if (!found.isPresent()) return Optional.empty();
    UploadedFile file = found.get();
    try {
- storageService.deleteIfExists(file.getOutputPath());
+ storageService.deleteIfExists(file.getOutputPath()); // 성공 건만 값이 있다

    } catch (Exception e) {
        log.warn("이전 산출물 삭제 실패: {}", file.getOutputPath(), e);
    }
    return Optional.of(parse(file));
}

private void parseProposal(UploadedFile file) throws Exception {
    LocalDateTime now = LocalDateTime.now();
- List<SlideContent> slides = pptExtractor.extract(Paths.get(file.getStoredPath()));
+ List<SlideContent> slides = pptExtractor.extract(Paths.get(originalPath(file)));

    if (slides.isEmpty()) throw new IllegalStateException("슬라이드가 없습니다");

    StringBuilder all = new StringBuilder();
    for (SlideContent s : slides) {
@@ -137,14 +140,15 @@ public class DocumentParseService {

        + "_bid_proposal.json";
        Path out = storageService.writeOutput(storageService.getProposalJsonDir(), fileName,
            mapper.writeValueAsBytes(doc));

- file.markParsed(inst.getName(), inst.getCategory(), year, out.toString(), slides.size(), join(notes));
+ logNotes(file, notes);
+ file.markParsed(inst.getName(), inst.getCategory(), year, out.toString(), now);

    }

private void parseRfp(UploadedFile file) throws Exception {
    LocalDateTime now = LocalDateTime.now();
- ExtractedDocument doc = rfpExtractor.extract(Paths.get(file.getStoredPath()));
+ ExtractedDocument doc = rfpExtractor.extract(Paths.get(originalPath(file)));

    List<String> lines = doc.lines();

    InstitutionMatcher.Match inst = institutionMatcher.match(doc.fullText(), institutions());
    List<String> notes = new ArrayList<String>();
@@ -160,10 +164,10 @@ public class DocumentParseService {

        inst.getCategory(), inst.getName(), now.format(DAY)) + ".md";
        Path out = storageService.writeOutput(storageService.getRfpMdDir(), fileName,
            markdown.getBytes(StandardCharsets.UTF_8));

- file.markParsed(inst.getName(), inst.getCategory(), notice.orElse(null),

```

```

-         out.toString(), null, join(notes));
+         logNotes(file, notes);
+         file.markParsed(inst.getName(), inst.getCategory(), notice.orElse(null), out.toString(), now);
    }

    private List<Institution> institutions() {
        return institutionMapper.findAll();
@@ -185,8 +189,20 @@ public class DocumentParseService {

        notes.add("기관 테이블에 없는 기관(" + inst.getName() + ") → 미분류");
    }
}

-     private static String join(List<String> notes) {
-
-         return notes.isEmpty() ? null : String.join(" / ", notes);
+
+         /** 참고 사항(기관 이상 등)도 컬럼이 없어 로그에만 남긴다. */
+         private static void logNotes(UploadedFile file, List<String> notes) {
+
+             if (!notes.isEmpty()) {
+
+                 log.info("파싱 참고: {} - {}", file.getOriginalName(), String.join(" / ", notes));
+
+             }
+
+         }
+
+         /**
+
+          * 원본 경로를 재구성한다. 파싱에 성공하면 저장경로가 산출물로 바뀌므로,
+
+          * 재파싱할 때 원본을 다시 찾으려면 {@code userdata + 원본파일명} 으로 계산해야 한다.
+
+          * ⚠ 그래서 업로드 때 <b>실제로 저장된 파일명</b>을 원본파일명에 넣는다(FileUploadService).
+
+          */
+
+         private String originalPath(UploadedFile file) {
+
+             return storageService.getUserdataDir().resolve(file.getOriginalName()).toString();
+
+         }
+
+     }
}

```

수정 service/FileUploadService.java

```

@@ -58,18 +58,22 @@ public class FileUploadService {

        continue;
    }
    try {
        Path saved = storageService.saveOriginal(file, originalName);
-         UploadedFile entity = new UploadedFile(docType, originalName, saved.toString());
+
+         // ⚠ 중복 이름이면 _yyyyMMddHHmmss 가 붙는다. **실제로 저장된 이름**을 넣어라
+
+         // 나중에 userdata + 원본파일명 으로 원본을 다시 찾을 수 있다(스키마 무변경안).
+
+         String savedName = saved.getFileName().toString();

```

```

+         UploadedFile entity = new UploadedFile(savedName, saved.toString());

        entity.setSystemUserNo(SystemUser.get());
        entity.setSystemUsedAt(entity.getUploadedAt());
        fileMapper.insert(entity);

        UploadedFile parsed = parseService.parse(entity);
-         boolean ok = "SUCCESS".equals(parsed.getParseStatus());

+         boolean ok = parsed.isParsed();

        results.add(new UploadResultItem(originalName, ok,
            ok ? parsed.getCategoryLabel() : null,
-            ok ? "파싱 완료 → " + parsed.getOutputFileName() : parsed.getParseMessage()));

+            ok ? "파싱 완료 → " + parsed.getOutputFileName()

+            : "파싱 실패 - 사유는 서버 로그를 확인하세요"));

    } catch (Exception e) {
        log.error("업로드 처리 실패: {}", originalName, e);
        results.add(new UploadResultItem(originalName, false, null, "오류: " + e.getMessage()));
    }

```

수정 mapper/UploadedFileMapper.xml

```

@@ -55,23 +55,8 @@

    -->
    <result property="systemUsedAt"    column="시스템사용일시"
        typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
    <result property="systemUserNo"    column="시스템사용자번호"/>
-    <!--

-    — 2026-09-23 파싱 전환 7열 —

-    ⚠ 위 감사 2열과 똑같은 함정이 있다. 여기 빠뜨리면 DB 에는 값이 들어가는데 조회하면
-    늘 null 이고, 읽어서 그대로 update 하는 경로에서 그 null 이 DB 로 되써진다.

-    -->

-    <result property="docType"        column="문서종류구분"

-        typeHandler="com.kb.uploader.config.DocumentTypeTypeHandler"/>

-    <result property="parseStatus"    column="파싱상태구분"

-        typeHandler="com.kb.uploader.config.ParseStatusTypeHandler"/>

-    <result property="parseMessage"   column="파싱메시지내용"/>

-    <result property="outputPath"     column="파싱산출물경로내용"/>

-    <result property="parsedAt"       column="파싱일시"

-        typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>

-    <result property="pageCount"      column="문서페이지수"/>

-    <result property="docDate"       column="문서일자"/>

    </resultMap>

    <!-- 파라미터 쪽 TypeHandler 표기가 길어 sql 조각으로 묶는다 -->
    <sql id="값목록">
@@ -96,19 +81,8 @@

```

```

        <otherwise>#{category,typeHandler=com.kb.uploader.config.InstitutionCategoryTypeHandler}</otherwise>
    </choose>
</sql>

```

- <!-- 파싱 7열의 값. insert 와 updateParseResult 가 같이 쓴다 -->

- <sql id="파싱값목록">

- #{docType,typeHandler=com.kb.uploader.config.DocumentTypeTypeHandler},

- #{parseStatus,typeHandler=com.kb.uploader.config.ParseStatusTypeHandler},

- #{parseMessage},

- #{outputPath},

- #{parsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},

- #{pageCount},

- #{docDate}

- </sql>

-

<!--

⚠ 같은 id 가 두 번 나오는 자리는 **방언 분기**다(databaseId). 접속한 DB 에 맞는
 쪽만 로드된다 - config/MyBatisConfig 가 값을 정한다.
 한쪽만 고치면 다른 DB 에서 조용히 낡는다. **항상 둘을 같이 고칠 것.**

@@ -117,28 +91,24 @@

<!-- MySQL: useGeneratedKeys로 AUTO_INCREMENT id 채움 -->

<insert id="insert" databaseId="mysql" useGeneratedKeys="true" keyProperty="id">

INSERT INTO TSKGIAF01

(기관명, 기관영업분류구분, 문서년, 분류상대구분, 원본파일명, 업로드일시, 저장경로내용,

- 시스템사용일시, 시스템사용자번호,

- 문서종류구분, 파싱상태구분, 파싱메시지내용, 파싱산출물경로내용, 파싱일시, 문서페이지수, 문서일자)

+ 시스템사용일시, 시스템사용자번호)

VALUES

(<include refid="값목록"/>,

- #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler}, #{systemUserNo},

- <include refid="파싱값목록"/>)

+ #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler}, #{systemUserNo})

</insert>

<!-- Oracle: 시퀀스에서 먼저 ID를 받아 직접 넣는다 (schema-oracle.sql 의 SQ_KGI_AGENTFILETABLE) -->

<insert id="insert" databaseId="oracle">

<selectKey keyProperty="id" resultType="long" order="BEFORE">

SELECT SQ_KGI_AGENTFILETABLE.NEXTVAL FROM DUAL

</selectKey>

INSERT INTO TSKGIAF01

(업로드파일일련번호, 기관명, 기관영업분류구분, 문서년, 분류상대구분, 원본파일명, 업로드일시,

- 저장경로내용, 시스템사용일시, 시스템사용자번호,

- 문서종류구분, 파싱상태구분, 파싱메시지내용, 파싱산출물경로내용, 파싱일시, 문서페이지수, 문서일자)

+ 저장경로내용, 시스템사용일시, 시스템사용자번호)

VALUES

(#{id}, <include refid="값목록"/>,

- #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler}, #{systemUserNo},

```

-      <include refid="파싱값목록"/>)
+      #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler}, #{systemUserNo})

</insert>

<update id="update">
    UPDATE TSKGIAF01
@@ -154,143 +124,27 @@

    WHERE 업로드파일일련번호 = #{id}
</update>

<!--
-    파싱 결과 기록 (2026-09-23 신설).

-    ⚠ JPA 였다면 더티체킹이 해 주던 UPDATE 다. 파싱 7열과 함께 **기관·분류상태도** 쓴다

-    - 파싱하면서 기관을 찾아 분류가 확정되기 때문이다.

+    파싱 결과 기록 (2026-09-23 신설, 스키마 무변경안).

+    ⚠ 새 컬럼이 없으므로 **기존 컬럼에 태운다**:
```

```

+        저장경로내용 = 산출물 경로(성공) 또는 원본 경로(실패) · 분류일시 = 파싱 시각 ·

+        문서년 = 제안서 연도, 실패 사유는 넣을 자리가 없어 로그에만 남는다.

-->
<update id="updateParseResult">
    UPDATE TSKGIAF01
    SET
-    문서종류구분      = #{docType,typeHandler=com.kb.uploader.config.DocumentTypeTypeHandler},
-    파싱상태구분      = #{parseStatus,typeHandler=com.kb.uploader.config.ParseStatusTypeHandler},
-    파싱메시지내용    = #{parseMessage},
-    파싱산출물경로내용 = #{outputPath},
-    파싱일시          = #{parsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
-    문서페이지수      = #{pageCount},
-    문서일자          = #{docDate},
+    저장경로내용      = #{storedPath},
+    기관명            = #{institutionName},
+    문서년            = #{year},

    기관영업분류구분 = <include refid="기관분류값"/>,
    분류상태구분      = #{status,typeHandler=com.kb.uploader.config.ClassificationStatusTypeHandler},
    분류일시          = #{classifiedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
    시스템사용일시    = #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
    시스템사용자번호 = #{systemUserNo}
    WHERE 업로드파일일련번호 = #{id}
</update>

- <!-- — 파싱 현황·대시보드 집계 (2026-09-23 신설) —————

-    ⚠ SQL 리터럴은 TypeHandler 를 타지 않는다. '01'..'02' 처럼 **코드**로 적을 것.

-    -->

-    <select id="countByParseStatus" resultType="long">

-    SELECT COUNT(*) FROM TSKGIAF01

```

```
- WHERE 파싱상태구분 = #{parseStatus,typeHandler=com.kb.uploader.config.ParseStatusTypeHandler}
- </select>
```

```
- <select id="countByDocType" resultType="long">
```

```
- SELECT COUNT(*) FROM TSKGIAF01
```

```
- WHERE 문서종류구분 = #{docType,typeHandler=com.kb.uploader.config.DocumentTypeTypeHandler}
```

```
- </select>
```

```
- <!-- 파싱은 됐는데 기관을 못 찾은 건. 기관분류가 NULL 인 행이 그것이다 -->
```

```
- <select id="countParsedUnclassified" resultType="long">
```

```
- SELECT COUNT(*) FROM TSKGIAF01
```

```
- WHERE 파싱상태구분 = '02' AND 기관영업분류구분 IS NULL
```

```
- </select>
```

```
- <select id="searchParseStatus" resultMap="uploadedFileMap">
```

```
- SELECT * FROM TSKGIAF01
```

```
- WHERE 분류상태구분 < & '04'
```

```
- <if test="docType != null">
```

```
- AND 문서종류구분 = #{docType,typeHandler=com.kb.uploader.config.DocumentTypeTypeHandler}
```

```
- </if>
```

```
- <if test="parseStatus != null">
```

```
- AND 파싱상태구분 = #{parseStatus,typeHandler=com.kb.uploader.config.ParseStatusTypeHandler}
```

```
- </if>
```

```
- <if test="unclassifiedOnly != null and unclassifiedOnly">
```

```
- AND 기관영업분류구분 IS NULL
```

```
- </if>
```

```
- ORDER BY 업로드파일일련번호 DESC
```

```
- </select>
```

```
- <!-- — 조회 API (2026-09-23 신설). 제안서 '01' / RFP '02', 파싱 성공 '02' 만 — -->
```

```
- <sql id="제안서조건">
```

```
- WHERE 문서종류구분 = '01' AND 파싱상태구분 = '02'
```

```
- <if test="institution != null">
```

```
- AND 기관명 LIKE '%' || #{institution} || '%'
```

```
- </if>
```

```

- <if test="docDate != null">
-     AND 문서일자 LIKE #{docDate} || '%'
- </if>
- </sql>
- <sql id="제안서조건mysql">
-     WHERE 문서종류구분 = '01' AND 파싱상태구분 = '02'
-     <if test="institution != null">
-         AND 기관명 LIKE CONCAT('%', #{institution}, '%')
-     </if>
-     <if test="docDate != null">
-         AND 문서일자 LIKE CONCAT("#{docDate}, '%')
-     </if>
- </sql>
- <sql id="RFP조건">
-     WHERE 문서종류구분 = '02' AND 파싱상태구분 = '02'
-     <if test="institution != null">
-         AND 기관명 LIKE '%' || #{institution} || '%'
-     </if>
-     <if test="docDate != null">
-         AND 문서일자 LIKE #{docDate} || '%'
-     </if>
- </sql>
- <sql id="RFP조건mysql">
-     WHERE 문서종류구분 = '02' AND 파싱상태구분 = '02'
-     <if test="institution != null">
-         AND 기관명 LIKE CONCAT('%', #{institution}, '%')
-     </if>
-     <if test="docDate != null">
-         AND 문서일자 LIKE CONCAT("#{docDate}, '%')
-     </if>
- </sql>
-
- <select id="searchProposals" databaseId="mysql" resultMap="uploadedFileMap">
-     SELECT * FROM TSKGIAF01 <include refid="제안서조건mysql"/>
-
-     ORDER BY 업로드파일일련번호 DESC

```

```

-      LIMIT #{limit} OFFSET #{offset}
-    </select>
-
-    <select id="searchProposals" databaseId="oracle" resultMap="uploadedFileMap">
-      SELECT * FROM (
-
-        SELECT t.*, ROWNUM rnum FROM (
-
-          SELECT * FROM TSKGIAF01 <include refid="제안서조건"/>
-
-          ORDER BY 업로드파일일련번호 DESC
-
-        ) t WHERE ROWNUM <= #{offset} + #{limit}
-
-        ) WHERE rnum > #{offset}
-    </select>
-
-    <select id="countProposals" databaseId="mysql" resultType="long">
-      SELECT COUNT(*) FROM TSKGIAF01 <include refid="제안서조건mysql"/>
-    </select>
-
-    <select id="countProposals" databaseId="oracle" resultType="long">
-      SELECT COUNT(*) FROM TSKGIAF01 <include refid="제안서조건"/>
-    </select>
-
-
-    <select id="searchRfps" databaseId="mysql" resultMap="uploadedFileMap">
-      SELECT * FROM TSKGIAF01 <include refid="RFP조건mysql"/>
-
-      ORDER BY 업로드파일일련번호 DESC
-
-      LIMIT #{limit} OFFSET #{offset}
-    </select>
-
-    <select id="searchRfps" databaseId="oracle" resultMap="uploadedFileMap">
-      SELECT * FROM (
-
-        SELECT t.*, ROWNUM rnum FROM (
-
-          SELECT * FROM TSKGIAF01 <include refid="RFP조건"/>
-
-          ORDER BY 업로드파일일련번호 DESC
-
-        ) t WHERE ROWNUM <= #{offset} + #{limit}
-
-        ) WHERE rnum > #{offset}
-    </select>
-
-    <select id="countRfps" databaseId="mysql" resultType="long">
-      SELECT COUNT(*) FROM TSKGIAF01 <include refid="RFP조건mysql"/>
-    </select>
-
-    <select id="countRfps" databaseId="oracle" resultType="long">
-      SELECT COUNT(*) FROM TSKGIAF01 <include refid="RFP조건"/>

```

- </select>

<!--
⚠ '04' = DELETED, '03' = REJECTED, '02' = CLASSIFIED (code/ClassificationStatus.java)
⚠ 이 둘은 파라미터에 시각 값이 없어 **컨트롤러가 LocalDateTime.now() 를 넘긴다.**
DB 함수(TO_CHAR(SYSDATE)/DATE_FORMAT(NOW()))를 쓰면 방언이 갈리고 테스트로

@@ -349,8 +203,9 @@

<!-- MySQL: LIMIT -->
<select id="findRecent" databaseId="mysql" resultMap="uploadedFileMap">
SELECT * FROM TSKGIAF01 ORDER BY 업로드파일일련번호 DESC LIMIT #{limit}
</select>

+

<!-- Oracle: ROWNUM 은 ORDER BY 보다 먼저 매겨지므로 정렬을 인라인 뷰로 감싼다 -->
<select id="findRecent" databaseId="oracle" resultMap="uploadedFileMap">
SELECT * FROM (
SELECT * FROM TSKGIAF01 ORDER BY 업로드파일일련번호 DESC

@@ -371,8 +226,9 @@

WHERE 기관명 LIKE CONCAT('%', #{keyword}, '%')
ORDER BY 업로드일시 DESC
LIMIT #{limit} OFFSET #{offset}
</select>

+

<!-- Oracle: ROWNUM 2단 중첩 페이징. 안쪽에서 상한, 바깥에서 하한을 건다 -->
<select id="findByInstitutionNameContaining" databaseId="oracle" resultMap="uploadedFileMap">
SELECT * FROM (
SELECT a.*, ROWNUM RN FROM (
SELECT a.*, ROWNUM RN FROM (
SELECT * FROM TSKGIAF01 ORDER BY 업로드파일일련번호 DESC

@@ -387,8 +243,9 @@

<select id="countByInstitutionNameContaining" databaseId="mysql" resultType="long">
SELECT COUNT(*) FROM TSKGIAF01
WHERE 기관명 LIKE CONCAT('%', #{keyword}, '%')
</select>

+

<!-- Oracle: || 로 연결 -->
<select id="countByInstitutionNameContaining" databaseId="oracle" resultType="long">
SELECT COUNT(*) FROM TSKGIAF01
WHERE 기관명 LIKE '%' || #{keyword} || '%'

@@ -402,8 +259,9 @@

AND (#{year} = '' OR 문서년 = #{year})
AND (#{keyword} = '' OR 원본파일명 LIKE CONCAT('%', #{keyword}, '%'))
ORDER BY 업로드일시 DESC
</select>

+

<!--
Oracle: || 로 연결.
⚠ Oracle 은 빈 문자열을 NULL 로 바꾸므로 `#{x} = ''` 는 **항상 거짓**이다.
"빈 값이면 조건을 걸지 않는다"는 원래 뜻을 살리려면 NULL 검사여야 한다.

@@ -417,5 +275,129 @@

AND (#{keyword} IS NULL OR 원본파일명 LIKE '%' || #{keyword} || '%')
ORDER BY 업로드일시 DESC
</select>

+ <!-- == 파싱 현황·대시보드·조회 API (2026-09-23, 스키마 무변경안) ==

+ ⚠ 컬럼을 늘리지 않았으므로 **파일명 규칙으로 판별**한다. 규칙 정보는

+ code/DocumentType (ofFileName · isOutputPath) 이고, 아래 조각이 그 SQL 판이다.

+ 한쪽을 고치면 반드시 다른 쪽도 고칠 것 - 어긋나면 화면 숫자와 목록이 따로 논다.

+ · 파싱 성공 = 저장경로내용이 .json 또는 .md (= 산출물이 만들어졌다)

+ · 파싱 실패 = 그 밖 (저장경로가 원본 그대로다)

+ · 제안서 = 원본파일명이 .ppt / .pptx

+ · RFP = 원본파일명이 .pdf / .hwp / .hwpX

+ ⚠ 삭제(04)·반려(03) 건은 파싱 집계에서 뺀다. 사람이 처리를 끝낸 행이다.

+ -->

+ <sql id="파싱성공">(저장경로내용 LIKE '%.json' OR 저장경로내용 LIKE '%.md')</sql>

+ <sql id="제안서원본">(원본파일명 LIKE '%.ppt' OR 원본파일명 LIKE '%.pptx')</sql>

+ <sql id="RFP원본">(원본파일명 LIKE '%.pdf' OR 원본파일명 LIKE '%.hwp' OR 원본파일명 LIKE '%.hwpX')</sql>

+ <sql id="파싱대상">(include refid="제안서원본"/> OR <include refid="RFP원본"/>)</sql>

+ <sql id="처리중">분류상태구분 NOT IN ('03', '04')</sql>

+

+ <select id="countParseSuccess" resultType="long">

+ SELECT COUNT(*) FROM TSKGIAF01

+ WHERE <include refid="처리중"/> AND <include refid="파싱성공"/>

+ </select>

+

+ <select id="countParseFailed" resultType="long">

+ SELECT COUNT(*) FROM TSKGIAF01

+ WHERE <include refid="처리중"/> AND <include refid="파싱대상"/> AND NOT <include refid="파싱성공"/>

+ </select>

+

+ <select id="countProposalDocs" resultType="long">

+ SELECT COUNT(*) FROM TSKGIAF01 WHERE <include refid="처리중"/> AND <include refid="제안서원본"/>

+ </select>

+

+ <select id="countRfpDocs" resultType="long">

+ SELECT COUNT(*) FROM TSKGIAF01 WHERE <include refid="처리중"/> AND <include refid="RFP원본"/>

+ </select>

+

+ <!-- 파싱은 됐는데 기관을 못 찾은 건. 기관분류 NULL 이 곧 "미분류" 다 -->

+ <select id="countParsedUnclassified" resultType="long">

+ SELECT COUNT(*) FROM TSKGIAF01

+ WHERE <include refid="처리중"/> AND <include refid="파싱성공"/> AND 기관영업분류구분 IS NULL

```

+ </select>
+
+ <select id="searchParseStatus" resultMap="uploadedFileMap">
+
+   SELECT * FROM TSKGIAF01
+
+   WHERE <include refid="처리중"/>
+
+   <if test="docType == 'BID_PROPOSAL'">AND <include refid="제안서원본"/></if>
+
+   <if test="docType == 'RFP'">AND <include refid="RFP원본"/></if>
+
+   <if test="parseStatus == 'SUCCESS'">AND <include refid="파싱성공"/></if>
+
+   <if test="parseStatus == 'FAILED'">AND <include refid="파싱대상"/> AND NOT <include refid="파싱성공"/></if>
+
+   <if test="unclassifiedOnly != null and unclassifiedOnly">AND 기관영업분류구분 IS NULL</if>
+
+   ORDER BY 업로드파일일련번호 DESC
+
+ </select>
+
+
+ <!-- — 조회 API. 파싱 성공한 것만 준다 —————>
+
+   ⚠ 날짜 조건은 **산출물 파일명 앞 토큰**을 본다(경로에 파일명이 들어 있다).
+
+   `%20240315\__%` 처럼 밑줄을 escape 해야 한다 - LIKE 에서 _ 는 한 글자 와일드카드다.
+
+ -->
+
+ <sql id="날짜조건mysql">
+
+   <if test="docDate != null">AND 저장경로내용 LIKE CONCAT('%', #{docDate}, '\_') ESCAPE '\\'\</if>
+
+ </sql>
+
+ <sql id="날짜조건oracle">
+
+   <if test="docDate != null">AND 저장경로내용 LIKE '%' || #{docDate} || '\_%' ESCAPE '\\'\</if>
+
+ </sql>
+
+ <sql id="기관조건mysql">
+
+   <if test="institution != null">AND 기관명 LIKE CONCAT('%', #{institution}, '%')</if>
+
+ </sql>
+
+ <sql id="기관조건oracle">
+
+   <if test="institution != null">AND 기관명 LIKE '%' || #{institution} || '%'\</if>
+
+ </sql>
+
+
+ <select id="searchProposals" databaseId="mysql" resultMap="uploadedFileMap">
+
+   SELECT * FROM TSKGIAF01
+
+   WHERE <include refid="처리중"/> AND <include refid="제안서원본"/> AND <include refid="파싱성공"/>
+
+   <include refid="기관조건mysql"/><include refid="날짜조건mysql"/>
+
+   ORDER BY 업로드파일일련번호 DESC

```

```

+      LIMIT #{limit} OFFSET #{offset}
+    </select>
+    <select id="searchProposals" databaseId="oracle" resultMap="uploadedFileMap">
+      SELECT * FROM (
+        SELECT t.*, ROWNUM rnum FROM (
+          SELECT * FROM TSKGIAF01
+          WHERE <include refid="처리중"/> AND <include refid="제안서원본"/> AND <include refid="파싱성공"/>
+          <include refid="기관조건oracle"/><include refid="날짜조건oracle"/>
+          ORDER BY 업로드파일일련번호 DESC
+        ) t WHERE ROWNUM <math>\leq \#{offset} + \#{limit}</math>
+      ) WHERE rnum > #{offset}
+    </select>
+    <select id="countProposals" databaseId="mysql" resultType="long">
+      SELECT COUNT(*) FROM TSKGIAF01
+      WHERE <include refid="처리중"/> AND <include refid="제안서원본"/> AND <include refid="파싱성공"/>
+      <include refid="기관조건mysql"/><include refid="날짜조건mysql"/>
+    </select>
+    <select id="countProposals" databaseId="oracle" resultType="long">
+      SELECT COUNT(*) FROM TSKGIAF01
+      WHERE <include refid="처리중"/> AND <include refid="제안서원본"/> AND <include refid="파싱성공"/>
+      <include refid="기관조건oracle"/><include refid="날짜조건oracle"/>
+    </select>
+
+    <select id="searchRfps" databaseId="mysql" resultMap="uploadedFileMap">
+      SELECT * FROM TSKGIAF01
+      WHERE <include refid="처리중"/> AND <include refid="RFP원본"/> AND <include refid="파싱성공"/>
+      <include refid="기관조건mysql"/><include refid="날짜조건mysql"/>
+      ORDER BY 업로드파일일련번호 DESC
+      LIMIT #{limit} OFFSET #{offset}
+    </select>
+    <select id="searchRfps" databaseId="oracle" resultMap="uploadedFileMap">
+      SELECT * FROM (
+        SELECT t.*, ROWNUM rnum FROM (
+          SELECT * FROM TSKGIAF01
+          WHERE <include refid="처리중"/> AND <include refid="RFP원본"/> AND <include refid="파싱성공"/>

```

```

+      <include refid="기관조건oracle"/><include refid="날짜조건oracle"/>
+      ORDER BY 업로드파일일련번호 DESC
+    ) t WHERE ROWNUM &lt;= #{offset} + #{limit}
+  ) WHERE rnum &gt; #{offset}
+ </select>
+ <select id="countRfps" databaseId="mysql" resultType="long">
+   SELECT COUNT(*) FROM TSKGIAF01
+   WHERE <include refid="처리중"/> AND <include refid="RFP원본"/> AND <include refid="파싱성공"/>
+   <include refid="기관조건mysql"/><include refid="날짜조건mysql"/>
+ </select>
+ <select id="countRfps" databaseId="oracle" resultType="long">
+   SELECT COUNT(*) FROM TSKGIAF01
+   WHERE <include refid="처리중"/> AND <include refid="RFP원본"/> AND <include refid="파싱성공"/>
+   <include refid="기관조건oracle"/><include refid="날짜조건oracle"/>
+ </select>
+
</mapper>

```

수정 templates/parse-status.html

```

@@ -81,10 +81,12 @@
    <span th:if="${f.parseStatus == 'SUCCESS'}" class="badge badge-done">성공</span>
    <span th:if="${f.parseStatus == 'FAILED'}" class="badge badge-pending">실패</span>
    <span th:if="${f.parseStatus != 'SUCCESS' and f.parseStatus != 'FAILED'}"
      class="badge badge-neutral" th:text="${f.parseStatus != null ? f.parseStatus : '-'}"></span>
-    <div th:if="${f.parseMessage != null}" style="font-size:11.5px;color:var(--text-2);margin-top:3px"
-      th:text="${f.parseMessage}"></div>
+    <!-- ⚠ 실패 사유는 저장하지 않는다(스키마 무변경안). 서버 로그를 봐야 한다. -->
+    <div th:if="${f.parseStatus == 'FAILED'}" style="font-size:11.5px;color:var(--text-2);margin-top:3px">
+      사유는 서버 로그 확인
+    </div>

  </td>
  <td>
    <a th:if="${f.outputPath != null}" th:href="@{/parse-status/{id}/output(id=${f.id})}"
      target="_blank" class="file-name" style="font-size:12.5px"

```

3. 전체 — 처음부터 적용하는 경우

아무것도 적용하지 않은 상태(파싱 전환 이전)에서 **처음부터** 갈 때 쓴다. 파일 53개. 이 경우에도 **DDL** 은 필요 없다.

여기에 없는 것 셋은 반입이 필요하다 — **jar 6종**(poi-scratchpad-hwplib-hwpplib· lucene-analyzers-nori 와 전이 의존 lucene-core·lucene-analyzers-common), **parse 패키지 자바 10개**, **agent-test.js**. 반입 패키지가 tools/uploader_파싱전환_반입_2026-09-23.zip 에 있다.

수정 config-envs/dev/application.properties

```
@@ -12,7 +12,21 @@ upload.base-dir=/app/uploader

mybatis.mapper-locations=classpath:mapper/*.xml
mybatis.type-aliases-package=com.kb.uploader.domain
mybatis.configuration.map-underscore-to-camel-case=true

-reclassification.cron=0 */5 * * * *

+# (2026-09-23 배치 폐지) reclassification.cron=0 */5 * * * *

logging.level.org.springframework.web.filter.CommonsRequestLoggingFilter=DEBUG
+

+# ---- 2026-09-23 파싱 전환: 원본 보관 / 산출물 경로 ----

+# .properties 는 ISO-8859-1 로 읽히므로 한글 경로는 \uXXXX 로 적는다.

+# \uc6d0\ubcf8/\... = 원본/전처리/입찰제안서_PPT

+# \uc804\ucc98\u99ac/RFP/\... = 전처리/RFP/파싱후

+upload.userdata-dir=${upload.base-dir}/userdata

+upload.proposal-json-dir=${upload.base-dir}/\uc6d0\ubcf8/\uc804\ucc98\u99ac/\uc785\ucc30\ucc81c\ucc548\ucc11c_PPT

+upload.rfp-md-dir=${upload.base-dir}/\uc804\ucc98\u99ac/RFP/\uc30c\ucc2f1\ucc6c4

+

+# 한글 품값·파일명 인코딩 고정

+server.servlet.encoding.charset=UTF-8

+server.servlet.encoding.enabled=true

+server.servlet.encoding.force=true

+server.tomcat.uri-encoding=UTF-8
```

수정 config-envs/local/application.properties

```
@@ -12,7 +12,21 @@ upload.base-dir=C:/uploader-local

mybatis.mapper-locations=classpath:mapper/*.xml
mybatis.type-aliases-package=com.kb.uploader.domain
mybatis.configuration.map-underscore-to-camel-case=true

-reclassification.cron=0 */5 * * * *
```

```

+# (2026-09-23 배치 폐지) reclassification.cron=0 */5 * * * *

logging.level.org.springframework.web.filter.CommonsRequestLoggingFilter=DEBUG
+

+# ---- 2026-09-23 파싱 전환: 원본 보관 / 산출물 경로 ----

+# .properties 는 ISO-8859-1 로 읽히므로 한글 경로는 \uXXXX 로 적는다.

+# \uc6d0\ubcf8/\... = 원본/전처리/입찰제안서_PPT

+# \uc804\ucc98\ub9ac/RFP/\... = 전처리/RFP/파싱후

+upload.userdata-dir=${upload.base-dir}/userdata

+upload.proposal-json-dir=${upload.base-dir}/\uc6d0\ubcf8/\uc804\ucc98\ub9ac/\uc785\ucc30\uc81c\uc548\uc11c_PPT

+upload.rfp-md-dir=${upload.base-dir}/\uc804\ucc98\ub9ac/RFP/\ud30c\uc2f1\ud6c4

+

+# 한글 품값·파일명 인코딩 고정

+server.servlet.encoding.charset=UTF-8

+server.servlet.encoding.enabled=true

+server.servlet.encoding.force=true

+server.tomcat.uri-encoding=UTF-8

```

수정 config-envs/out-local/application.properties

```

@@ -14,7 +14,21 @@ upload.base-dir=C:/uploader-out-local

mybatis.mapper-locations=classpath:mapper/*.xml
mybatis.type-aliases-package=com.kb.uploader.domain
mybatis.configuration.map-underscore-to-camel-case=true

-reclassification.cron=0 */5 * * * *

+# (2026-09-23 배치 폐지) reclassification.cron=0 */5 * * * *

logging.level.org.springframework.web.filter.CommonsRequestLoggingFilter=DEBUG
+

+# ---- 2026-09-23 파싱 전환: 원본 보관 / 산출물 경로 ----

+# .properties 는 ISO-8859-1 로 읽히므로 한글 경로는 \uXXXX 로 적는다.

+# \uc6d0\ubcf8/\... = 원본/전처리/입찰제안서_PPT

+# \uc804\ucc98\ub9ac/RFP/\... = 전처리/RFP/파싱후

+upload.userdata-dir=${upload.base-dir}/userdata

+upload.proposal-json-dir=${upload.base-dir}/\uc6d0\ubcf8/\uc804\ucc98\ub9ac/\uc785\ucc30\uc81c\uc548\uc11c_PPT

+upload.rfp-md-dir=${upload.base-dir}/\uc804\ucc98\ub9ac/RFP/\ud30c\uc2f1\ud6c4

+

+# 한글 품값·파일명 인코딩 고정

+server.servlet.encoding.charset=UTF-8

```

```
+server.servlet.encoding.enabled=true
```

```
+server.servlet.encoding.force=true
```

```
+server.tomcat.uri-encoding=UTF-8
```

수정 config-envs/prod/application.properties

```
@@ -5,5 +5,19 @@ upload.base-dir=/app/uploader
```

```
mybatis.mapper-locations=classpath:mapper/*.xml  
mybatis.type-aliases-package=com.kb.uploader.domain  
mybatis.configuration.map-underscore-to-camel-case=true
```

```
-reclassification.cron=0 */5 * * * *
```

```
+# (2026-09-23 배치 폐지) reclassification.cron=0 */5 * * * *
```

```
+
```

```
+# ---- 2026-09-23 파싱 전환: 원본 보관 / 산출물 경로 ----
```

```
+# .properties 는 ISO-8859-1 로 읽히므로 한글 경로는 \uXXXX 로 적는다.
```

```
+# \uc6d0\ubcf8/... = 원본/전처리/입찰제안서_PPT
```

```
+# \uc804\ucc98\uub9ac/RFP/... = 전처리/RFP/파싱후
```

```
+upload.userdata-dir=${upload.base-dir}/userdata
```

```
+upload.proposal-json-dir=${upload.base-dir}/\uc6d0\ubcf8/\uc804\ucc98\uub9ac/\uc785\ucc30\ucc81c\ucc548\ucc11c_PPT
```

```
+upload.rfp-md-dir=${upload.base-dir}/\uc804\ucc98\uub9ac/RFP/\ud30c\uc2f1\udd6c4
```

```
+
```

```
+# 한글 품값·파일명 인코딩 고정
```

```
+server.servlet.encoding.charset=UTF-8
```

```
+server.servlet.encoding.enabled=true
```

```
+server.servlet.encoding.force=true
```

```
+server.tomcat.uri-encoding=UTF-8
```

수정 config-envs/stg/application.properties

```
@@ -12,5 +12,19 @@ upload.base-dir=/app/uploader
```

```
mybatis.mapper-locations=classpath:mapper/*.xml  
mybatis.type-aliases-package=com.kb.uploader.domain  
mybatis.configuration.map-underscore-to-camel-case=true
```

```
-reclassification.cron=0 */5 * * * *
```

```
+# (2026-09-23 배치 폐지) reclassification.cron=0 */5 * * * *
```

```
+
```

```
+# ---- 2026-09-23 파싱 전환: 원본 보관 / 산출물 경로 ----
```

```
+# .properties 는 ISO-8859-1 로 읽히므로 한글 경로는 \uXXXX 로 적는다.
```

```
+# \uc6d0\ubcf8/... = 원본/전처리/입찰제안서_PPT
```

```

## \uc804\ucc98\uub9ac/RFP/... = 전처리/RFP/파싱후

+upload.userdata-dir=${upload.base-dir}/userdata

+upload.proposal-json-dir=${upload.base-dir}/\uc6d0\ubcf8/\uc804\ucc98\uub9ac/\uc785\ucc30\uc81c\uc548\uc11c_PPT

+upload.rfp-md-dir=${upload.base-dir}/\uc804\ucc98\uub9ac/RFP/\ud30c\uc2f1\ud6c4

+

## 한글 품값·파일명 인코딩 고정

+server.servlet.encoding.charset=UTF-8

+server.servlet.encoding.enabled=true

+server.servlet.encoding.force=true

+server.tomcat.uri-encoding=UTF-8

```

수정 pom.xml

```

@@ -66,8 +66,34 @@
        <artifactId>pdfbox</artifactId>
        <version>2.0.29</version>
    </dependency>
    <!-- Apache POI (XLSX) -->
+   <!-- — 2026-09-23 파싱 전환으로 추가 (폐쇄망 반입 시 전이 의존까지 함께 반입할 것) — -->
+
+   <!-- .ppt(97-2003) 슬라이드 추출 -->
+
+   <dependency>
+
+       <groupId>org.apache.poi</groupId>
+
+       <artifactId>poi-scratchpad</artifactId>
+
+       <version>5.2.3</version>
+
+   </dependency>
+
+   <!-- HWP 본문·표 -->
+
+   <dependency>
+
+       <groupId>kr.dogfoot</groupId>
+
+       <artifactId>hwplib</artifactId>
+
+       <version>1.1.11</version>
+
+   </dependency>
+
+   <!-- HWPX 본문·표 -->
+
+   <dependency>
+
+       <groupId>kr.dogfoot</groupId>
+
+       <artifactId>hwpplib</artifactId>
+
+       <version>1.0.9</version>
+
+   </dependency>
+
+   <!-- 한국어 형태소 분석 (TextRank 요약). lucene-core 8.11.4 가 전이 의존으로 따라온다 -->

```

```
+ <dependency>
+   <groupId>org.apache.lucene</groupId>
+   <artifactId>lucene-analyzers-nori</artifactId>
+   <version>8.11.4</version>
+ </dependency>
+
+   <dependency>
+     <groupId>org.apache.poi</groupId>
+     <artifactId>poi-ooxml</artifactId>
+     <version>5.2.3</version>
```

신규 code/DocumentType.java

```
new file mode 100644
@@ -0,0 +1,110 @@
+package com.kb.uploader.code;
+
+import java.util.Arrays;
+import java.util.Collections;
+import java.util.HashSet;
+import java.util.LinkedHashMap;
+import java.util.Map;
+import java.util.Set;
+
+/**
+ * 문서종류 - 사내 인포타임 {@code 구분코드2} 로 저장하는 코드값 (2026-09-23 신설).
+ *
+ * <p>자바가 쓰는 값은 {@code "BID_PROPOSAL"}·{@code "RFP"} 이고, DB 에는
+ * {@code '01'}·{@code '02'} 로 들어간다. 변환은 {@code DocumentTypeHandler} 한 곳뿐이다
+ * - {@link ClassificationStatus} 와 같은 방식이다.
+ *
+ * <p>⚠ <b>Mapper XML 의 SQL 리터럴은 이 변환을 타지 않는다.</b>
+ * {@code WHERE 문서종류구분 = '01'} 처럼 <b>코드</b>로 적어야 한다.
+ * 그 자리에 {@code 'BID_PROPOSAL'} 을 적으면 오류 없이 0건이 나온다.
+ *
+ * <p>확장자 판정과 화면 표시명도 여기 둔다. 문서 종류를 늘릴 때 고칠 곳이 한 군데가 되게 하려는 것이다.
+ */
+public final class DocumentType {
```

```

+
+   public static final String BID_PROPOSAL = "BID_PROPOSAL";
+
+   public static final String RFP           = "RFP";
+
+
+   private static final Map<String, String> NAME_TO_CODE = new LinkedHashMap<String, String>();
+   private static final Map<String, String> CODE_TO_NAME = new LinkedHashMap<String, String>();
+   private static final Map<String, String> LABEL = new LinkedHashMap<String, String>();
+   private static final Map<String, Set<String>> EXTENSIONS = new LinkedHashMap<String, Set<String>>();
+
+
+   static {
+
+       put("01", BID_PROPOSAL, "입찰제안서", "ppt", "pptx");
+
+       put("02", RFP, "입찰공고문(RFP)", "pdf", "hwp", "hwp");
+
+   }
+
+
+   private static void put(String code, String name, String label, String... extensions) {
+
+       NAME_TO_CODE.put(name, code);
+
+       CODE_TO_NAME.put(code, name);
+
+       LABEL.put(name, label);
+
+       EXTENSIONS.put(name, Collections.unmodifiableSet(new HashSet<String>(Arrays.asList(extensions))));
+
+   }
+
+
+   public static Map<String, String> nameToCode() {
+
+       return Collections.unmodifiableMap(NAME_TO_CODE);
+
+   }
+
+
+   public static Map<String, String> codeToName() {
+
+       return Collections.unmodifiableMap(CODE_TO_NAME);
+
+   }
+
+
+   /** 화면 표시명. 모르는 값이면 그대로 돌려준다(옛 행은 문서종류가 null 이다). */
+
+   public static String label(String name) {
+
+       String label = LABEL.get(name);
+
+       return label != null ? label : name;
+
+   }
+
+

```

```

+ public static Set<String> extensions(String name) {
+     Set<String> exts = EXTENSIONS.get(name);
+     return exts != null ? exts : Collections.<String>emptySet();
+ }
+
+ /** 이 종류가 받아들이는 확장자인가. */
+ public static boolean accepts(String name, String filename) {
+     return extensions(name).contains(extensionOf(filename));
+ }
+
+ public static String extensionOf(String filename) {
+     if (filename == null) return "";
+     int dot = filename.lastIndexOf('.');
+     return dot < 0 ? "" : filename.substring(dot + 1).toLowerCase();
+ }
+
+ // — 스키마 무변경안(2026-09-23)의 판별 규칙 —————
+ // 컬럼을 늘리지 않으므로 문서종류·파싱 여부를 **파일명에서** 읽는다.
+ // SQL(UploadedFileMapper.xml)도 같은 규칙을 LIKE 로 표현한다 – 한쪽을 바꾸면 다른 쪽도 바꿀 것.
+
+ /** 원본 확장자로 문서종류를 판별한다. 규칙 밖(md·xlsx 등 옛 업로드)이면 null. */
+ public static String ofFileName(String originalName) {
+     String ext = extensionOf(originalName);
+     for (Map.Entry<String, Set<String>> e : EXTENSIONS.entrySet()) {
+         if (e.getValue().contains(ext)) return e.getKey();
+     }
+     return null;
+ }
+
+ /** 산출물 확장자. 제안서는 JSON, RFP 요약은 Markdown 이다. */
+ public static boolean isOutputPath(String path) {
+     String ext = extensionOf(path);
+     return "json".equals(ext) || "md".equals(ext);
+ }
+
+

```

```

+  /**
+   * 산출물 파일명 앞의 날짜 토큰. {@code 20240315_지자체_...} → {@code 20240315},
+   * {@code 2025_대학교_...} → {@code 2025}. 형식이 아니면 null.
+   */
+   public static String leadingDate(String outputFileName) {
+       if (outputFileName == null) return null;
+       int bar = outputFileName.indexOf('_');
+       if (bar != 4 && bar != 8) return null;
+       String head = outputFileName.substring(0, bar);
+       for (int i = 0; i < head.length(); i++) {
+           if (!Character.isDigit(head.charAt(i))) return null;
+       }
+       return head;
+   }
+
+   private DocumentType() {}
+}

```

신규 code/ParseStatus.java

```

new file mode 100644
@@ -0,0 +1,45 @@
+package com.kb.uploader.code;
+
+import java.util.Collections;
+import java.util.LinkedHashMap;
+import java.util.Map;
+
+/**
+ * 파싱상태 - 사내 인포타임 {@code 구분코드2} 로 저장하는 코드값 (2026-09-23 신설).
+ *
+ * <p>⚠️ {@link ClassificationStatus}(분류상태)와 <b>다른 축</b>이다.
+ * 분류상태는 "기관분류를 확정했는가", 파싱상태는 "문서를 읽어 산출물을 만들었는가" 다.
+ * 파싱은 성공했는데 기관 테이블에 없는 기관이라 미분류로 남는 건이 정상적으로 존재한다.
+ *
+ * <p>⚠️ SQL 리터럴은 변환을 타지 않는다 - {@code WHERE 파싱상태구분 = '03'} 처럼 코드로 적을 것.
+ */

```

```

+public final class ParseStatus {
+
+
+    public static final String PENDING = "PENDING";
+
+    public static final String SUCCESS = "SUCCESS";
+
+    public static final String FAILED = "FAILED";
+
+
+    private static final Map<String, String> NAME_TO_CODE = new LinkedHashMap<String, String>();
+
+    private static final Map<String, String> CODE_TO_NAME = new LinkedHashMap<String, String>();
+
+
+    static {
+
+        put("01", PENDING);
+
+        put("02", SUCCESS);
+
+        put("03", FAILED);
+
+    }
+
+
+    private static void put(String code, String name) {
+
+        NAME_TO_CODE.put(name, code);
+
+        CODE_TO_NAME.put(code, name);
+
+    }
+
+
+    public static Map<String, String> nameToCode() {
+
+        return Collections.unmodifiableMap(NAME_TO_CODE);
+
+    }
+
+
+    public static Map<String, String> codeToName() {
+
+        return Collections.unmodifiableMap(CODE_TO_NAME);
+
+    }
+
+
+    private ParseStatus() {}
+
+}

```

신규 config/DocumentTypeHandler.java

new file mode 100644

@@ -0,0 +1,19 @@

```

+package com.kb.uploader.config;

```

```

+
+import com.kb.uploader.code.DocumentType;
+
+import java.util.Map;
+
+/** {@code BID_PROPOSAL} ↔ {@code '01'} - 자세한 이유는 {@link CodeTypeHandler}. */
+public class DocumentTypeHandler extends CodeTypeHandler {
+
+    @Override
+    protected Map<String, String> nameToCode() {
+        return DocumentType.nameToCode();
+    }
+
+    @Override
+    protected Map<String, String> codeToName() {
+        return DocumentType.codeToName();
+    }
+}

```

수정 config/MyBatisConfig.java

```

@@ -1,7 +1,9 @@
package com.kb.uploader.config;

import org.apache.ibatis.mapping.DatabaseIdProvider;
+import org.apache.ibatis.type.JdbcType;
+import org.mybatis.spring.boot.autoconfigure.ConfigurationCustomizer;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import javax.sql.DataSource;
@@ -56,8 +58,33 @@ public class MyBatisConfig {
    }
    };
}

+ /**
+  * {@code null} 파라미터를 바인딩할 때 쓸 JDBC 타입을 {@code NULL} 로 고정한다.
+  *
+  * <p><b>왜 두는가.</b> MyBatis 기본값은 {@code OTHER}{@code java.sql.Types.OTHER}, 1111)
+  * 인데 <b>Oracle 드라이버가 이 값을 거부한다</b> - {@code ORA: 부적합한 열 유형: 1111}.
+  * 업로드 직후의 {@code 기관영업분류구분}·{@code 문서년}·{@code 분류일시} 처럼

```

```

+ * <b>분류 전에는 비어 있는 게 정상인 컬럼</b>이 전부 여기에 걸린다.
+ *
+ * <p>문장마다 {@code jdbcType=} 을 붙이는 방법도 있지만, 새 nullable 컬럼이 생길 때마다
+ * 빠뜨리면 그 문장에서만 터진다. 전역으로 한 번 고정하는 편이 안전하다.
+ *
+ * <p>⚠ <b>테스트로는 못 잡는다.</b> H2/MySQL 은 {@code OTHER} 를 받아 주므로
+ * 위 {@code resolve} 주석의 "런타임 테스트는 oracle 분기를 실행하지 않는다"는 공백에
+ * 그대로 해당한다 - 실제로 내부망 WebLogic+Oracle 에서 처음 드러났다.
+ */
+ @Bean
+ public ConfigurationCustomizer jdbcTypeForNullCustomizer() {
+     return new ConfigurationCustomizer() {
+         @Override
+         public void customize(org.apache.ibatis.session.Configuration configuration) {
+             configuration.setJdbcTypeForNull(JdbcType.NULL);
+         }
+     };
+ }
+
+ /**
+  * @param product {@code DatabaseMetaData.getDatabaseProductName()}
+  * @throws IllegalStateException 모르는 벤더
+  */

```

신규 config/ParseStatusTypeHandler.java

```

new file mode 100644
@@ -0,0 +1,19 @@
+package com.kb.uploader.config;
+
+import com.kb.uploader.code.ParseStatus;
+
+import java.util.Map;
+
+/** {@code SUCCESS} ⇄ {@code '02'} - 자세한 이유는 {@link CodeTypeHandler}. */
+public class ParseStatusTypeHandler extends CodeTypeHandler {
+
+    @Override

```

```

+   protected Map<String, String> nameToCode() {
+       return ParseStatus.nameToCode();
+   }
+
+   @Override
+   protected Map<String, String> codeToName() {
+       return ParseStatus.codeToName();
+   }
+}

```

수정 controller/KGI10000\$DashboardView.java

```

@@ -25,18 +25,22 @@ public class KGI10000$DashboardView {
    @RequestParam(required = false) String keyword,
    @RequestParam(required = false, defaultValue = "1") int page,
    Model model) {
+
+   // — 2026-09-23 파싱 전환: KPI 측이 분류 → 파싱으로 바뀌었다 —
+
+   long total      = fileMapper.countAll();
-   long classified  = fileMapper.countByStatus("CLASSIFIED");
-
-   long unclassified = fileMapper.countByStatus("UNCLASSIFIED");
-
-   long excluded    = fileMapper.countByStatus("REJECTED") + fileMapper.countByStatus("DELETED");
+
+   long parseSuccess = fileMapper.countParseSuccess();
+   long parseFailed  = fileMapper.countParseFailed();
+   long unclassified = fileMapper.countParsedUnclassified();
+
+   model.addAttribute("total", total);
-   model.addAttribute("classified", classified);
+   model.addAttribute("parseSuccess", parseSuccess);
+   model.addAttribute("parseFailed", parseFailed);
+
+   model.addAttribute("unclassified", unclassified);
-   model.addAttribute("excluded", excluded);
-   model.addAttribute("unclassifiedCount", unclassified);
+   model.addAttribute("proposalCount", fileMapper.countProposalDocs());
+   model.addAttribute("rfpCount", fileMapper.countRfpDocs());
+
+   // 좌측 네비 배지 - 미분류 건수에서 파싱 실패 건수로 바뀌었다
+   model.addAttribute("parseFailedCount", parseFailed);
+
+   List<UploadedFile> recent = fileMapper.findRecent(10);
+   model.addAttribute("recentFiles", recent);

```

@@ -1,8 +1,8 @@

package com.kb.uploader.controller;

+import com.kb.uploader.code.DocumentType;

import com.kb.uploader.dto.UploadResultItem;

-import com.kb.uploader.mapper.InstitutionMapper;

import com.kb.uploader.mapper.UploadedFileMapper;

import com.kb.uploader.service.FileUploadService;

import org.springframework.stereotype.Controller;

import org.springframework.ui.Model;

@@ -13,32 +13,41 @@ import org.springframework.web.multipart.MultipartFile;

import java.util.List;

import java.util.stream.Collectors;

+/**

+ * 업로드 실행 (2026-09-23 파싱 전환).

+ *

+ * <p>업로드 창구가 둘로 갈렸다. 문서 종류마다 받는 확장자가 다르고 만드는 산출물도 다르다.

+ * 기관·연도·분류를 손으로 넣던 파라미터는 없앴다 - 이제 문서 본문에서 찾는다.

+ */

@Controller

@RequestMapping("/upload")

public class KGI11100\$UploadAction {

private final FileUploadService uploadService;

private final UploadedFileMapper fileMapper;

- private final InstitutionMapper instMapper;

public KGI11100\$UploadAction(FileUploadService uploadService,

- UploadedFileMapper fileMapper,

- InstitutionMapper instMapper) {

+ UploadedFileMapper fileMapper) {

this.uploadService = uploadService;

this.fileMapper = fileMapper;

- this.instMapper = instMapper;

}

- @PostMapping

- public String execute(@RequestParam("files") List<MultipartFile> files,

- @RequestParam(defaultValue = "") String institution,

- @RequestParam(defaultValue = "") String year,

- @RequestParam(defaultValue = "") String category,

- Model model) {

- populateCategories(model);

```

-     model.addAttribute("unclassifiedCount", fileMapper.countByStatus("UNCLASSIFIED"));

+     /** 임찰제안서 (ppt, pptx) */
+     @PostMapping("/proposal")
+     public String executeProposal(@RequestParam("files") List<MultipartFile> files, Model model) {
+         return handle(DocumentType.BID_PROPOSAL, files, model);
+     }

+
+     /** 임찰공고문 RFP (pdf, hwp, hwpX) */
+     @PostMapping("/rfp")
+     public String executeRfp(@RequestParam("files") List<MultipartFile> files, Model model) {
+         return handle(DocumentType.RFP, files, model);
+     }

+
+     private String handle(String docType, List<MultipartFile> files, Model model) {
+         model.addAttribute("parseFailedCount", fileMapper.countParseFailed());

        List<MultipartFile> validFiles = files.stream()
            .filter(f -> !f.isEmpty())
            .collect(Collectors.toList());
@@ -46,18 +55,10 @@ public class KGI11100$UploadAction {

        model.addAttribute("errorMessage", "파일을 선택해 주세요.");
        return "upload";
    }

-     List<UploadResultItem> results = uploadService.upload(validFiles, institution, year, category);
+     List<UploadResultItem> results = uploadService.upload(docType, validFiles);

    model.addAttribute("results", results);
+    model.addAttribute("uploadedType", DocumentType.label(docType));

    return "upload";
}

-
-     private void populateCategories(Model model) {
-
-         List<String> categories = instMapper.findAll().stream()
-
-             .map(i -> i.getCategory())
-
-             .filter(c -> c != null && !c.trim().isEmpty())
-
-             .distinct().sorted()
-
-             .collect(Collectors.toList());
-
-         model.addAttribute("categories", categories);
-     }
-}

```

```
\ No newline at end of file
+}
```

삭제 controller/KGI12000\$FileStatusList.java

```
deleted file mode 100644
```

```
@@ -1,34 +0,0 @@
```

```
-package com.kb.uploader.controller;
```

```
-
```

```
-import com.kb.uploader.code.InstitutionCategory;
```

```
-import com.kb.uploader.domain.UploadedFile;
```

```
-import com.kb.uploader.mapper.UploadedFileMapper;
```

```
-import org.springframework.stereotype.Controller;
```

```
-import org.springframework.ui.Model;
```

```
-import org.springframework.web.bind.annotation.GetMapping;
```

```
-import org.springframework.web.bind.annotation.RequestMapping;
```

```
-
```

```
-import java.util.List;
```

```
-
```

```
-@Controller
```

```
-@RequestMapping("/file-status")
```

```
-public class KGI12000$FileStatusList {
```

```
-
```

```
-
```

```
-    private final UploadedFileMapper fileMapper;
```

```
-
```

```
-    public KGI12000$FileStatusList(UploadedFileMapper fileMapper) {
```

```
-        this.fileMapper = fileMapper;
```

```
-    }
```

```
-
```

```
-    @GetMapping
```

```
-    public String execute(Model model) {
```

```
-        List<UploadedFile> unclassified = fileMapper.findByStatus("UNCLASSIFIED");
```

```
-        List<UploadedFile> unknownFiles = fileMapper.findClassifiedByUnknownInstitution();
```

```
-        model.addAttribute("files", unclassified);
```

```
-        model.addAttribute("unknownFiles", unknownFiles);
```

```
-        model.addAttribute("categories", InstitutionCategory.names());
```

```
-    model.addAttribute("unclassifiedCount", (long) unclassified.size());
-
-    return "file-status";
-
-    }
-
-}
```

신규 controller/KGI12000\$ParseStatusList.java

new file mode 100644

@@ -0,0 +1,54 @@

```
+package com.kb.uploader.controller;
+
+import com.kb.uploader.code.DocumentType;
+import com.kb.uploader.domain.UploadedFile;
+import com.kb.uploader.mapper.UploadedFileMapper;
+import org.springframework.stereotype.Controller;
+import org.springframework.ui.Model;
+import org.springframework.util.StringUtils;
+import org.springframework.web.bind.annotation.GetMapping;
+import org.springframework.web.bind.annotation.RequestMapping;
+import org.springframework.web.bind.annotation.RequestParam;
+
+import java.util.List;
+
+/**
+ * 파싱 현황 목록 (2026-09-23 신설, 옛 "파일 처리 상태" 화면을 대신한다).
+ *
+ * <p>분류 축 하나로 보던 화면을 <b>파싱 축</b>으로 바꾼 것이다. 파싱 성공/실패와
+ * 기관 미분류가 서로 다른 축이라 필터도 따로 둔다.
+ */
+@Controller
+@RequestMapping("/parse-status")
+public class KGI12000$ParseStatusList {
+
+    private final UploadedFileMapper fileMapper;
+
+    public KGI12000$ParseStatusList(UploadedFileMapper fileMapper) {
```

```

+     this.fileMapper = fileMapper;
+ }
+
+ @GetMapping
+ public String execute(@RequestParam(required = false) String docType,
+                      @RequestParam(required = false) String parseStatus,
+                      @RequestParam(required = false) String category,
+                      Model model) {
+     boolean unclassifiedOnly = "미분류".equals(category);
+     List<UploadedFile> files = fileMapper.searchParseStatus(
+         emptyOrNull(docType), emptyOrNull(parseStatus), unclassifiedOnly);
+
+     model.addAttribute("files", files);
+     model.addAttribute("docTypes", DocumentType.nameToCode().keySet());
+     model.addAttribute("docType", docType);
+     model.addAttribute("parseStatus", parseStatus);
+     model.addAttribute("category", category);
+     model.addAttribute("successCount", fileMapper.countParseSuccess());
+     model.addAttribute("parseFailedCount", fileMapper.countParseFailed());
+     model.addAttribute("unclassifiedCount", fileMapper.countParsedUnclassified());
+     return "parse-status";
+ }
+
+ private static String emptyOrNull(String s) {
+     return StringUtils.hasText(s) ? s : null;
+ }
+}

```

수정 controller/KGI12100\$FileStatusClassify.java

```

@@ -1,10 +1,10 @@
package com.kb.uploader.controller;

-import java.time.LocalDateTime;
-
-import com.kb.uploader.code.SystemUser;
+import com.kb.uploader.domain.UploadedFile;
+
+import com.kb.uploader.mapper.UploadedFileMapper;
-import com.kb.uploader.service.FileStorageService;

```

```

+import com.kb.uploader.service.DocumentParseService;

+import com.kb.uploader.service.InstitutionService;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.PathVariable;
@@ -12,54 +12,69 @@ import org.springframework.web.bind.annotation.PostMapping;

import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

+import java.util.Optional;

+
+/**
+ * 기관을 지정해 다시 분류한다 (2026-09-23 파싱 전환에 맞춰 동작 교체).
+ *
+ * <p>종전에는 파일을 {@code classified/} 폴더로 <b>옮기는</b> 것이 분류였다. 이제는 폴더를 옮기지
+ * 않으므로 이 화면의 일은 <b>기관을 기관 테이블에 등록하고 그 문서를 재파싱</b>하는 것이다.
+ * 재파싱하면 InstitutionMatcher 가 방금 등록한 기관을 찾아 기관분류가 확정되고,
+ * 산출물 파일명도 {@code {날짜}_{기관분류}_{기관명}_...} 으로 다시 만들어진다.
+ *
+ * <p>⚠ 기관명만 바꾸고 재파싱을 하지 않으면 산출물 파일명이 옛 기관명 그대로 남아
+ * DB 와 파일이 어긋난다. 그래서 두 가지를 한 번에 한다.
+ */
@Controller
@RequestMapping("/file-status")
public class KGI12100$FileStatusClassify {

    private static final Logger log = LoggerFactory.getLogger(KGI12100$FileStatusClassify.class);

    private final UploadedFileMapper fileMapper;
-    private final FileStorageService storageService;

+    private final InstitutionService institutionService;
+    private final DocumentParseService parseService;

    public KGI12100$FileStatusClassify(UploadedFileMapper fileMapper,
-        FileStorageService storageService) {

+        InstitutionService institutionService,
+        DocumentParseService parseService) {

        this.fileMapper = fileMapper;
-        this.storageService = storageService;

+        this.institutionService = institutionService;
+        this.parseService = parseService;

```

```

}

@PostMapping("/{id}/classify")
public String execute(@PathVariable Long id,
    @RequestParam String category,
    @RequestParam(defaultValue = "") String institution,
-    @RequestParam(defaultValue = "") String year,

    RedirectAttributes ra) {
-    fileMapper.findById(id).ifPresent(file -> {

-        if (!institution.trim().isEmpty()) {

-            file.setInstitutionName(institution.trim());

-        }

-        if (!year.trim().isEmpty()) {

-            file.setYear(year.trim());

-        }

-        String instName = file.getInstitutionName() != null ? file.getInstitutionName() : "알수없음";

-        try {

-            java.nio.file.Path source = java.nio.file.Paths.get(file.getStoredPath());

-            java.nio.file.Path dest = storageService.moveToClassified(

-                source,

-                category,

-                file.getYear() != null ? file.getYear() : "미확인",

-                instName

-            );

-            file.classify(category, dest.toString());

-            file.setSystemUserNo(SystemUser.get());

-            file.setSystemUsedAt(

-                file.getClassifiedAt() != null ? file.getClassifiedAt() : LocalDateTime.now());

-            fileMapper.update(file);

-        } catch (Exception e) {

-            log.warn("수동 분류 처리 실패: {}", file.getOriginalName(), e);

+        Optional<UploadedFile> found = fileMapper.findById(id);

+        if (!found.isPresent()) {

+            ra.addFlashAttribute("error", "파일을 찾을 수 없습니다 (id=" + id + ")");

+            return "redirect:/file-status/classified";

+        }

+        String name = institution.trim();

+        if (name.isEmpty()) {

```

```

+         ra.addFlashAttribute("error", "기관명을 입력해 주세요.");
+
+         return "redirect:/file-status/classified";
+     }
+
+     try {
+         institutionService.save(name, category.trim());
+
+         Optional<UploadedFile> result = parseService.reparse(id);
+
+         if (result.isPresent() && result.get().isParsed()) {
+
+             ra.addFlashAttribute("message",
+
+                 "기관 등록 후 재파싱 완료: " + name + " → " + result.get().getOutputFileName());
+
+         } else {
+
+             // ⚠ 실패 사유는 컬럼이 없어 저장하지 않는다(스키마 무변경안) - 로그를 봐야 한다.
+
+             ra.addFlashAttribute("error",
+
+                 "기관은 등록했으나 재파싱에 실패했습니다 - 사유는 서버 로그를 확인하세요");
+
+         }
-     });
-
-     ra.addFlashAttribute("message", "분류 처리 완료");
-
-     return "redirect:/file-status";
+
+     } catch (Exception e) {
+
+         log.warn("기관 지정 재파싱 실패: id={}", id, e);
+
+         ra.addFlashAttribute("error", "처리 실패: " + e.getMessage());
+
+     }
+
+     return "redirect:/file-status/classified";
+
+ }
+
+ }

```

신규 controller/KGI12110\$ParseReparse.java

```

new file mode 100644
@@ -0,0 +1,44 @@
+package com.kb.uploader.controller;
+
+import com.kb.uploader.domain.UploadedFile;
+import com.kb.uploader.service.DocumentParseService;
+import org.springframework.stereotype.Controller;
+import org.springframework.web.bind.annotation.PathVariable;
+import org.springframework.web.bind.annotation.PostMapping;
+import org.springframework.web.bind.annotation.RequestMapping;

```

```
+import org.springframework.web.servlet.mvc.support.RedirectAttributes;
+
+
+import java.util.Optional;
+
+/**
+ * 수동 재파싱 (2026-09-23 신설).
+ *
+ * <p>종전의 5분 주기 재분류 배치를 대신한다. 기관 정보를 고친 뒤 다시 돌리고 싶을 때 누른다.
+ * 감사 컬럼의 시스템사용자번호는 SystemUserInterceptor 가 이 클래스명에서 뽑아 넣는다.
+ */
+@Controller
+@RequestMapping("/parse-status")
+public class KGI12110$ParseReparse {
+
+    private final DocumentParseService parseService;
+
+    public KGI12110$ParseReparse(DocumentParseService parseService) {
+        this.parseService = parseService;
+    }
+
+    @PostMapping("/{id}/reparse")
+    public String execute(@PathVariable Long id, RedirectAttributes ra) {
+        Optional<UploadedFile> result = parseService.reparse(id);
+
+        if (!result.isPresent()) {
+            ra.addFlashAttribute("error", "파일을 찾을 수 없습니다 (id=" + id + ")");
+        } else if (result.get().isParsed()) {
+            ra.addFlashAttribute("message", "재파싱 완료: " + result.get().getOriginalName()
+                + " → " + result.get().getOutputFileName());
+        } else {
+            // ⚠ 실패 사유는 저장하지 않는다(스키마 무변경안). 로그를 볼 것.
+            ra.addFlashAttribute("error", "재파싱 실패: " + result.get().getOriginalName()
+                + " - 사유는 서버 로그를 확인하세요");
+        }
+
+        return "redirect:/parse-status";
+    }
+}
```

```
+}
```

신규 controller/KGI12120\$ParseOutput.java

```
new file mode 100644
```

```
@@ -0,0 +1,56 @@
```

```
+package com.kb.uploader.controller;
```

```
+
```

```
+import com.kb.uploader.domain.UploadedFile;
```

```
+import com.kb.uploader.mapper.UploadedFileMapper;
```

```
+import org.springframework.http.ContentDisposition;
```

```
+import org.springframework.http.HttpHeaders;
```

```
+import org.springframework.http.HttpStatus;
```

```
+import org.springframework.http.MediaType;
```

```
+import org.springframework.http.ResponseEntity;
```

```
+import org.springframework.stereotype.Controller;
```

```
+import org.springframework.web.bind.annotation.*;
```

```
+
```

```
+import java.io.IOException;
```

```
+import java.nio.charset.StandardCharsets;
```

```
+import java.nio.file.Files;
```

```
+import java.nio.file.Path;
```

```
+import java.nio.file.Paths;
```

```
+import java.util.Optional;
```

```
+
```

```
+/**
```

```
+ * 파싱 산출물(JSON/MD) 보기·내려받기 (2026-09-23 신설).
```

```
+ *
```

```
+ * <p>⚠ Content-Type 을 text/plain; charset=UTF-8 로 고정한다. 브라우저가 인코딩을 추측하면
```

```
+ * 한글 요약이 깨져 보인다.
```

```
+ */
```

```
+@Controller
```

```
+@RequestMapping("/parse-status")
```

```
+public class KGI12120$ParseOutput {
```

```
+
```

```
+    private final UploadedFileMapper fileMapper;
```

```

+
+ public KGI12120$ParseOutput(UploadedFileMapper fileMapper) {
+     this.fileMapper = fileMapper;
+ }
+
+ @GetMapping("/{id}/output")
+ @ResponseBody
+ public ResponseEntity<byte[]> execute(@PathVariable Long id,
+                                     @RequestParam(defaultValue = "false") boolean download)
+     throws IOException {
+     Optional<UploadedFile> file = fileMapper.findById(id);
+     if (!file.isPresent() || file.get().getOutputPath() == null) {
+         return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
+     }
+     Path path = Paths.get(file.get().getOutputPath());
+     if (!Files.exists(path)) {
+         return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
+     }
+     ContentDisposition disposition = ContentDisposition.builder(download ? "attachment" : "inline")
+         .filename(path.getFileName().toString(), StandardCharsets.UTF_8).build();
+     return ResponseEntity.ok()
+         .header(HttpHeaders.CONTENT_DISPOSITION, disposition.toString())
+         .contentType(new MediaType("text", "plain", StandardCharsets.UTF_8))
+         .body(Files.readAllBytes(path));
+ }
+}

```

수정 controller/KGI12200\$FileStatusDelete.java

```

@@ -22,7 +22,7 @@ public class KGI12200$FileStatusDelete {
    @PostMapping("/{id}/delete")
    public String execute(@PathVariable Long id, RedirectAttributes ra) {
        fileMapper.softDeleteById(id, SystemUser.get(), LocalDateTime.now());
        ra.addFlashAttribute("message", "파일이 목록에서 제거되었습니다.");
-       return "redirect:/file-status";
+       return "redirect:/parse-status";
    }
}

```

수정 controller/KGI12300\$FileStatusReject.java

```
@@ -22,7 +22,7 @@ public class KGI12300$FileStatusReject {  
  
    @PostMapping("/{id}/reject")  
    public String execute(@PathVariable Long id, RedirectAttributes ra) {  
        fileMapper.rejectById(id, SystemUser.get(), LocalDateTime.now());  
        ra.addFlashAttribute("message", "반려 처리되었습니다.");  
-        return "redirect:/file-status";  
  
+        return "redirect:/parse-status";  
  
    }  
}
```

수정 controller/KGI12400\$ClassifiedFileList.java

```
@@ -20,14 +20,21 @@ public class KGI12400$ClassifiedFileList {  
  
    public KGI12400$ClassifiedFileList(UploadedFileMapper fileMapper) {  
        this.fileMapper = fileMapper;  
    }  
  
+    /**  
+     * 파싱은 됐는데 <b>기관을 못 찾은</b> 건을 모은다 (2026-09-23 파싱 전환에 맞춰 조건 교체).  
+     *  
+     * <p>중전 조건은 {@code 기관명 = '알수없음' AND 분류상태구분 = '02'} 였는데, 새 파싱 경로는  
+     * 기관을 못 찾으면 이름을 "기관미상" 으로 두고 분류상태를 미분류로 남긴다. 그래서  
+     * <b>기관영업분류구분이 NULL</b> 인 것을 기준으로 바꿨다 - 그 값이 곧 "미분류" 다.  
+     */  
  
    @GetMapping  
    public String execute(Model model) {  
-        List<UploadedFile> classifiedFiles = fileMapper.findByStatus("CLASSIFIED");  
  
-        long unclassified = fileMapper.countByStatus("UNCLASSIFIED");  
  
+        List<UploadedFile> classifiedFiles =  
  
+            fileMapper.searchParseStatus(null, "SUCCESS", Boolean.TRUE);  
  
        model.addAttribute("classifiedFiles", classifiedFiles);  
        model.addAttribute("categories", InstitutionCategory.names());  
-        model.addAttribute("unclassifiedCount", unclassified);  
  
+        model.addAttribute("parseFailedCount", fileMapper.countParseFailed());  
  
        return "classified-rework";  
    }  
}
```

신규 controller/ProposalApiController.java

```
new file mode 100644  
@@ -0,0 +1,118 @@  
  
+package com.kb.uploader.controller;  
  
+
```

```

+import com.fasterxml.jackson.databind.JsonNode;

+import com.fasterxml.jackson.databind.ObjectMapper;

+import com.fasterxml.jackson.databind.node.ObjectNode;

+import com.kb.uploader.code.DocumentType;

+import com.kb.uploader.domain.UploadedFile;

+import com.kb.uploader.mapper.UploadedFileMapper;

+import org.springframework.http.HttpStatus;

+import org.springframework.http.ResponseEntity;

+import org.springframework.util.StringUtils;

+import org.springframework.web.bind.annotation.*;

+

+import java.io.IOException;

+import java.nio.file.Files;

+import java.nio.file.Path;

+import java.nio.file.Paths;

+import java.time.format.DateTimeFormatter;

+import java.util.ArrayList;

+import java.util.Collections;

+import java.util.LinkedHashMap;

+import java.util.List;

+import java.util.Map;

+import java.util.Optional;

+

+/**
+ * 입찰제안서 조회 REST API (외부 agent 호출용, 인증 없음).
+ *
+ * <p>같은 저장소의 {@code FileSearchApiController} 와 같은 방식이라 화면번호를 쓰지 않는다.
+ */
+@RestController
+@RequestMapping("/api/proposals")
+public class ProposalApiController {

+

+    private static final DateTimeFormatter TS = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

+

+    private final UploadedFileMapper fileMapper;

```

```

+ private final ObjectMapper mapper;
+
+ public ProposalApiController(UploadedFileMapper fileMapper, ObjectMapper mapper) {
+     this.fileMapper = fileMapper;
+     this.mapper = mapper;
+ }
+
+ /** 파싱에 성공한 입찰제안서 목록. 모든 조건은 선택. */
+ @GetMapping
+ public Map<String, Object> search(@RequestParam(required = false) String institution,
+     @RequestParam(required = false) String year,
+     @RequestParam(defaultValue = "1") int page,
+     @RequestParam(defaultValue = "20") int size) {
+     int p = Math.max(page, 1);
+     int s = Math.min(Math.max(size, 1), 100);
+
+     // ⚠ Oracle 은 빈 문자열을 NULL 로 취급한다. 조건이 비면 null 을 넘긴다(기존 search 와 같은 규칙).
+     String inst = trimOrNull(institution);
+     String docDate = trimOrNull(year);
+
+     List<UploadedFile> rows = fileMapper.searchProposals(inst, docDate, (p - 1) * s, s);
+     long total = fileMapper.countProposals(inst, docDate);
+
+     List<Map<String, Object>> items = new ArrayList<Map<String, Object>>();
+     for (UploadedFile f : rows) items.add(summary(f));
+
+     Map<String, Object> body = new LinkedHashMap<String, Object>();
+     body.put("total", total);
+     body.put("page", p);
+     body.put("size", s);
+     body.put("totalPages", (int) Math.ceil((double) total / s));
+     body.put("items", items);
+
+     return body;
+ }
+
+ /** 제안서 한 건의 파싱 JSON 전체 (slides 포함). */

```

```

+ @GetMapping("/{id}")
+ public ResponseEntity<?> detail(@PathVariable Long id) throws IOException {
+     Optional<UploadedFile> found = fileMapper.findById(id);
+     if (!found.isPresent() || !DocumentType.BID_PROPOSAL.equals(found.get().getDocType())) {
+         return error(HttpStatus.NOT_FOUND, "입찰제안서를 찾을 수 없습니다 (id=" + id + ")");
+     }
+     UploadedFile f = found.get();
+     if (!f.isParsed()) {
+         return error(HttpStatus.NOT_FOUND, "파싱에 성공하지 않은 제안서입니다 (id=" + id + ")");
+     }
+     Path json = Paths.get(f.getOutputPath());
+     if (!Files.exists(json)) {
+         return error(HttpStatus.NOT_FOUND, "파싱 JSON 파일이 없습니다: " + json.getFileName());
+     }
+     JsonNode parsed = mapper.readTree(json.toFile());
+     ObjectNode body = mapper.createObjectNode();
+     body.put("id", f.getId());
+     body.put("institutionName", f.getInstitutionName());
+     body.put("institutionCategory", f.getCategoryLabel());
+     body.put("year", f.getDocDate());
+     body.put("jsonFileName", json.getFileName().toString());
+     body.setAll((ObjectNode) parsed);
+     return ResponseEntity.ok(body);
+ }
+
+ private static Map<String, Object> summary(UploadedFile f) {
+     Map<String, Object> m = new LinkedHashMap<String, Object>();
+     m.put("id", f.getId());
+     m.put("fileName", f.getOriginalName());
+     m.put("institutionName", f.getInstitutionName());
+     m.put("institutionCategory", f.getCategoryLabel());
+     m.put("year", f.getDocDate());
+     m.put("jsonFileName", f.getOutputFileName());
+     m.put("uploadedAt", f.getUploadedAt() != null ? f.getUploadedAt().format(TS) : null);
+     m.put("parsedAt", f.getParsedAt() != null ? f.getParsedAt().format(TS) : null);

```

```

+     return m;
+ }
+
+ private static ResponseEntity<Map<String, String>> error(HttpStatus status, String message) {
+     return ResponseEntity.status(status).body(Collections.singletonMap("error", message));
+ }
+
+ private static String trimToNull(String s) {
+     return StringUtils.hasText(s) ? s.trim() : null;
+ }
+
+}

```

신규 controller/RfpApiController.java

new file mode 100644

@@ -0,0 +1,139 @@

```

+package com.kb.uploader.controller;
+
+import com.kb.uploader.code.DocumentType;
+import com.kb.uploader.domain.UploadedFile;
+import com.kb.uploader.mapper.UploadedFileMapper;
+import org.springframework.http.HttpHeaders;
+import org.springframework.http.HttpStatus;
+import org.springframework.http.MediaType;
+import org.springframework.http.ResponseEntity;
+import org.springframework.util.StringUtils;
+import org.springframework.web.bind.annotation.*;
+
+import java.io.IOException;
+import java.nio.charset.StandardCharsets;
+import java.nio.file.Files;
+import java.nio.file.Path;
+import java.nio.file.Paths;
+import java.time.format.DateTimeFormatter;
+import java.util.ArrayList;
+import java.util.Collections;

```

```

+import java.util.LinkedHashMap;

+import java.util.List;

+import java.util.Map;

+import java.util.Optional;

+

+/**
+ * 입찰공고문(RFP) 요약 조회 REST API (외부 agent 호출용, 인증 없음).
+ *
+ * <p>제안서는 산출물이 JSON 이라 그대로 내려주면 되지만 RFP 산출물은 Markdown 이라 쓰는 쪽이 갈린다.
+ * 그래서 같은 주소에서 두 가지로 준다.
+ * <ul>
+ * <li>기본(JSON): 메타데이터 + {@code markdown} 필드에 원문 문자열</li>
+ * <li>{@code ?format=text} 또는 {@code Accept: text/markdown}: md 원문 그대로 (LLM 입력용)</li>
+ * </ul>
+ */
+@RestController
+@RequestMapping("/api/rfps")
+public class RfpApiController {

+

+    private static final DateTimeFormatter TS = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

+    private static final MediaType MARKDOWN = new MediaType("text", "markdown", StandardCharsets.UTF_8);

+

+    private final UploadedFileMapper fileMapper;

+

+    public RfpApiController(UploadedFileMapper fileMapper) {

+        this.fileMapper = fileMapper;

+    }

+

+    /** 파싱에 성공한 RFP 목록. 모든 조건은 선택. */

+    @GetMapping

+    public Map<String, Object> search(@RequestParam(required = false) String institution,

+                                     @RequestParam(required = false) String noticeDate,

+                                     @RequestParam(defaultValue = "1") int page,

+                                     @RequestParam(defaultValue = "20") int size) {

+        int p = Math.max(page, 1);

```

```

+     int s = Math.min(Math.max(size, 1), 100);
+
+     String inst = trimToNull(institution);
+
+     String docDate = trimToNull(noticeDate);
+
+
+     List<UploadedFile> rows = fileMapper.searchRfps(inst, docDate, (p - 1) * s, s);
+
+     long total = fileMapper.countRfps(inst, docDate);
+
+
+     List<Map<String, Object>> items = new ArrayList<Map<String, Object>>();
+
+     for (UploadedFile f : rows) items.add(summary(f));
+
+
+     Map<String, Object> body = new LinkedHashMap<String, Object>();
+
+     body.put("total", total);
+
+     body.put("page", p);
+
+     body.put("size", s);
+
+     body.put("totalPages", (int) Math.ceil((double) total / s));
+
+     body.put("items", items);
+
+     return body;
+
+ }
+
+
+ /**
+  * RFP 한 건의 요약.
+  *
+  * @param format {@code json}(기본) 이면 메타데이터 + markdown 필드, {@code text} 면 md 원문
+  */
+
+ @GetMapping("/{id}")
+ public ResponseEntity<?> detail(@PathVariable Long id,
+
+     @RequestParam(required = false) String format,
+
+     @RequestHeader(name = HttpHeaders.ACCEPT, required = false) String accept)
+
+     throws IOException {
+
+     Optional<UploadedFile> found = fileMapper.findById(id);
+
+     if (!found.isPresent() || !DocumentType.RFP.equals(found.get().getDocType())) {
+
+         return error(HttpStatus.NOT_FOUND, "입찰공고문(RFP)을 찾을 수 없습니다 (id=" + id + ")");
+
+     }
+
+     UploadedFile f = found.get();
+
+     if (!f.isParsed()) {

```

```

+         return error(HttpStatus.NOT_FOUND, "파싱에 성공하지 않은 공고문입니다 (id=" + id + ")");
+     }
+
+     Path md = Paths.get(f.getOutputPath());
+
+     if (!Files.exists(md)) {
+
+         return error(HttpStatus.NOT_FOUND, "요약 MD 파일이 없습니다: " + md.getFileName());
+
+     }
+
+     String markdown = new String(Files.readAllBytes(md), StandardCharsets.UTF_8);
+
+
+     if (wantsRawMarkdown(format, accept)) {
+
+         return ResponseEntity.ok().contentType(MARKDOWN).body(markdown);
+
+     }
+
+     Map<String, Object> body = summary(f);
+
+     body.put("markdown", markdown);
+
+     return ResponseEntity.ok(body);
+ }
+
+
+ /** {@code ?format=text} 가 우선이고, 없으면 Accept 헤더가 md/plain 만 받겠다고 한 경우. */
+ private static boolean wantsRawMarkdown(String format, String accept) {
+
+     if (StringUtils.hasText(format)) {
+
+         String f = format.trim().toLowerCase();
+
+         return "text".equals(f) || "md".equals(f) || "markdown".equals(f);
+
+     }
+
+     if (!StringUtils.hasText(accept)) return false;
+
+     String a = accept.toLowerCase();
+
+     if (a.contains("application/json") || a.contains("*/")) return false;
+
+     return a.contains("text/markdown") || a.contains("text/plain");
+
+ }
+
+
+ private static Map<String, Object> summary(UploadedFile f) {
+
+     Map<String, Object> m = new LinkedHashMap<String, Object>();
+
+     m.put("id", f.getId());
+
+     m.put("fileName", f.getOriginalName());
+
+     m.put("institutionName", f.getInstitutionName());
+
+     m.put("institutionCategory", f.getCategoryLabel());
+
+     m.put("noticeDate", f.getDocDate());

```

```

+     m.put("mdFileName", f.getOutputFileName());
+
+     m.put("uploadedAt", f.getUploadedAt() != null ? f.getUploadedAt().format(TS) : null);
+
+     m.put("parsedAt", f.getParsedAt() != null ? f.getParsedAt().format(TS) : null);
+
+     return m;
+
+ }
+
+
+ private static ResponseEntity<Map<String, String>> error(HttpStatus status, String message) {
+
+     return ResponseEntity.status(status).body(Collections.singletonMap("error", message));
+
+ }
+
+
+ private static String trimToNull(String s) {
+
+     return StringUtils.hasText(s) ? s.trim() : null;
+
+ }
+
+}

```

수정 domain/UploadedFile.java

```

@@ -26,10 +26,124 @@ public class UploadedFile {

    private String status;
    private LocalDateTime uploadedAt;
    private LocalDateTime classifiedAt;

+    /** 기관 테이블에 없을 때의 기관분류 값. ⚠ InstitutionCategory 코드에는 없는 값이다. */
+
+    public static final String UNKNOWN_CATEGORY = "미분류";
+
+
+    public UploadedFile() {}
+
+    /**
+
+     * 파싱 전환 후의 업로드 (2026-09-23).
+
+     *
+
+     * <p>⚠ 문서종류·파싱상태를 <b>컬럼으로 두지 않는다</b>(스키마 무변경안). 문서종류는
+
+     * 원본 확장자로, 파싱 성공 여부는 저장경로의 확장자로 판별한다 - getDocType().isParsed() 참조.
+
+     */
+
+    public UploadedFile(String originalName, String storedPath) {
+
+        this.originalName = originalName;
+
+        this.storedPath = storedPath;
+
+        this.status = "UNCLASSIFIED";
+
+        this.uploadedAt = LocalDateTime.now();
+
+    }

```

```

+
+ /**
+  * 파싱 성공 기록 (스키마 무변경안).
+
+  *
+  * <p>산출물 경로를 {@code 저장경로내용} 에 넣는다. 원본 경로는
+  * {@code userdata + 원본파일명} 으로 언제나 재구성되므로 따로 보관하지 않는다.
+  * 파싱 시각은 {@code 분류일시} 에 넣는다 - 이 흐름에서 둘은 같은 시각이다.
+  *
+  * <p>⚠ 기관을 못 찾으면 분류상태는 UNCLASSIFIED 로 남는다. 그래도 산출물은 만들어졌으므로
+  * 화면에서는 "파싱 성공 + 기관 미분류" 로 보인다(판별은 저장경로 확장자로 한다).
+  */
+
+ public void markParsed(String institutionName, String category, String docDate,
+
+         String outputPath, LocalDateTime parsedAt) {
+
+     this.institutionName = institutionName;
+
+     this.category = category;
+
+     this.storedPath = outputPath;
+
+     this.classifiedAt = parsedAt;
+
+     if (docDate != null && docDate.length() == 4) {
+
+         this.year = docDate;          // 제안서 연도는 기존 문서년(4) 에 그대로 들어간다
+
+     }
+
+     this.status = (category != null && !UNKNOWN_CATEGORY.equals(category))
+
+         ? "CLASSIFIED" : "UNCLASSIFIED";
+
+ }
+
+ /**
+  * 파싱 실패 기록.
+
+  *
+  * <p>⚠ <b>실패 사유는 저장하지 않는다</b>(널을 컬럼이 없다 - 스키마 무변경안의 유일한 손실).
+  * 사유는 서버 로그에만 남는다. 저장경로는 원본 그대로 두며, 그 사실이 곧 "파싱 실패" 표시다.
+  */
+
+ public void markParseFailed(String originalPath) {
+
+     this.storedPath = originalPath;
+
+     this.classifiedAt = null;
+
+     this.status = "UNCLASSIFIED";
+
+ }

```

```

+
+ /**
+  * 화면에 보여 줄 기관분류. DB 에는 "미분류" 코드가 없어 NULL 로 저장하므로
+  * 읽을 때 되돌린다(Mapper XML 의 {@code 기관분류값} 조각 참조).
+  */
+
+ public String getCategoryLabel() {
+
+     return category == null ? UNKNOWN_CATEGORY : category;
+
+ }
+
+
+ // — 파생값 (스키마 무변경안) —————
+
+ // 컬럼을 늘리지 않으려고 아래 값들은 **저장하지 않고 그때그때 계산**한다.
+
+ // 규칙이 한곳(DocumentType)에 모여 있어 SQL 과 화면이 같은 기준을 쓴다.
+
+
+ /** 문서종류. 원본 확장자로 판별한다. 규칙 밖이면 null(옛 방식 업로드). */
+
+ public String getDocType() {
+
+     return com.kb.uploader.code.DocumentType.ofFileName(originalName);
+
+ }
+
+
+ public String getDocTypeLabel() {
+
+     String type = getDocType();
+
+     return type == null ? "이전 방식" : com.kb.uploader.code.DocumentType.label(type);
+
+ }
+
+
+ /**
+  * 파싱 성공 여부. <b>저장경로가 산출물(.json.md)이면 성공</b>이다.
+  * 원본 확장자 그대로면 아직 산출물이 없다는 뜻이라 실패로 본다.
+  */
+
+ public boolean isParsed() {
+
+     return com.kb.uploader.code.DocumentType.isOutputPath(storedPath);
+
+ }
+
+
+ /** 화면·API 가 쓰는 파싱상태 문자열. 컬럼이 아니라 계산값이다. */
+
+ public String getParseStatus() {
+
+     if (getDocType() == null) return null;      // 옛 방식 행
+
+     return isParsed() ? "SUCCESS" : "FAILED";

```

```

+     }
+
+     /** 산출물 경로. 파싱 전/실패면 null. */
+     public String getOutputPath() {
+         return isParsed() ? storedPath : null;
+     }
+
+     /** 파싱 시각. 분류일시를 그대로 쓴다. */
+     public LocalDateTime getParsedAt() {
+         return isParsed() ? classifiedAt : null;
+     }
+
+     /**
+      * 제안서 연도(4) 또는 RFP 공고일(8). 산출물 파일명의 앞 토큰이 그 값이다
+      * ({@code 20240315_지자체_서울특별시_20260923.md}). 없으면 기존 문서년으로 떨어진다.
+      */
+     public String getDocDate() {
+         String head = com.kb.uploader.code.DocumentType.leadingDate(getOutputFileName());
+         return head != null ? head : year;
+     }
+
+     /** 산출물 파일명만. 화면 표시용. */
+     public String getOutputFileName() {
+         String path = getOutputPath();
+         if (path == null) return null;
+         int slash = Math.max(path.lastIndexOf('/'), path.lastIndexOf('\\'));
+         return slash < 0 ? path : path.substring(slash + 1);
+     }
+
+     public UploadedFile(String originalName, String storedPath,
+         String year, String institutionName) {
+         this.originalName = originalName;
+         this.storedPath = storedPath;
+
+@@ -72,5 +186,6 @@ public class UploadedFile {
+
+     public String getSystemUserNo() { return systemUserNo; }
+     public void setSystemUserNo(String systemUserNo) { this.systemUserNo = systemUserNo; }
+     public LocalDateTime getSystemUsedAt() { return systemUsedAt; }
+     public void setSystemUsedAt(LocalDateTime systemUsedAt) { this.systemUsedAt = systemUsedAt; }
+
+

```

```
}
```

삭제 dto/ParsedFileName.java

deleted file mode 100644

@@ -1,21 +0,0 @@

```
-package com.kb.uploader.dto;
-
-
-public class ParsedFileName {
-    private final String year;
-    private final String institutionName;
-    private final String description;
-    private final String extension;
-
-
-    public ParsedFileName(String year, String institutionName,
-                           String description, String extension) {
-        this.year = year;
-        this.institutionName = institutionName;
-        this.description = description;
-        this.extension = extension;
-    }
-
-
-    public String getYear() { return year; }
-    public String getInstitutionName() { return institutionName; }
-    public String getDescription() { return description; }
-    public String getExtension() { return extension; }
-}
```

삭제 job/ReclassificationJob.java

deleted file mode 100644

@@ -1,65 +0,0 @@

```
-package com.kb.uploader.job;
-
-
-
-import com.kb.uploader.code.SystemUser;
-import com.kb.uploader.domain.UploadedFile;
-import com.kb.uploader.mapper.UploadedFileMapper;
-import com.kb.uploader.service.ClassificationService;
```

```

-import org.slf4j.Logger;

-import org.slf4j.LoggerFactory;

-import org.springframework.stereotype.Component;

-

-import java.util.List;

-

-/**
- * 미분류로 남은 파일을 다시 분류해 본다. <b>언제 도는지는 여기서 정하지 않는다.</b>
- *
- * <p>⚠ 종전에는 이 메서드에 {@code @Scheduled(cron = "${reclassification.cron}")} 이
- * 붙어 있었다(2026-08-31 제거). 그러면 Spring 이 자기 타이머 스레드 풀을 만들고,
- * <b>WAS 에서 애플이 스레드를 만드는 것은 금지</b>다(설계 §2.54). 반복 실행은
- * {@link ReclassificationTrigger} 가 {@link BackgroundScheduler} 를 통해 건다 -
- * WebLogic 에서는 CommonJ TimerManager 가 그 스레드를 준다.
- *
- * <p>덕분에 이 클래스는 <b>"한 번 돌리는 일"만</b> 안다. 수동 실행이 필요해지면
- * 그냥 부르면 된다.
- */
-@Component
-public class ReclassificationJob {
-
-    private static final Logger log = LoggerFactory.getLogger(ReclassificationJob.class);
-
-    private final UploadedFileMapper fileMapper;
-    private final ClassificationService classificationService;
-
-    public ReclassificationJob(UploadedFileMapper fileMapper,
-                               ClassificationService classificationService) {
-        this.fileMapper = fileMapper;
-        this.classificationService = classificationService;
-    }
-
-    public void reclassify() {
-        // ⚠ 배치는 컨트롤러가 없어 SystemUserInterceptor 가 안 돈다. 직접 넣고
-        // finally 에서 반드시 지운다 - ThreadLocal 이라 안 지우면 그 스레드를

```

```
// 물려받은 다음 작업에 'BATCH01' 이 새어 나간다.

SystemUser.set(SystemUser.BATCH);

try {

    List<UploadedFile> unclassified = fileMapper.findByStatus("UNCLASSIFIED");

    if (unclassified.isEmpty()) {

        log.debug("재분류할 파일 없음");

        return;

    }

    int success = 0;

    for (UploadedFile file : unclassified) {

        try {

            if (classificationService.classify(file)) {

                success++;

            }

        } catch (Exception e) {

            log.warn("재처리 실패: {} - {}", file.getOriginalName(), e.getMessage());

        }

    }

    log.info("재분류 완료: {}건 성공 / {}건 대상", success, unclassified.size());

} finally {

    SystemUser.clear();

}

}
```

삭제 job/ReclassificationTrigger.java

```
deleted file mode 100644
@@ -1,109 +0,0 @@

-package com.kb.uploader.job;

-

-import org.slf4j.Logger;

-

-import org.slf4j.LoggerFactory;

-

-import org.springframework.beans.factory.annotation.Value;

-

-import org.springframework.scheduling.support.CronExpression;

-

-import org.springframework.stereotype.Component;
```

```

-
-import javax.annotation.PostConstruct;
-import javax.annotation.PreDestroy;
-import java.time.Duration;
-import java.time.LocalDateTime;
-import java.util.concurrent.atomic.AtomicBoolean;
-
-/**
- * {@code reclassification.cron} 에 따라 {@link ReclassificationJob} 을 반복 실행한다.
- *
- * <p><b>왜 {@code @Scheduled} 가 아닌가.</b> {@code @EnableScheduling} 은 Spring 이
- * <b>자기 타이머 스레드 풀을 직접 만들게</b> 한다 - WAS 금지 사항이다(설계 §2·§4,
- * {@link BackgroundScheduler} 의 설명 참조).
- *
- * <p><b>왜 "한 번 돌고 다시 예약"인가.</b> CommonJ TimerManager 는 고정 주기(밀리초)를
- * 받지 cron 을 모른다. 밀리초 주기를 따로 두면 {@code reclassification.cron} 과
- * <b>두 별</b>이 되어 조용히 갈린다. 그래서 주기의 근거를 cron 한 곳으로 두고, 매 실행
- * 뒤 <b>다음 cron 시각까지의 지연</b>을 계산해 1회성 예약을 다시 건다.
- *
- * <p>⚠ <b>다음 예약은 {@code finally} 에서 건다.</b> 한 번의 재분류가 실패했다고
- * 반복이 영영 멈추면, 증상이 "미분류 파일이 계속 쌓인다"로만 나타나 원인을 찾기 어렵다.
- *
- * <p>⚠ <b>고정 주기(fixed-rate)가 아니라 실행 후 재계산이다.</b> 한 번이 cron 간격보다
- * 오래 걸리면 그 주기는 건너뛴다 - 재분류가 겹쳐 도는 것보다 낫다(같은 파일을 두
- * 스레드가 옮기면 경로가 어긋난다).
- */
-@Component
-public class ReclassificationTrigger {
-
-    private static final Logger log = LoggerFactory.getLogger(ReclassificationTrigger.class);
-
-    private static final String NAME = "reclassification";
-
-    private final BackgroundScheduler scheduler;
-
-    private final ReclassificationJob job;

```

```

- private final String cron;
-
-
- /** 컨텍스트가 닫힌 뒤에 다음 예약을 걸지 않기 위한 스위치. */
- private final AtomicBoolean running = new AtomicBoolean(false);
-
- private CronExpression expression;
-
- public ReclassificationTrigger(BackgroundScheduler scheduler,
-                               ReclassificationJob job,
-                               @Value("${reclassification.cron:}") String cron) {
-     this.scheduler = scheduler;
-     this.job = job;
-     this.cron = cron;
- }
-
- @PostConstruct
- public void start() {
-     if (cron == null || cron.trim().isEmpty()) {
-         // 끄는 것을 지원한다 - 내부망에서 재분류를 배치로 돌리기로 하면 이 값을 비운다.
-         log.warn("reclassification.cron 이 비어 있다 - 자동 재분류를 하지 않는다.");
-         return;
-     }
-     try {
-         this.expression = CronExpression.parse(cron.trim());
-     } catch (IllegalArgumentException e) {
-         // 기동 때 소리 내어 죽는다. 조용히 넘기면 "왜 재분류가 안 되지"로 남는다.
-         throw new IllegalStateException(
-             "reclassification.cron 을 해석하지 못했다: '" + cron + "'", e);
-     }
-     running.set(true);
-     log.info("자동 재분류: cron='{ }' · 실행 방식={}", cron.trim(), scheduler.describe());
-     scheduleNext();
- }
-
- @PreDestroy

```

```

- public void stop() {
-     running.set(false);
- }
-
- /** 다음 cron 시각까지의 지연으로 1회성 예약을 건다. */
- private void scheduleNext() {
-     if (!running.get()) {
-         return;
-     }
-
-     LocalDateTime now = LocalDateTime.now();
-     LocalDateTime next = expression.next(now);
-
-     if (next == null) {
-         // 예: 지나간 날짜만 가리키는 식. 더 돌 일이 없다.
-
-         log.warn("cron '{}' 에 다음 실행 시각이 없다 - 자동 재분류를 멈춘다.", cron);
-
-         running.set(false);
-         return;
-     }
-
-     long delay = Duration.between(now, next).toMillis();
-
-     scheduler.scheduleOnce(NAME, delay, this::runOnce);
- }
-
- /** 한 번 돌고, 성공과 무관하게 다음을 예약한다. */
- void runOnce() {
-     try {
-         job.reclassify();
-     } catch (RuntimeException e) {
-         log.warn("자동 재분류가 예외로 끝났다 - 다음 주기에 다시 시도한다.", e);
-     } finally {
-         scheduleNext();
-     }
- }
- }
-}

```

수정 mapper/UploadedFileMapper.java

```

@@ -44,5 +44,44 @@ public interface UploadedFileMapper {

```

```

        List<UploadedFile> search(@Param("institution") String institution,
                                @Param("year") String year,
                                @Param("keyword") String keyword);
+
+
+ // — 2026-09-23 파싱 전환 (스키마 무변경안) —————
+
+ // ⚠️ 컬럼을 늘리지 않았다. 문서종류·파싱 성공 여부는 XML 이 **파일명 확장자**로 판별한다.
+
+ // 판별 규칙 정보는 code/DocumentType 이고 XML 의 sql 조각이 그 SQL 판이다.
+
+
+ /** 파싱 결과 기록. 저장경로내용(산출물/원본)·분류일시(파싱시각)·문서년에 태운다. */
+
+ void updateParseResult(UploadedFile file);
+
+
+ long countParseSuccess();
+
+
+ long countParseFailed();
+
+
+ long countProposalDocs();
+
+
+ long countRfpDocs();
+
+
+ /** 파싱은 성공했는데 기관을 못 찾은 건(기관분류 NULL). */
+
+ long countParsedUnclassified();
+
+
+ /** 파싱 현황 화면. docType 은 "BID_PROPOSAL"·"RFP", parseStatus 는 "SUCCESS"·"FAILED". */
+
+ List<UploadedFile> searchParseStatus(@Param("docType") String docType,
+
+                                     @Param("parseStatus") String parseStatus,
+
+                                     @Param("unclassifiedOnly") Boolean unclassifiedOnly);
+
+
+ List<UploadedFile> searchProposals(@Param("institution") String institution,
+
+                                   @Param("docDate") String docDate,
+
+                                   @Param("offset") int offset,
+
+                                   @Param("limit") int limit);
+
+
+ long countProposals(@Param("institution") String institution,
+
+                     @Param("docDate") String docDate);
+
+
+ List<UploadedFile> searchRfps(@Param("institution") String institution,

```

```

+         @Param("docDate") String docDate,
+         @Param("offset") int offset,
+         @Param("limit") int limit);
+
+ long countRfps(@Param("institution") String institution,
+         @Param("docDate") String docDate);
+
+ }

```

삭제 service/ClassificationService.java

deleted file mode 100644

@@ -1,53 +0,0 @@

```

-package com.kb.uploader.service;
-
-import java.time.LocalDateTime;
-import com.kb.uploader.code.SystemUser;
-import com.kb.uploader.domain.Institution;
-import com.kb.uploader.domain.UploadedFile;
-import com.kb.uploader.mapper.InstitutionMapper;
-import com.kb.uploader.mapper.UploadedFileMapper;
-import org.slf4j.Logger;
-import org.slf4j.LoggerFactory;
-import org.springframework.stereotype.Service;
-
-import java.nio.file.Path;
-import java.nio.file.Paths;
-import java.util.Optional;
-
-@Service
-public class ClassificationService {
-
-    private static final Logger log = LoggerFactory.getLogger(ClassificationService.class);
-
-    private final InstitutionMapper institutionMapper;
-    private final UploadedFileMapper fileMapper;
-    private final FileStorageService storageService;
-
-}

```



```

+import com.fasterxml.jackson.databind.SerializationFeature;

+import com.kb.uploader.code.DocumentType;

+import com.kb.uploader.code.SystemUser;

+import com.kb.uploader.domain.Institution;

+import com.kb.uploader.domain.UploadedFile;

+import com.kb.uploader.mapper.InstitutionMapper;

+import com.kb.uploader.mapper.UploadedFileMapper;

+import com.kb.uploader.parse.DateDetector;

+import com.kb.uploader.parse.ExtractedDocument;

+import com.kb.uploader.parse.InstitutionMatcher;

+import com.kb.uploader.parse.PptExtractor;

+import com.kb.uploader.parse.ProposalDocument;

+import com.kb.uploader.parse.RfpExtractor;

+import com.kb.uploader.parse.RfpSummaryWriter;

+import com.kb.uploader.parse.SlideContent;

+import org.slf4j.Logger;

+import org.slf4j.LoggerFactory;

+import org.springframework.stereotype.Service;

+

+import java.nio.charset.StandardCharsets;

+import java.nio.file.Path;

+import java.nio.file.Paths;

+import java.time.LocalDateTime;

+import java.time.format.DateTimeFormatter;

+import java.util.ArrayList;

+import java.util.List;

+import java.util.Optional;

+

+/**
+ * 업로드 원본(userdata)을 파싱해 산출물을 만든다 (2026-09-23 신설).
+ *
+ * <ul>
+ * <li>입찰제안서: {@code {연도}_{기관분류}_{기관명}_{파싱yyyyMMdd}_bid_proposal.json}</li>
+ * <li>RFP: {@code {공고yyyyMMdd}_{기관분류}_{기관명}_{파싱yyyyMMdd}.md}</li>
+ * </ul>

```

+ *

+ * <p>종전의 {@code ClassificationService}{파일명으로 기관을 찾아 폴더를 옮기던 것}를 대신한다.

+ * 원본은 옮기지 않고 {@code userdata/} 에 그대로 둔다.

+ *

+ * <p>⚠ 감사 2열은 이 클래스가 채운다. 파싱은 배치·화면 양쪽에서 불리므로

+ * {@link SystemUser#get()} 값을 그대로 쓰고, 시각은 파싱일시와 같은 값으로 둔다.

+ */

+@Service

+public class DocumentParseService {

+

+ private static final Logger log = LoggerFactory.getLogger(DocumentParseService.class);

+

+ static final String UNKNOWN_NOTICE_DATE = "공고일미상";

+ private static final DateTimeFormatter DAY = DateTimeFormatter.ofPattern("yyyyMMdd");

+ private static final DateTimeFormatter STAMP = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

+

+ private final ObjectMapper mapper = new ObjectMapper().enable(SerializationFeature.INDENT_OUTPUT);

+

+ private final PptExtractor pptExtractor;

+ private final RfpExtractor rfpExtractor;

+ private final InstitutionMatcher institutionMatcher;

+ private final DateDetector dateDetector;

+ private final RfpSummaryWriter rfpSummaryWriter;

+ private final FileStorageService storageService;

+ private final InstitutionMapper institutionMapper;

+ private final UploadedFileMapper fileMapper;

+

+ public DocumentParseService(PptExtractor pptExtractor, RfpExtractor rfpExtractor,

+ InstitutionMatcher institutionMatcher, DateDetector dateDetector,

+ RfpSummaryWriter rfpSummaryWriter, FileStorageService storageService,

+ InstitutionMapper institutionMapper, UploadedFileMapper fileMapper) {

+ this.pptExtractor = pptExtractor;

+ this.rfpExtractor = rfpExtractor;

+ this.institutionMatcher = institutionMatcher;

+ this.dateDetector = dateDetector;

```

+     this.rfpSummaryWriter = rfpSummaryWriter;
+
+     this.storageService = storageService;
+
+     this.institutionMapper = institutionMapper;
+
+     this.fileMapper = fileMapper;
+ }
+
+
+ /** 파싱하고 결과(성공/실패)를 DB 에 기록한다. 예외를 밖으로 던지지 않는다. */
+ public UploadedFile parse(UploadedFile file) {
+
+     try {
+
+         String docType = file.getDocType();    // 원본 확장자로 판별한다(스키마 무변경안)
+
+         if (DocumentType.BID_PROPOSAL.equals(docType)) {
+
+             parseProposal(file);
+
+         } else if (DocumentType.RFP.equals(docType)) {
+
+             parseRfp(file);
+
+         } else {
+
+             throw new IllegalStateException("문서 종류가 없는 이전 방식 업로드입니다");
+
+         }
+
+     } catch (Exception e) {
+
+         // ⚠ 실패 사유를 담은 컬럼이 없다(스키마 무변경안). 로그가 유일한 기록이므로
+
+         // 스택까지 남긴다 - 화면에는 "실패" 표시만 나간다.
+
+         log.warn("파싱 실패: {} - {}", file.getOriginalName(),
+
+             e.getMessage() != null ? e.getMessage() : e.getClass().getSimpleName(), e);
+
+         file.markParseFailed(originalPath(file));
+
+     }
+
+     stampAudit(file);
+
+     fileMapper.updateParseResult(file);
+
+     return file;
+ }
+
+
+ /** 이전 산출물을 지우고 userdata 원본으로 다시 파싱한다. */
+ public Optional<UploadedFile> reparse(Long id) {
+
+     Optional<UploadedFile> found = fileMapper.findById(id);
+
+     if (!found.isPresent()) return Optional.empty();
+
+     UploadedFile file = found.get();
+
+     try {

```

```

+         storageService.deleteIfExists(file.getOutputPath()); // 성공 건만 값이 있다
+     } catch (Exception e) {
+         log.warn("이전 산출물 삭제 실패: {}", file.getOutputPath(), e);
+     }
+     return Optional.of(parse(file));
+ }

+
+ private void parseProposal(UploadedFile file) throws Exception {
+     LocalDateTime now = LocalDateTime.now();
+     List<SlideContent> slides = pptExtractor.extract(Paths.get(originalPath(file)));
+     if (slides.isEmpty()) throw new IllegalStateException("슬라이드가 없습니다");
+
+     StringBuilder all = new StringBuilder();
+     for (SlideContent s : slides) {
+         all.append(s.getTitle()).append('\n').append(s.getTextFull()).append('\n');
+     }
+
+     InstitutionMatcher.Match inst = institutionMatcher.match(all.toString(), institutions());
+
+     List<String> notes = new ArrayList<String>();
+     Optional<String> detected = dateDetector.detectProposalYear(slides);
+     String year;
+     if (detected.isPresent()) {
+         year = detected.get();
+     } else {
+         notes.add("제안서 연도를 찾지 못해 업로드 연도로 대체");
+         year = String.valueOf(now.getYear());
+     }
+
+     note(notes, inst);
+
+     ProposalDocument doc = new ProposalDocument(file.getOriginalName(), now.format(STAMP), slides);
+     String fileName = String.join("_", year, inst.getCategory(), inst.getName(), now.format(DAY))
+         + "_bid_proposal.json";
+
+     Path out = storageService.writeOutput(storageService.getProposalJsonDir(), fileName,
+         mapper.writeValueAsBytes(doc));
+
+

```

```

+     logNotes(file, notes);
+
+     file.markParsed(inst.getName(), inst.getCategory(), year, out.toString(), now);
+ }
+
+
+ private void parseRfp(UploadedFile file) throws Exception {
+     LocalDateTime now = LocalDateTime.now();
+
+     ExtractedDocument doc = rfpExtractor.extract(Paths.get(originalPath(file)));
+
+     List<String> lines = doc.lines();
+
+
+     InstitutionMatcher.Match inst = institutionMatcher.match(doc.fullText(), institutions());
+
+     List<String> notes = new ArrayList<String>();
+
+     Optional<String> notice = dateDetector.detectNoticeDate(lines);
+
+     if (!notice.isPresent()) notes.add("공고일을 찾지 못함");
+
+     note(notes, inst);
+
+
+     String markdown = rfpSummaryWriter.write(doc, new RfpSummaryWriter.Meta(
+
+         file.getOriginalName(), inst.getName(), inst.getCategory(),
+
+         notice.orElse(null), now.format(STAMP)));
+
+
+     String fileName = String.join("_", notice.orElse(UNKNOWN_NOTICE_DATE),
+
+         inst.getCategory(), inst.getName(), now.format(DAY)) + ".md";
+
+     Path out = storageService.writeOutput(storageService.getRfpMdDir(), fileName,
+
+         markdown.getBytes(StandardCharsets.UTF_8));
+
+
+     logNotes(file, notes);
+
+     file.markParsed(inst.getName(), inst.getCategory(), notice.orElse(null), out.toString(), now);
+ }
+
+
+ private List<Institution> institutions() {
+
+     return institutionMapper.findAll();
+
+ }
+
+
+ /**
+
+  * 감사 2월. 규칙(2026-09-08 확정)대로 <b>그 행을 마지막으로 건드린 시각</b>을 넣는다.
+
+  * 파싱은 분류일시가 없을 수도 있어 파싱일시를 쓴다.

```

```

+    */
+    private static void stampAudit(UploadedFile file) {
+        file.setSystemUserNo(SystemUser.get());
+        file.setSystemUsedAt(file.getParsedAt() != null ? file.getParsedAt() : LocalDateTime.now());
+    }
+
+    private static void note(List<String> notes, InstitutionMatcher.Match inst) {
+        if (InstitutionMatcher.UNKNOWN_NAME.equals(inst.getName())) {
+            notes.add("기관명을 찾지 못함");
+        } else if (!inst.isRegistered() && UploadedFile.UNKNOWN_CATEGORY.equals(inst.getCategory())) {
+            notes.add("기관 테이블에 없는 기관(" + inst.getName() + ") → 미분류");
+        }
+    }
+
+    /** 참고 사항(기관 미상 등)도 컬럼이 없어 로그에만 남긴다. */
+    private static void logNotes(UploadedFile file, List<String> notes) {
+        if (!notes.isEmpty()) {
+            log.info("파싱 참고: {} - {}", file.getOriginalName(), String.join(" / ", notes));
+        }
+    }
+
+    /**
+     * 원본 경로를 재구성한다. 파싱에 성공하면 저장경로가 산출물로 바뀌므로,
+     * 재파싱할 때 원본을 다시 찾으려면 {@code userdata + 원본파일명} 으로 계산해야 한다.
+     * ⚠ 그래서 업로드 때 <b>실제로 저장된 파일명</b>을 원본파일명에 넣는다(FileUploadService).
+     */
+    private String originalPath(UploadedFile file) {
+        return storageService.getUserdataDir().resolve(file.getOriginalName()).toString();
+    }
+}

```

삭제 service/FileParserService.java

deleted file mode 100644

@@ -1,38 +0,0 @@

-package com.kb.uploader.service;

```
-
-
-import com.kb.uploader.dto.ParsedFileName;
-
-import org.springframework.stereotype.Service;
-
-import java.util.Arrays;
-
-import java.util.HashSet;
-
-import java.util.Optional;
-
-import java.util.Set;
-
-
-@Service
-
-public class FileParserService {
-
-
-    private static final Set<String> ALLOWED_EXTENSIONS =
-
-        new HashSet<>(Arrays.asList("pdf", "hwp", "md"));
-
-
-    public Optional<ParsedFileName> parse(String filename) {
-
-        if (filename == null || filename.isEmpty()) return Optional.empty();
-
-
-        int dotIdx = filename.lastIndexOf('.');
-
-        if (dotIdx < 0) return Optional.empty();
-
-
-        String ext = filename.substring(dotIdx + 1).toLowerCase();
-
-        if (!ALLOWED_EXTENSIONS.contains(ext)) return Optional.empty();
-
-
-        String nameWithoutExt = filename.substring(0, dotIdx);
-
-        String[] tokens = nameWithoutExt.split("_", -1);
-
-
-        if (tokens.length < 3) return Optional.empty();
-
-        if (!tokens[0].matches("\\d{4}") return Optional.empty();
-
-
-        String year = tokens[0];
-
-        String institutionName = tokens[1];
-
-        String description = String.join("_",
-
-            Arrays.copyOfRange(tokens, 2, tokens.length));
-
-
-        return Optional.of(new ParsedFileName(year, institutionName, description, ext));
-
-    }
-}
```

```
- }
```

```
-}
```

수정 service/FileStorageService.java

```
@@ -5,8 +5,9 @@ import org.slf4j.LoggerFactory;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;
import java.io.IOException;
+import java.io.InputStream;

import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
@@ -18,11 +19,21 @@ public class FileStorageService {

    private static final DateTimeFormatter TS_FMT =
        DateTimeFormatter.ofPattern("yyyyMMddHHmmss");

    private final String baseDir;
+    // — 2026-09-23 파싱 전환: 원본 보관 / 산출물 경로 —
+
+    private final String userdataDir;
+
+    private final String proposalJsonDir;
+
+    private final String rfpMdDir;

-    public FileStorageService(@Value("${upload.base-dir}") String baseDir) {
+    public FileStorageService(@Value("${upload.base-dir}") String baseDir,
+
+        @Value("${upload.userdata-dir}") String userdataDir,
+
+        @Value("${upload.proposal-json-dir}") String proposalJsonDir,
+
+        @Value("${upload.rfp-md-dir}") String rfpMdDir) {

        this.baseDir = baseDir;
+    this.userdataDir = userdataDir;
+
+    this.proposalJsonDir = proposalJsonDir;
+
+    this.rfpMdDir = rfpMdDir;

    }

    public Path saveToUnclassified(MultipartFile file, String filename) throws IOException {
        Path dir = Paths.get(baseDir, "unclassified");
@@ -35,9 +46,14 @@ public class FileStorageService {

        ? filename.substring(0, dot) + "_" + ts + filename.substring(dot)
        : filename + "_" + ts;
        target = dir.resolve(filename);
    }
-    file.transferTo(target.toFile());

+    // transferTo(File)은 내부적으로 Part.write(경로)를 호출하는데, 서블릿 스펙상 그 경로는
+
+    // multipart 임시 위치 기준 상대경로다. WebLogic은 절대경로를 그대로 이어 붙여
+
+    // "<도메인>/.../C:/uploader-local/..." 같은 잘못된 경로를 만든다. 스트림 복사로 우회한다.
```

```

+     try (InputStream in = file.getInputStream()) {
+
+         Files.copy(in, target, StandardCopyOption.REPLACE_EXISTING);
+
+     }
+
+     return target;
+ }
+
+ public Path moveToClassified(Path source, String category,
@@ -93,5 +109,68 @@ public class FileStorageService {
+
+     public Path buildUnclassifiedPath(String filename) {
+         return Paths.get(baseDir, "unclassified", filename);
+     }
+
+
+ // — 2026-09-23 파싱 전환 —————
+
+ // 원본을 옮기지 않고 userdata 에 보관하고, 파싱 산출물을 따로 쓴다.
+
+
+ /** 원본 보관. 같은 이름이 있으면 _yyyyMMddHHmmss 를 붙인다. */
+
+     public Path saveOriginal(MultipartFile file, String filename) throws IOException {
+
+         Path dir = Paths.get(userdataDir);
+
+         Files.createDirectories(dir);
+
+         filename = safeFileName(filename);
+
+         Path target = dir.resolve(filename);
+
+         if (Files.exists(target)) {
+
+             int dot = filename.lastIndexOf('.');
+
+             String ts = LocalDateTime.now().format(TS_FMT);
+
+             filename = (dot >= 0)
+
+                 ? filename.substring(0, dot) + "_" + ts + filename.substring(dot)
+
+                 : filename + "_" + ts;
+
+             target = dir.resolve(filename);
+
+         }
+
+         // ⚠ transferTo(File) 을 쓰지 않는 이유는 saveToUnclassified 주석 참조(WebLogic 경로 문제).
+
+         try (InputStream in = file.getInputStream()) {
+
+             Files.copy(in, target, StandardCopyOption.REPLACE_EXISTING);
+
+         }
+
+         return target;
+
+     }
+
+
+
+ /** 산출물(JSON·MD)을 쓴다. 같은 이름이 있으면 _2, _3 ... 을 붙인다. */

```

```

+ public Path writeOutput(Path dir, String filename, byte[] content) throws IOException {
+     Files.createDirectories(dir);
+     filename = safeFileName(filename);
+     Path target = dir.resolve(filename);
+     int dot = filename.lastIndexOf('.');
+     String base = dot >= 0 ? filename.substring(0, dot) : filename;
+     String ext = dot >= 0 ? filename.substring(dot) : "";
+     for (int n = 2; Files.exists(target); n++) {
+         target = dir.resolve(base + "_" + n + ext);
+     }
+     Files.write(target, content);
+     return target;
+ }

```

```

+ /** 재파싱 때 이전 산출물을 지운다. 없으면 조용히 넘어간다. */

```

```

+ public void deleteIfExists(String path) throws IOException {
+     if (path == null || path.trim().isEmpty()) return;
+     Files.deleteIfExists(Paths.get(path));
+ }

```

```

+ /**

```

```

+ * 파일명에서 경로 구분자를 떼어 낸다.

```

```

+ * ⚠ 브라우저에 따라 전체 경로를 보내고, 악의적 요청은 "../" 를 섞는다.

```

```

+ * 그대로 resolve 하면 저장 디렉터리 밖으로 나간다.

```

```

+ */

```

```

+ public static String safeFileName(String name) {
+     if (name == null) return "unnamed";
+     String only = name.replace('\\', '/');
+     int slash = only.lastIndexOf('/');
+     if (slash >= 0) only = only.substring(slash + 1);
+     only = only.trim();
+     return only.isEmpty() ? "unnamed" : only;
+ }

```

```

+ public Path getUserdataDir() { return Paths.get(userdataDir); }

```

```

+ public Path getProposalJsonDir() { return Paths.get(proposalJsonDir); }

+ public Path getRfpMdDir() { return Paths.get(rfpMdDir); }

}

```

수정 service/FileUploadService.java

@@ -1,85 +1,82 @@

```

package com.kb.uploader.service;

+import com.kb.uploader.code.DocumentType;

import com.kb.uploader.code.SystemUser;
import com.kb.uploader.domain.UploadedFile;
- import com.kb.uploader.dto.ParsedFileName;

import com.kb.uploader.dto.UploadResultItem;
import com.kb.uploader.mapper.UploadedFileMapper;
+import org.slf4j.Logger;

+import org.slf4j.LoggerFactory;

import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;

import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
- import java.util.Optional;

+/**
+ * 업로드 오케스트레이션 (2026-09-23 파싱 전환).
+ *
+ * <p>종전에는 파일명을 해석해 기관을 알아낸 뒤 폴더를 옮겼다. 이제는 <b>확장자만 검증</b>하고
+ * 원본을 userdata 에 보관한 다음 {@link DocumentParseService} 가 문서 내용을 읽는다.
+ *
+ * <p>⚠ 한 파일이 실패해도 나머지는 계속 처리한다 - 여러 건을 한 번에 올리는 화면이라
+ * 중간에 멈추면 무엇이 처리됐는지 알 수 없다.
+ */

@Service
public class FileUploadService {

- private final FileParserService parserService;

+ private static final Logger log = LoggerFactory.getLogger(FileUploadService.class);

+

private final FileStorageService storageService;
- private final ClassificationService classificationService;

+ private final DocumentParseService parseService;

private final UploadedFileMapper fileMapper;

- public FileUploadService(FileParserService parserService,

```

```

-         FileStorageService storageService,
-         ClassificationService classificationService,
+     public FileUploadService(FileStorageService storageService,
+         DocumentParseService parseService,
+         UploadedFileMapper fileMapper) {
-         this.parserService = parserService;
-         this.storageService = storageService;
-         this.classificationService = classificationService;
+         this.parseService = parseService;
-         this.fileMapper = fileMapper;
-     }
-     public List<UploadResultItem> upload(List<MultipartFile> files) {
-         return upload(files, "", "", "");
-     }
-
-     public List<UploadResultItem> upload(List<MultipartFile> files,
-         String instOverride, String yearOverride, String categoryOverride) {
-         List<UploadResultItem> results = new ArrayList<>();
+     /**
+      * @param docType {@link DocumentType#BID_PROPOSAL} 또는 {@link DocumentType#RFP}
+      */
+     public List<UploadResultItem> upload(String docType, List<MultipartFile> files) {
+         List<UploadResultItem> results = new ArrayList<UploadResultItem>();
+         for (MultipartFile file : files) {
-             String originalName = file.getOriginalFilename();
+             String originalName = FileStorageService.safeFileName(file.getOriginalFilename());
+             if (file.isEmpty()
+                 && (file.getOriginalFilename() == null || file.getOriginalFilename().isEmpty())) {
+                 continue; // 파일을 고르지 않고 전송한 빈 파트
+             }
+             if (!DocumentType.accepts(docType, originalName)) {
+                 results.add(new UploadResultItem(originalName, false, null,
+                     "허용되지 않는 확장자 ("
+                     + String.join(", ", DocumentType.extensions(docType)) + "만 가능"));
+                 continue;
+             }
+         }
+         try {

```

```

- Optional<ParsedFileName> parsed = parserService.parse(originalName);
- if (!parsed.isPresent()) {
-     boolean hasOverride = instOverride != null && !instOverride.trim().isEmpty();
-     Path saved = storageService.saveToUnclassified(file, originalName);
-     UploadedFile entity = new UploadedFile(
-         originalName, saved.toString(),
-         (yearOverride != null && !yearOverride.trim().isEmpty()) ? yearOverride.trim() : null,
-         hasOverride ? instOverride.trim() : "알수없음");
-     entity.setSystemUserNo(SystemUser.get());
-     entity.setSystemUsedAt(entity.getUploadedAt());
-     fileMapper.insert(entity);
-     if (hasOverride) {
-         boolean ok = classificationService.classify(entity);
-         results.add(new UploadResultItem(originalName, ok,
-             ok ? entity.getCategory() : null,
-             ok ? "수동 입력으로 분류 완료" : "미분류 (기관 미등록 - 기관 관리에서 등록 필요)"));
-     } else {
-         results.add(new UploadResultItem(originalName, false, null,
-             "파일명 형식 불일치 (년도_기관명_설명.확장자 필요)"));
-     }
-     continue;
- }
- ParsedFileName p = parsed.get();
- Path saved = storageService.saveToUnclassified(file, originalName);
- UploadedFile entity = new UploadedFile(
-     originalName, saved.toString(), p.getYear(), p.getInstitutionName());
+ Path saved = storageService.saveOriginal(file, originalName);
+ // ⚠ 중복 이름이면 _yyyyMMddHHmmss 가 붙는다. **실제로 저장된 이름**을 넣어야
+ // 나중에 userdata + 원본파일명 으로 원본을 다시 찾을 수 있다(스키마 무변경안).
+ String savedName = saved.getFileName().toString();
+ UploadedFile entity = new UploadedFile(savedName, saved.toString());
-     entity.setSystemUserNo(SystemUser.get());
-     entity.setSystemUsedAt(entity.getUploadedAt());
-     fileMapper.insert(entity);
+ entity.setSystemUsedAt(entity.getUploadedAt());
+ fileMapper.insert(entity);

```

```

-         boolean ok = classificationService.classify(entity);

-         results.add(new UploadResultItem(
-             originalName, ok,
-             ok ? entity.getCategory() : null,
-             ok ? "분류 완료" : "미분류 (기관 미등록)"));

+         UploadedFile parsed = parseService.parse(entity);
+         boolean ok = parsed.isParsed();
+         results.add(new UploadResultItem(originalName, ok,
+             ok ? parsed.getCategoryLabel() : null,
+             ok ? "파싱 완료 → " + parsed.getOutputFileName()
+             : "파싱 실패 - 사유는 서버 로그를 확인하세요"));

        } catch (Exception e) {
-         results.add(new UploadResultItem(originalName, false, null,
-             "오류: " + e.getMessage()));
+         log.error("업로드 처리 실패: {}", originalName, e);
+         results.add(new UploadResultItem(originalName, false, null, "오류: " + e.getMessage()));
        }
    }
    return results;
}

```

수정 service/InstitutionService.java

```

@@ -21,16 +21,13 @@ public class InstitutionService {

    private static final ObjectMapper MAPPER = new ObjectMapper();

    private final InstitutionMapper institutionMapper;
-    private final ClassificationService classificationService;

    private final UploadedFileMapper uploadedFileMapper;

    public InstitutionService(InstitutionMapper institutionMapper,
-        ClassificationService classificationService,

        UploadedFileMapper uploadedFileMapper) {
        this.institutionMapper = institutionMapper;
-        this.classificationService = classificationService;

        this.uploadedFileMapper = uploadedFileMapper;
    }

    public List<Institution> findAll() {
@@ -73,10 +70,10 @@ public class InstitutionService {

        Institution created = new Institution(name, category);
        created.setSystemUserNo(SystemUser.get());
        institutionMapper.insert(created);
    }

-    List<UploadedFile> pending = uploadedFileMapper.findByStatus("UNCLASSIFIED");

```

```

- pending.forEach(f -> classificationService.classify(f));

+ // 2026-09-23: 자동 재분류를 없앴다. 기관을 고친 뒤 다시 돌리려면

+ // 파싱 현황 화면의 [재파싱] 을 누른다.

}

public byte[] exportToXlsx() throws IOException {
    try (Workbook wb = new XSSFWorkbook()) {
@@ -116,8 +113,8 @@ public class InstitutionService {

        institutionMapper.insert(created);
    }
}

- List<UploadedFile> pending = uploadedFileMapper.findByStatus("UNCLASSIFIED");

- pending.forEach(f -> classificationService.classify(f));

+ // 2026-09-23: 자동 재분류를 없앴다. 기관을 고친 뒤 다시 돌리려면

+ // 파싱 현황 화면의 [재파싱] 을 누른다.

}
}

```

수정 mapper/InstitutionMapper.xml

```

@@ -25,8 +25,10 @@

    <result property="category"    column="기관영업분류구분"
            typeHandler="com.kb.uploader.config.InstitutionCategoryTypeHandler"/>
    <result property="modifiedAt"  column="시스템사용일시"
            typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
+ <!-- ⚠ UploadedFileMapper 의 같은 자리와 같은 이유다(2026-09-09) - 읽기 누락. -->
+
+ <result property="systemUserNo" column="시스템사용자번호"/>

</resultMap>

<sql id="값목록">
    #{name},

```

수정 mapper/UploadedFileMapper.xml

```

@@ -45,21 +45,44 @@

            typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
    <result property="classifiedAt"    column="분류일시"
            typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
    <result property="storedPath"     column="저장경로내용"/>
+ <!--

+ ⚠ 감사 2열은 **읽기에도 반드시 있어야 한다**(2026-09-09 추가). 종전에는 쓰기

+ (`#{...}`)만 있고 여기가 빠져 있어 DB 에는 값이 들어가는데 조회하면 늘

+ null 이었다 - 화면이 비어 보이는 게 첫 증상이다. 더 나쁜 건

+ KGI12100$FileStatusClassify 처럼 **findById 로 읽어 그대로 update 하는 경로**

+ 에서 그 null 이 DB 로 되써진다는 점이다.

+ -->

```

```

+ <result property="systemUsedAt"    column="시스템사용일시"
+
+     typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
+
+ <result property="systemUserNo"    column="시스템사용자번호"/>
+
+ </resultMap>
+
+ <!-- 파라미터 쪽 TypeHandler 표기가 길어 sql 조각으로 묶는다 -->
+ <sql id="값목록">
+     #{institutionName},
+     #{category,typeHandler=com.kb.uploader.config.InstitutionCategoryTypeHandler},
+
+     <include refid="기관분류값"/>,
+
+     #{year},
+     #{status,typeHandler=com.kb.uploader.config.ClassificationStatusTypeHandler},
+     #{originalName},
+     #{uploadedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
+     #{storedPath}
+ </sql>
+
+ <!--
+
+     ⚠ 파싱 전환(2026-09-23) 이후 기관분류가 **미분류** 일 수 있다. 그 값은
+
+     code/InstitutionCategory 의 코드표(01~05)에 없어서 TypeHandler 가 예외를 던진다.
+
+     DDL 의 CHECK 제약도 코드 5종만 받는다. 그래서 **미분류는 NULL 로 저장**한다.
+
+     화면 표기는 UploadedFile.getCategoryLabel() 이 NULL 을 "미분류" 로 되돌린다.
+
+ -->
+
+ <sql id="기관분류값">
+
+     <choose>
+
+         <when test="category == null or category == '미분류'">NULL</when>
+
+         <otherwise>#{category,typeHandler=com.kb.uploader.config.InstitutionCategoryTypeHandler}</otherwise>
+
+     </choose>
+
+ </sql>
+
+
+ <!--
+
+     ⚠ 같은 id 가 두 번 나오는 자리는 **방언 분기**다(databaseId). 접속한 DB 에 맞는
+     쪽만 로드된다 - config/MyBatisConfig 가 값을 정한다.
+     한쪽만 고치면 다른 DB 에서 조용히 낡는다. **항상 둘을 같이 고칠 것.**
+
+ @@ -92,16 +115,36 @@
+
+     SET
+     저장경로내용      = #{storedPath},
+     기관명           = #{institutionName},
+     문서년           = #{year},
+     기관영업분류구분 = #{category,typeHandler=com.kb.uploader.config.InstitutionCategoryTypeHandler},
+
+     기관영업분류구분 = <include refid="기관분류값"/>,
+
+     분류상태구분     = #{status,typeHandler=com.kb.uploader.config.ClassificationStatusTypeHandler},
+     분류일시         = #{classifiedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
+     시스템사용일시   = #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
+     시스템사용자번호 = #{systemUserNo}
+     WHERE 업로드파일일련번호 = #{id}

```

```

</update>

+ <!--

+   파싱 결과 기록 (2026-09-23 신설, 스키마 무변경안).

+   ⚠ 새 컬럼이 없으므로 **기존 컬럼에 태운다**:

+       저장경로내용 = 산출물 경로(성공) 또는 원본 경로(실패) · 분류일시 = 파싱 시각 ·

+       문서년 = 제안서 연도. 실패 사유는 널을 자리가 없어 로그에만 남는다.

+   -->

+ <update id="updateParseResult">

+   UPDATE TSKGIAF01

+   SET

+   저장경로내용      = #{storedPath},

+   기관명            = #{institutionName},

+   문서년            = #{year},

+   기관영업분류구분 = <include refid="기관분류값"/>,

+   분류상태구분      = #{status,typeHandler=com.kb.uploader.config.ClassificationStatusTypeHandler},

+   분류일시          = #{classifiedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},

+   시스템사용일시    = #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},

+   시스템사용자번호 = #{systemUserNo}

+   WHERE 업로드파일일련번호 = #{id}

+ </update>

+

+ <!--

+   ⚠ '04' = DELETED, '03' = REJECTED, '02' = CLASSIFIED (code/ClassificationStatus.java)

+   ⚠ 이 둘은 파라미터에 시각 값이 없어 **컨트롤러가 LocalDateTime.now() 를 넘긴다.**

+   DB 함수(TO_CHAR(SYSDATE)/DATE_FORMAT(NOW()))를 쓰면 방언이 갈리고 테스트로

+   @@ -160,8 +203,9 @@

+

+ <!-- MySQL: LIMIT -->

+ <select id="findRecent" databaseId="mysql" resultMap="uploadedFileMap">

+   SELECT * FROM TSKGIAF01 ORDER BY 업로드파일일련번호 DESC LIMIT #{limit}

+ </select>

+

+ <!-- Oracle: ROWNUM 은 ORDER BY 보다 먼저 매겨지므로 정렬을 인라인 뷰로 감싼다 -->

+ <select id="findRecent" databaseId="oracle" resultMap="uploadedFileMap">

+   SELECT * FROM (

+     SELECT * FROM TSKGIAF01 ORDER BY 업로드파일일련번호 DESC

+   )

+   WHERE 기관명 LIKE CONCAT('%', #{keyword}, '%')

+   ORDER BY 업로드일시 DESC

+   LIMIT #{limit} OFFSET #{offset}

+ </select>

+

+ <!-- Oracle: ROWNUM 2단 중첩 페이징. 안쪽에서 상한, 바깥에서 하한을 건다 -->

+ <select id="findByInstitutionNameContaining" databaseId="oracle" resultMap="uploadedFileMap">

```

```

SELECT * FROM (
  SELECT a.*, ROWNUM RN FROM (
@@ -198,8 +243,9 @@

<select id="countByInstitutionNameContaining" databaseId="mysql" resultType="long">
  SELECT COUNT(*) FROM TSKGIAF01
  WHERE 기관명 LIKE CONCAT('%', #{keyword}, '%')
</select>

```

```

+

<!-- Oracle: || 로 연결 -->
<select id="countByInstitutionNameContaining" databaseId="oracle" resultType="long">
  SELECT COUNT(*) FROM TSKGIAF01
  WHERE 기관명 LIKE '%' || #{keyword} || '%'
@@ -213,8 +259,9 @@

```

```

  AND (#{year} = '' OR 문서년 = #{year})
  AND (#{keyword} = '' OR 원본파일명 LIKE CONCAT('%', #{keyword}, '%'))
  ORDER BY 업로드일시 DESC
</select>
+

```

```

<!--
Oracle: || 로 연결.
⚠ Oracle 은 빈 문자열을 NULL 로 바꾸므로 `#{x} = ''` 는 **항상 거짓**이다.
"빈 값이면 조건을 걸지 않는다"는 원래 뜻을 살리려면 NULL 검사여야 한다.
@@ -228,5 +275,129 @@

```

```

  AND (#{keyword} IS NULL OR 원본파일명 LIKE '%' || #{keyword} || '%')
  ORDER BY 업로드일시 DESC
</select>

```

```

+ <!-- == 파싱 현황·대시보드·조회 API (2026-09-23, 스키마 무변경안) ==

```

```

+ ⚠ 컬럼을 늘리지 않았으므로 **파일명 규칙으로 판별**한다. 규칙 정보는

```

```

+ code/DocumentType (ofFileName · isOutputPath) 이고, 아래 조각이 그 SQL 판이다.

```

```

+ 한쪽을 고치면 반드시 다른 쪽도 고칠 것 - 어긋나면 화면 숫자와 목록이 따로 논다.

```

```

+ · 파싱 성공 = 저장경로내용이 .json 또는 .md (= 산출물이 만들어졌다)

```

```

+ · 파싱 실패 = 그 밖 (저장경로가 원본 그대로다)

```

```

+ · 제안서 = 원본파일명이 .ppt / .pptx

```

```

+ · RFP = 원본파일명이 .pdf / .hwp / .hwpX

```

```

+ ⚠ 삭제(04)·반려(03) 건은 파싱 집계에서 뺀다. 사람이 처리를 끝낸 행이다.

```

```

+ -->

```

```

+ <sql id="파싱성공">(저장경로내용 LIKE '%.json' OR 저장경로내용 LIKE '%.md')</sql>

```

```

+ <sql id="제안서원본">(원본파일명 LIKE '%.ppt' OR 원본파일명 LIKE '%.pptx')</sql>

```

```

+ <sql id="RFP원본">(원본파일명 LIKE '%.pdf' OR 원본파일명 LIKE '%.hwp' OR 원본파일명 LIKE '%.hwpX')</sql>

```

```

+ <sql id="파싱대상">(include refid="제안서원본"/> OR <include refid="RFP원본"/>)</sql>

```

```

+ <sql id="처리중">분류상태구분 NOT IN ('03', '04')</sql>

```

```

+

```

```

+ <select id="countParseSuccess" resultType="long">

```

```

+ SELECT COUNT(*) FROM TSKGIAF01

```

```

+ WHERE <include refid="처리중"/> AND <include refid="파싱성공"/>

```

```

+ </select>
+
+ <select id="countParseFailed" resultType="long">
+
+   SELECT COUNT(*) FROM TSKGIAF01
+
+   WHERE <include refid="처리중"/> AND <include refid="파싱대상"/> AND NOT <include refid="파싱성공"/>
+ </select>
+
+
+ <select id="countProposalDocs" resultType="long">
+
+   SELECT COUNT(*) FROM TSKGIAF01 WHERE <include refid="처리중"/> AND <include refid="제안서원본"/>
+ </select>
+
+
+ <select id="countRfpDocs" resultType="long">
+
+   SELECT COUNT(*) FROM TSKGIAF01 WHERE <include refid="처리중"/> AND <include refid="RFP원본"/>
+ </select>
+
+
+ <!-- 파싱은 됐는데 기관을 못 찾은 건. 기관분류 NULL 이 곧 "미분류" 다 -->
+
+ <select id="countParsedUnclassified" resultType="long">
+
+   SELECT COUNT(*) FROM TSKGIAF01
+
+   WHERE <include refid="처리중"/> AND <include refid="파싱성공"/> AND 기관영업분류구분 IS NULL
+ </select>
+
+
+ <select id="searchParseStatus" resultMap="uploadedFileMap">
+
+   SELECT * FROM TSKGIAF01
+
+   WHERE <include refid="처리중"/>
+
+   <if test="docType == 'BID_PROPOSAL'">AND <include refid="제안서원본"/></if>
+
+   <if test="docType == 'RFP'">AND <include refid="RFP원본"/></if>
+
+   <if test="parseStatus == 'SUCCESS'">AND <include refid="파싱성공"/></if>
+
+   <if test="parseStatus == 'FAILED'">AND <include refid="파싱대상"/> AND NOT <include refid="파싱성공"/></if>
+
+   <if test="unclassifiedOnly != null and unclassifiedOnly">AND 기관영업분류구분 IS NULL</if>
+
+   ORDER BY 업로드파일일련번호 DESC
+ </select>
+
+
+ <!-- — 조회 API. 파싱 성공한 것만 준다 —————
+
+   ⚠ 날짜 조건은 **산출물 파일명 앞 토큰**을 본다(경로에 파일명이 들어 있다).
+
+   `%20240315\__%` 처럼 밑줄을 escape 해야 한다 - LIKE 에서 _ 는 한 글자 와일드카드다.

```

```

+ -->

+ <sql id="날짜조건mysql">

+   <if test="docDate != null">AND 저장경로내용 LIKE CONCAT('%', #{docDate}, '%') ESCAPE '\\'\</if>

+ </sql>

+ <sql id="날짜조건oracle">

+   <if test="docDate != null">AND 저장경로내용 LIKE '%' || #{docDate} || '% ' ESCAPE '\\'\</if>

+ </sql>

+ <sql id="기관조건mysql">

+   <if test="institution != null">AND 기관명 LIKE CONCAT('%', #{institution}, '%')</if>

+ </sql>

+ <sql id="기관조건oracle">

+   <if test="institution != null">AND 기관명 LIKE '%' || #{institution} || '%'\</if>

+ </sql>

+

+ <select id="searchProposals" databaseId="mysql" resultMap="uploadedFileMap">

+   SELECT * FROM TSKGIAF01

+   WHERE <include refid="처리중"/> AND <include refid="제안서원본"/> AND <include refid="파싱성공"/>

+   <include refid="기관조건mysql"/><include refid="날짜조건mysql"/>

+   ORDER BY 업로드파일일련번호 DESC

+   LIMIT #{limit} OFFSET #{offset}

+ </select>

+ <select id="searchProposals" databaseId="oracle" resultMap="uploadedFileMap">

+   SELECT * FROM (

+     SELECT t.*, ROWNUM rnum FROM (

+       SELECT * FROM TSKGIAF01

+       WHERE <include refid="처리중"/> AND <include refid="제안서원본"/> AND <include refid="파싱성공"/>

+       <include refid="기관조건oracle"/><include refid="날짜조건oracle"/>

+       ORDER BY 업로드파일일련번호 DESC

+     ) t WHERE ROWNUM <= #{offset} + #{limit}

+   ) WHERE rnum > #{offset}

+ </select>

+ <select id="countProposals" databaseId="mysql" resultType="long">

+   SELECT COUNT(*) FROM TSKGIAF01

+   WHERE <include refid="처리중"/> AND <include refid="제안서원본"/> AND <include refid="파싱성공"/>

+   <include refid="기관조건mysql"/><include refid="날짜조건mysql"/>

```

```

+ </select>

+ <select id="countProposals" databaseId="oracle" resultType="long">
+   SELECT COUNT(*) FROM TSKGIAF01
+   WHERE <include refid="처리중"/> AND <include refid="제안서원본"/> AND <include refid="파싱성공"/>
+   <include refid="기관조건oracle"/><include refid="날짜조건oracle"/>
+ </select>

+

+ <select id="searchRfps" databaseId="mysql" resultMap="uploadedFileMap">
+   SELECT * FROM TSKGIAF01
+   WHERE <include refid="처리중"/> AND <include refid="RFP원본"/> AND <include refid="파싱성공"/>
+   <include refid="기관조건mysql"/><include refid="날짜조건mysql"/>
+   ORDER BY 업로드파일일련번호 DESC
+   LIMIT #{limit} OFFSET #{offset}
+ </select>

+ <select id="searchRfps" databaseId="oracle" resultMap="uploadedFileMap">
+   SELECT * FROM (
+     SELECT t.*, ROWNUM rnum FROM (
+       SELECT * FROM TSKGIAF01
+       WHERE <include refid="처리중"/> AND <include refid="RFP원본"/> AND <include refid="파싱성공"/>
+       <include refid="기관조건oracle"/><include refid="날짜조건oracle"/>
+       ORDER BY 업로드파일일련번호 DESC
+     ) t WHERE ROWNUM &lt;= #{offset} + #{limit}
+   ) WHERE rnum &gt; #{offset}
+ </select>

+ <select id="countRfps" databaseId="mysql" resultType="long">
+   SELECT COUNT(*) FROM TSKGIAF01
+   WHERE <include refid="처리중"/> AND <include refid="RFP원본"/> AND <include refid="파싱성공"/>
+   <include refid="기관조건mysql"/><include refid="날짜조건mysql"/>
+ </select>

+ <select id="countRfps" databaseId="oracle" resultType="long">
+   SELECT COUNT(*) FROM TSKGIAF01
+   WHERE <include refid="처리중"/> AND <include refid="RFP원본"/> AND <include refid="파싱성공"/>
+   <include refid="기관조건oracle"/><include refid="날짜조건oracle"/>
+ </select>
+

```

```
</mapper>
```

수정 templates/classified-rework.html

```
@@ -46,16 +46,15 @@
```

```
    <thead>
    <tr>
      <th style="width:28%">파일명</th>
      <th style="width:16%">기관명</th>
-      <th style="width:7%">연도</th>
-
-      <th style="width:10%">카테고리</th>
-
-      <th style="width:39%">재분류</th>
+      <th style="width:9%">연도/공고일</th>
+      <th style="width:10%">기관분류</th>
+      <th style="width:37%">기관 지정 후 재파싱</th>
    </tr>
    </thead>
    <tbody>
-    <tr th:each="file : ${classifiedFiles}"
-      th:style="${(file.institutionName == '알수없음' or file.year == null or file.year == '미확인') ? 'background:rgb(220,38,38,.03)' : ''}">
+    <tr th:each="file : ${classifiedFiles}" style="background:rgba(220,38,38,.03)">
+
+      <!-- 파일명 -->
+      <td>
+        <span th:with="ext=${#strings.toLowerCase(file.originalName.substring(file.originalName.lastIndexOf('.') +
1))}"
+          th:class="${'file-ext' + (ext == 'pdf' ? ' ext-pdf' : (ext == 'hwp' ? ' ext-hwp' : (ext == 'md' ? ' ext
-md' : '')))}"
-      </td>
```

```
@@ -63,23 +62,19 @@
```

```
        <span class="file-name" th:text="${file.originalName}"></span>
      </td>
      <!-- 기관명 -->
      <td>
-        <span th:if="${file.institutionName == '알수없음'}"
-
-          style="color:var(--red);font-weight:600" th:text="${file.institutionName}"></span>
-
-        <span th:if="${file.institutionName != '알수없음'}"
-
-          style="color:var(--text-2)" th:text="${file.institutionName}"></span>
+        <span style="color:var(--red);font-weight:600"
+
+          th:text="${file.institutionName != null ? file.institutionName : '기관미상'}"></span>
      </td>
      <!-- 연도 -->
      <td>
-        <span th:if="${file.year == null or file.year == '미확인'}"
-
-          style="color:var(--amber);font-weight:600">미확인</span>
-
-        <span th:if="${file.year != null and file.year != '미확인'}"
-
-          style="color:var(--text-2)" th:text="${file.year}"></span>
```

```

+       <span th:if="${file.docDate == null}" style="color:var(--amber);font-weight:600">미확인</span>
+       <span th:if="${file.docDate != null}" style="color:var(--text-2)" th:text="${file.docDate}"></span>
+
+       </td>
+       <!-- 현재 카테고리 -->
+       <td>
+           <span class="category-chip" th:text="${file.category}"></span>
+
+           <span class="category-chip" th:text="${file.categoryLabel}"></span>
+
+       </td>
+       <!-- 재분류 폼 -->
+       <td>
+           <form class="rework-form"
@@ -88,17 +83,14 @@
+
+           <select name="category" class="form-control form-control-sm" style="width:110px" required>
+               <option th:each="cat : ${categories}"
+                   th:value="${cat}"
+                   th:text="${cat}"
+                   th:selected="${cat == file.category}"></option>
+
+               <option th:selected="${cat == file.categoryLabel}"></option>
+
+           </select>
+           <input type="text" name="institution" class="form-control form-control-sm"
+               style="width:130px" placeholder="기관명"
+               th:value="${file.institutionName}" required>
+
+           <input type="text" name="year" class="form-control form-control-sm"
+
+               style="width:64px" placeholder="연도" maxlength="4"
+
+               th:value="${file.year}">
+
+           <button type="submit" class="btn btn-primary btn-sm">재분류</button>
+
+               th:value="${file.institutionName != '기관미상' ? file.institutionName : ''}" required>
+
+           <button type="submit" class="btn btn-primary btn-sm">등록 후 재파싱</button>
+
+           <span class="rework-result" style="display:none;font-size:12px;font-weight:600"></span>
+       </form>
+   </td>
+</tr>

```

수정 templates/dashboard.html

```

@@ -15,9 +15,9 @@
+
+   </div>
+   <div class="kpi-body">
+       <div class="kpi-label">전체 파일</div>
+       <div class="kpi-value" th:text="${total}">0</div>
+       <div class="kpi-sub">누적 업로드</div>
+
+       <div class="kpi-sub" th:text="' 제안서 ' + ${proposalCount} + ' · RFP ' + ${rfpCount}">누적 업로드</div>
+
+   </div>
+</div>
+
+   <div class="kpi-card">
@@ -26,11 +26,11 @@
+
+       <polyline points="20 6 9 17 4 12"/>
+   </svg>

```

```

    </div>
    <div class="kpi-body">
-      <div class="kpi-label">분류 완료</div>
-
-      <div class="kpi-value success" th:text="${classified}">0</div>
-
-      <div class="kpi-sub">기관 분류됨</div>
+
+      <div class="kpi-label">파싱 성공</div>
+
+      <div class="kpi-value success" th:text="${parseSuccess}">0</div>
+
+      <div class="kpi-sub">산출물 생성됨</div>

    </div>
  </div>

  <div class="kpi-card">
@@ -41,11 +41,11 @@

    <line x1="12" y1="16" x2="12.01" y2="16"/>
    </svg>
  </div>
  <div class="kpi-body">
-    <div class="kpi-label">미분류</div>
-
-    <div th:class="${'kpi-value' + (unclassified > 0 ? ' danger' : '')}" th:text="${unclassified}">0</div>
-
-    <div class="kpi-sub">즉시 처리 필요</div>
+
+    <div class="kpi-label">파싱 실패</div>
+
+    <div th:class="${'kpi-value' + (parseFailed > 0 ? ' danger' : '')}" th:text="${parseFailed}">0</div>
+
+    <div class="kpi-sub">재파싱 필요</div>

  </div>
</div>

<div class="kpi-card">
@@ -55,24 +55,24 @@

    <line x1="4.93" y1="4.93" x2="19.07" y2="19.07"/>
    </svg>
  </div>
  <div class="kpi-body">
-    <div class="kpi-label">반려 / 삭제</div>
-
-    <div class="kpi-value" style="color:var(--text-3,#999)" th:text="${excluded}">0</div>
-
-    <div class="kpi-sub">처리 제외됨</div>
+
+    <div class="kpi-label">기관 미분류</div>
+
+    <div class="kpi-value" style="color:var(--text-3,#999)" th:text="${unclassified}">0</div>
+
+    <div class="kpi-sub">기관 등록 후 재파싱</div>

  </div>
</div>
</div>

<!-- 경고 배너 (미분류 > 0 일 때만) -->
- <div th:if="${unclassified > 0}" class="alert-banner">
+ <div th:if="${parseFailed > 0}" class="alert-banner">

  <svg class="icon" style="width:15px;height:15px;flex-shrink:0" viewBox="0 0 24 24">
    <path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z"/>

```

```

<line x1="12" y1="9" x2="12" y2="13"/>
<line x1="12" y1="17" x2="12.01" y2="17"/>
</svg>
-   미분류 파일 <strong th:text="${unclassified}"></strong>건이 처리 대기 중입니다.

-   <a th:href="@{/file-status}" style="font-weight:700;margin-left:4px;color:inherit;text-decoration:underline">파일 처리
상태 바로가기</a>

+   파싱 실패 <strong th:text="${parseFailed}"></strong>건이 있습니다.

+   <a th:href="@{/parse-status(parseStatus='FAILED')}" style="font-weight:700;margin-left:4px;color:inherit;text-decorat
ion:underline">파싱 현황 바로가기</a>

</div>

<!-- 검색 -->
<div class="section-header">
@@ -108,10 +108,9 @@

        <span class="file-name" th:text="${f.originalName}"></span>
</td>
<td th:text="${f.institutionName}"></td>
<td>
-       <span th:if="${f.category != null}" class="category-chip" th:text="${f.category}"></span>

-       <span th:if="${f.category == null}" style="color:var(--text-3)">--</span>

+       <span class="category-chip" th:text="${f.categoryLabel}"></span>

</td>
<td>
    <span th:if="${f.status == 'CLASSIFIED'}" class="badge badge-done">분류 완료</span>
    <span th:if="${f.status == 'UNCLASSIFIED'}" class="badge badge-pending">미분류</span>
@@ -161,10 +160,9 @@

        <span class="file-name" th:text="${f.originalName}"></span>
</td>
<td th:text="${f.institutionName}"></td>
<td>
-       <span th:if="${f.category != null}" class="category-chip" th:text="${f.category}"></span>

-       <span th:if="${f.category == null}" style="color:var(--text-3)">--</span>

+       <span class="category-chip" th:text="${f.categoryLabel}"></span>

</td>
<td>
    <span th:if="${f.status == 'CLASSIFIED'}" class="badge badge-done">분류 완료</span>
    <span th:if="${f.status == 'UNCLASSIFIED'}" class="badge badge-pending">미분류</span>

```

삭제 templates/file-status.html

deleted file mode 100644

@@ -1,150 +0,0 @@

```

<!DOCTYPE html>

<html xmlns:th="http://www.thymeleaf.org"

-   th:replace="layout :: layout(~{::title}, '파일 처리 상태', ~{::section})">

<head><title>파일 처리 상태</title></head>

<body>

<section>

```

```
-
- <!-- 페이지 타이틀 + 미분류 배지 -->
- <div class="section-header" style="margin-bottom:20px">
-   <div style="display:flex;align-items:center;gap:10px">
-       <span class="section-title" style="font-size:22px">파일 미처리 상태</span>
-       <span class="badge badge-pending" th:text="${unclassifiedCount} + '건 미분류'"></span>
-   </div>
- </div>
-
- <!-- 완료/오류 메시지 -->
- <div th:if="${message}" class="alert-success" th:text="${message}"></div>
-
- <!-- 파일 없음 -->
- <div th:if="${files.isEmpty()}"
-     style="padding:40px;text-align:center;color:var(--text-3);background:var(--surface-2,#f8f9fa);border-radius:8px;border:1px dashed var(--border,#e0e0e0)">
-     미처리 파일이 없습니다.
- </div>
-
- <!-- 미처리 파일 목록 -->
- <div th:if="${!files.isEmpty()}" class="table-card">
-     <table>
-         <thead>
-             <tr>
-                 <th style="width:32%">파일명</th>
-                 <th style="width:14%">기관명</th>
-                 <th style="width:6%">연도</th>
-                 <th style="width:28%">분류</th>
-                 <th style="width:20%"></th>
-             </tr>
-         </thead>
-         <tbody>
-             <tr th:each="file : ${files}">
-                 <!-- 파일명 -->
-                 <td>
```

```

-         <span th:with="ext=${#strings.toLowerCase(file.originalName.substring(file.originalName.lastIndexOf('.') +
1)))}"
-             th:class="${'file-ext' + (ext == 'pdf' ? ' ext-pdf' : (ext == 'hwp' ? ' ext-hwp' : (ext == 'md' ? ' ext
-md' : '')))}"
-             th:text="${#strings.toUpperCase(ext)}"></span>
-         <span class="file-name" th:text="${file.originalName}"></span>
-     </td>
-     <!-- 기관명 -->
-     <td style="color:var(--text-2)" th:text="${file.institutionName}"></td>
-     <!-- 연도 -->
-     <td style="color:var(--text-2)" th:text="${file.year != null ? file.year : '-'}"></td>
-     <!-- 분류 품 -->
-     <td>
-         <form th:action="@{/file-status/{id}/classify(id=${file.id})}" method="post"
-             style="display:flex;gap:6px;align-items:center;flex-wrap:wrap">
-             <select name="category" class="form-control form-control-sm" style="width:110px" required>
-                 <option value="">카테고리</option>
-                 <option th:each="cat : ${categories}" th:value="${cat}" th:text="${cat}"></option>
-             </select>
-             <input type="text" name="institution" class="form-control form-control-sm"
-                 style="width:120px" placeholder="기관명 (선택)"
-                 th:value="${file.institutionName}">
-             <input type="text" name="year" class="form-control form-control-sm"
-                 style="width:64px" placeholder="연도"
-                 maxLength="4"
-                 th:value="${file.year}">
-             <button type="submit" class="btn btn-primary btn-sm">분류</button>
-         </form>
-     </td>
-     <!-- 반려 / 삭제 -->
-     <td style="text-align:right">
-         <div style="display:flex;gap:4px;justify-content:flex-end">
-             <form th:action="@{/file-status/{id}/reject(id=${file.id})}" method="post">
-                 <button type="submit" class="btn btn-outline btn-sm"
-                     onclick="return confirm('반려 처리하시겠습니까?')">반려</button>
-             </form>

```

```

-         <form th:action="@{/file-status/{id}/delete(id=${file.id})}" method="post">
-
-             <button type="submit" class="btn btn-danger-outline btn-sm"
-
-                 onclick="return confirm('목록에서 완전히 제거하시겠습니까?')">삭제</button>
-
-         </form>
-
-     </div>
-
- </td>
-
- </tr>
-
- </tbody>
-
- </table>
-
- </div>
-
-
- <!-- 알수없음 기관 재분류 섹션 -->
-
- <div th:if="${!unknownFiles.isEmpty()}" style="margin-top:36px">
-
-     <div class="section-header" style="margin-bottom:12px">
-
-         <div style="display:flex;align-items:center;gap:10px">
-
-             <span class="section-title" style="font-size:16px">기관 미확인 파일 재분류</span>
-
-             <span class="badge badge-pending" th:text="${#lists.size(unknownFiles)} + '건'"></span>
-
-         </div>
-
-         <span style="font-size:11px;color:var(--text-3)">&ldquo;알수없음&rdquo; 기관으로 분류된 파일 – 올바른 기관명과 카테고리를 지정하면 재분류됩니다.</span>
-
-     </div>
-
-     <div class="table-card">
-
-         <table>
-
-             <thead>
-
-                 <tr>
-
-                     <th style="width:36%">파일명</th>
-
-                     <th style="width:7%">연도</th>
-
-                     <th style="width:46%">재분류</th>
-
-                     <th style="width:11%"></th>
-
-                 </tr>
-
-             </thead>
-
-             <tbody>
-
-                 <tr th:each="file : ${unknownFiles}">
-
-                     <td>
-
-                         <span th:with="ext=${#strings.toLowerCase(file.originalName.substring(file.originalName.lastIndexOf('.') + 1))}"

```

```

-         th:class="${'file-ext' + (ext == 'pdf' ? ' ext-pdf' : (ext == 'hwp' ? ' ext-hwp' : (ext == 'md' ? ' e
xt-md' : '')))}"
-
-         th:text="${#strings.toUpperCase(ext)}"</span>
-
-         <span class="file-name" th:text="${file.originalName}"></span>
-
-     </td>
-
-     <td style="color:var(--text-2)" th:text="${file.year != null ? file.year : '-'}"></td>
-
-     <td>
-
-         <form th:action="@{/file-status/{id}/classify(id=${file.id})}" method="post"
-
-             style="display:flex;gap:6px;align-items:center;flex-wrap:wrap">
-
-             <select name="category" class="form-control form-control-sm" style="width:110px" required>
-
-                 <option value="">카테고리</option>
-
-                 <option th:each="cat : ${categories}" th:value="${cat}" th:text="${cat}"></option>
-
-             </select>
-
-             <input type="text" name="institution" class="form-control form-control-sm"
-
-                 style="width:130px" placeholder="기관명 (필수)" required>
-
-             <input type="text" name="year" class="form-control form-control-sm"
-
-                 style="width:64px" placeholder="연도"
-
-                 maxlength="4"
-
-                 th:value="${file.year}">
-
-             <button type="submit" class="btn btn-primary btn-sm">재분류</button>
-
-         </form>
-
-     </td>
-
-     <td style="text-align:right">
-
-         <div style="display:flex;gap:4px;justify-content:flex-end">
-
-             <form th:action="@{/file-status/{id}/reject(id=${file.id})}" method="post">
-
-                 <button type="submit" class="btn btn-outline btn-sm"
-
-                     onclick="return confirm('반려 처리하시겠습니까?')">반려</button>
-
-             </form>
-
-             <form th:action="@{/file-status/{id}/delete(id=${file.id})}" method="post">
-
-                 <button type="submit" class="btn btn-danger-outline btn-sm"
-
-                     onclick="return confirm('목록에서 완전히 제거하시겠습니까?')">삭제</button>
-
-             </form>
-
-         </div>
-
-     </td>
-
- </tr>
-
- </tbody>

```

```
- </table>
```

```
- </div>
```

```
- </div>
```

```
-
```

```
-</section>
```

```
-</body>
```

```
-</html>
```

수정 templates/layout.html

```
@@ -384,9 +384,98 @@
```

```
    /* — Inline form — */
    .form-inline { display: flex; gap: 6px; align-items: center; flex-wrap: wrap; }
    /* — SVG icon — */
    .icon { width:16px; height:16px; stroke:currentColor; fill:none; stroke-width:1.8; stroke-linecap:round; stroke-linejoin:round; }
- </style>
```

```
+ /* — agent 테스트 패널 — */
```

```
+ .agent-panel {
```

```
+   position: fixed; top: 0; right: 0; bottom: 0; width: 480px; max-width: 100vw;
```

```
+   background: var(--surface); border-left: 1px solid var(--border);
```

```
+   box-shadow: var(--shadow-high); transform: translateX(100%);
```

```
+   transition: transform .18s ease; z-index: 120; display: flex; flex-direction: column;
```

```
+}
```

```
+ .agent-panel.open { transform: translateX(0); }
```

```
+ .agent-panel-head {
```

```
+   display: flex; align-items: center; gap: 8px; padding: 14px 16px;
```

```
+   border-bottom: 1px solid var(--border); background: var(--kb-navy); color: #fff;
```

```
+}
```

```
+ .agent-panel-head .title { font-size: 14px; font-weight: 600; flex: 1; }
```

```
+ .agent-close {
```

```
+   background: transparent; border: 0; color: inherit; cursor: pointer;
```

```
+   font-size: 20px; line-height: 1; padding: 2px 6px; border-radius: var(--r-sm);
```

```
+}
```

```
+ .agent-close:hover { background: rgba(255,255,255,.15); }
```

```
+ .agent-panel-body { padding: 16px; overflow-y: auto; flex: 1; }
```

```
+ .agent-section-title { font-size: 12px; font-weight: 700; color: var(--text-3); letter-spacing: .04em; margin: 0 0 8px; text-transform: uppercase; }
```

```
+ .agent-api-item {
```

```
+ border: 1px solid var(--border); border-radius: var(--r-md); padding: 10px 12px;

+ cursor: pointer; margin-bottom: 8px; background: var(--bg);

+}

+.agent-api-item.selected { border-color: var(--kb-navy); background: var(--surface); box-shadow: 0 0 0 2px rgba(27,42,107,.1); }

+.agent-api-item .name { font-size: 13px; font-weight: 600; color: var(--text-1); }

+.agent-api-item .path { font-family: var(--mono); font-size: 11.5px; color: var(--text-2); margin-top: 2px; word-break: break-all; }

+.agent-field { display: flex; flex-direction: column; gap: 4px; margin-bottom: 10px; }

+.agent-field label { font-size: 12px; color: var(--text-2); font-weight: 500; }

+.agent-url {

+ font-family: var(--mono); font-size: 11.5px; color: var(--text-2);

+ background: var(--bg); border: 1px dashed var(--border); border-radius: var(--r-sm);

+ padding: 6px 8px; word-break: break-all; margin: 8px 0 12px;

+}

+.agent-status { display: flex; gap: 8px; align-items: center; margin-bottom: 10px; font-size: 12px; }

+.agent-pill { display: inline-flex; padding: 2px 8px; border-radius: 999px; font-weight: 700; font-size: 11.5px; }

+.agent-pill.ok { background: var(--green-bg); color: var(--green); }

+.agent-pill.err { background: var(--red-bg); color: var(--red); }

+.agent-json {

+ font-family: var(--mono); font-size: 11.5px; white-space: pre; overflow: auto;

+ max-height: 320px; background: var(--bg); border: 1px solid var(--border);

+ border-radius: var(--r-sm); padding: 10px; color: var(--text-1);

+}

+/* — 대화형 요청 로그 — */

+.agent-chat { display: flex; flex-direction: column; gap: 8px; margin-bottom: 14px; }

+.agent-bubble {

+ max-width: 88%; padding: 8px 11px; border-radius: 12px; font-size: 12.5px; line-height: 1.55;

+ white-space: pre-wrap; word-break: break-word;

+}

+.agent-bubble.me { align-self: flex-end; background: var(--kb-navy); color: #fff; border-bottom-right-radius: 4px; }

+.agent-bubble.bot { align-self: flex-start; background: var(--bg); border: 1px solid var(--border); color: var(--text-1); border-bottom-left-radius: 4px; }

+.agent-caret { display: inline-block; width: 6px; color: inherit; animation: agentCaret 1s steps(1) infinite; }

+@keyframes agentCaret { 50% { opacity: 0; } }

+.agent-dots { display: inline-flex; gap: 4px; align-items: center; padding: 2px 0; }
```

```

+.agent-dots i {
+  width: 5px; height: 5px; border-radius: 50%; background: var(--text-3); display: block;
+  animation: agentDot 1.1s ease-in-out infinite;
+}

+.agent-dots i:nth-child(2) { animation-delay: .18s; }
+.agent-dots i:nth-child(3) { animation-delay: .36s; }

+@keyframes agentDot { 0%, 60%, 100% { opacity: .25; transform: translateY(0); } 30% { opacity: 1; transform: translateY(-3px); } }

+@media (prefers-reduced-motion: reduce) {
+  .agent-caret, .agent-dots i { animation: none; }
+}

+.agent-result-table { width: 100%; border-collapse: collapse; font-size: 12.5px; margin-bottom: 12px; }

+.agent-result-table th, .agent-result-table td { border-bottom: 1px solid var(--border); padding: 6px 4px; text-align: left; vertical-align: top; }

+.agent-result-table th { color: var(--text-3); font-weight: 600; font-size: 11.5px; }

+/* — 크게보기 모달 — */

+.agent-overlay { position: fixed; inset: 0; background: rgba(15,28,77,.45); z-index: 130; display: none; }
+.agent-overlay.open { display: block; }

+.agent-modal {
+  position: fixed; inset: 24px; z-index: 140; background: var(--surface);
+  border-radius: var(--r-lg); box-shadow: var(--shadow-high); display: none;
+  flex-direction: column; overflow: hidden;
+}

+.agent-modal.open { display: flex; }

+.agent-modal-head { display: flex; align-items: center; gap: 12px; padding: 14px 18px; border-bottom: 1px solid var(--border); }

+.agent-modal-head .title { font-size: 15px; font-weight: 700; color: var(--text-1); flex: 1; }

+.agent-modal-meta { font-size: 12px; color: var(--text-2); display: flex; gap: 10px; flex-wrap: wrap; padding: 10px 18px; border-bottom: 1px solid var(--border); background: var(--bg); }

+.agent-tabs { display: flex; gap: 4px; padding: 10px 18px 0; }

+.agent-tab { border: 1px solid var(--border); border-bottom: 0; background: var(--bg); color: var(--text-2);
+  padding: 6px 14px; border-radius: var(--r-sm) var(--r-sm) 0 0; cursor: pointer; font-size: 12.5px; }

+.agent-tab.active { background: var(--surface); color: var(--text-1); font-weight: 600; }

+.agent-modal-body { padding: 16px 18px; overflow-y: auto; flex: 1; border-top: 1px solid var(--border); }

+.slide-card { border: 1px solid var(--border); border-radius: var(--r-md); padding: 12px 14px; margin-bottom: 10px; background: var(--surface); }

+.slide-card-head { display: flex; align-items: center; gap: 8px; margin-bottom: 6px; }

```

```

+.slide-no { font-family: var(--mono); font-size: 11.5px; color: #fff; background: var(--kb-navy); border-radius: var(--r-sm); padding: 2px 7px; }

+.slide-title { font-size: 13.5px; font-weight: 600; color: var(--text-1); flex: 1; }

+.slide-text { font-size: 12.5px; color: var(--text-2); white-space: pre-wrap; line-height: 1.6; }

+</style>

</head>
<body>

<!-- — 사이드바 — -->
@@ -423,26 +512,27 @@

    </svg>
    파일 업로드
  </a>

-   <a th:href="@{/file-status}"
-       th:class="${'nav-item' + (#request.getRequestURI() == '/file-status' ? ' active' : '')}"
+   <a th:href="@{/parse-status}"
+       th:class="${'nav-item' + (#request.getRequestURI() == '/parse-status' ? ' active' : '')}"

    <svg class="nav-icon icon" viewBox="0 0 24 24">
      <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/>
      <polyline points="14 2 14 8 20 8"/>
    </svg>
-   파일 처리 상태
-   <span th:if="${unclassifiedCount != null and unclassifiedCount > 0}"
-       class="nav-badge" th:text="${unclassifiedCount}"></span>
+   파싱 현황
+   <!-- 2026-09-23: 배지가 미분류 건수 → 파싱 실패 건수로 바꿨었다 -->
+   <span th:if="${parseFailedCount != null and parseFailedCount > 0}"
+       class="nav-badge" th:text="${parseFailedCount}"></span>

  </a>

  <a th:href="@{/file-status/classified}"
      th:class="${'nav-item' + (#request.getRequestURI().startsWith('/file-status/classified') ? ' active' : '')}"
      <svg class="nav-icon icon" viewBox="0 0 24 24">
        <path d="M11 4H4a2 2 0 0 0-2 2v14a2 2 0 0 0 2 2h14a2 2 0 0 0 2-2v-7"/>
        <path d="M18.5 2.5a2.121 2.121 0 0 1 3 3L12 15l-4 1 1-4 9.5-9.5z"/>
      </svg>
-   분류 재작업
+   기관 미분류

  </a>

  <a th:href="@{/institutions}"
      th:class="${'nav-item' + (#request.getRequestURI().startsWith('/institutions') ? ' active' : '')}"
@@ -461,8 +551,15 @@

  <main class="main">
    <header class="topbar">
      <span class="topbar-title" th:text="${pageTitle}">파일 업로드 관리</span>
      <div class="topbar-divider"></div>
+     <button type="button" class="btn btn-outline btn-sm" id="agentTestOpen">

```

```

+       <svg class="icon" viewBox="0 0 24 24" style="margin-right:4px">
+
+       <polyline points="16 18 22 12 16 6"/>
+
+       <polyline points="8 6 2 12 8 18"/>
+
+     </svg>
+
+     agent 테스트
+
+   </button>
+
+   <div class="topbar-user">
+     <div class="avatar">관</div>
+     관리자
+   </div>
@@ -472,6 +569,40 @@
+
+   </div>
+ </main>
+
+ <script th:src="@{/js/bootstrap.bundle.min.js}"></script>
+<!-- — agent 테스트 패널 — -->
+
+<aside class="agent-panel" id="agentPanel" th:attr="data-base=@{/}">
+
+  <div class="agent-panel-head">
+
+    <span class="title">agent 테스트 - 제공 API 호출</span>
+
+    <button type="button" class="agent-close" id="agentTestClose" aria-label="닫기">&times;</button>
+
+  </div>
+
+  <div class="agent-panel-body">
+
+    <p class="agent-section-title">API 목록</p>
+
+    <div id="agentApiList"></div>
+
+
+
+    <p class="agent-section-title" style="margin-top:18px">파라미터</p>
+
+    <div id="agentParams"></div>
+
+    <div class="agent-url" id="agentUrl">—</div>
+
+    <button type="button" class="btn btn-primary btn-sm" id="agentSend">요청</button>
+
+
+
+    <div id="agentResult" style="margin-top:16px"></div>
+
+  </div>
+</aside>
+
+
+<div class="agent-overlay" id="agentOverlay"></div>
+
+<div class="agent-modal" id="agentModal">
+
+  <div class="agent-modal-head">
+
+    <span class="title" id="agentModalTitle">제안서</span>
+
+    <button type="button" class="btn btn-outline btn-sm" id="agentModalClose">닫기</button>

```

```

+ </div>

+ <div class="agent-modal-meta" id="agentModalMeta"></div>

+ <!-- 탭은 문서 종류(제안서/RFP)에 따라 agent-test.js 가 채운다 -->

+ <div class="agent-tabs" id="agentModalTabs">

+ </div>

+ <div class="agent-modal-body" id="agentModalBody"></div>

+</div>

+

+<script th:src="@{/js/bootstrap.bundle.min.js}"></script>

+<script th:src="@{/js/agent-test.js}"></script>

</body>
</html>

```

신규 templates/parse-status.html

```

new file mode 100644
@@ -0,0 +1,120 @@

+<!DOCTYPE html>

+<html xmlns:th="http://www.thymeleaf.org"

+    th:replace="layout :: layout(~{::title}, '파싱 현황', ~{::section})">

+<head><title>파싱 현황</title></head>

+<body>

+<section>

+

+    <div class="section-header" style="margin-bottom:20px">

+        <div style="display:flex;align-items:center;gap:10px;flex-wrap:wrap">

+            <span class="section-title" style="font-size:16px">파싱 현황</span>

+            <span class="badge badge-done" th:text="'성공 ' + ${successCount}"></span>

+            <span th:if="${parseFailedCount > 0}" class="badge badge-pending"

+                th:text="'실패 ' + ${parseFailedCount}"></span>

+            <span th:if="${unclassifiedCount > 0}" class="badge badge-neutral"

+                th:text="'기관 미분류 ' + ${unclassifiedCount}"></span>

+        </div>

+    </div>

+

+    <div th:if="${message}" class="alert-success" th:text="${message}"></div>

+    <div th:if="${error}" class="alert-error" th:text="${error}"></div>

```

```

+
+ <!-- 필터 -->
+ <form method="get" th:action="@{/parse-status}" class="search-bar">
+   <select name="docType" class="form-control" style="max-width:180px">
+     <option value="">문서종류 전체</option>
+     <option th:each="t : ${docTypes}" th:value="${t}"
+       th:text="${t == 'BID_PROPOSAL' ? '입찰제안서' : '입찰공고문(RFP)'}"
+       th:selected="${docType == t}"></option>
+   </select>
+   <select name="parseStatus" class="form-control" style="max-width:160px">
+     <option value="">파싱상태 전체</option>
+     <option value="SUCCESS" th:selected="${parseStatus == 'SUCCESS'}">성공</option>
+     <option value="FAILED" th:selected="${parseStatus == 'FAILED'}">실패</option>
+     <option value="PENDING" th:selected="${parseStatus == 'PENDING'}">대기</option>
+   </select>
+   <select name="category" class="form-control" style="max-width:160px">
+     <option value="">기관분류 전체</option>
+     <option value="미분류" th:selected="${category == '미분류'}">미분류만</option>
+   </select>
+   <button type="submit" class="btn btn-primary">조회</button>
+   <a th:href="@{/parse-status}" class="btn btn-outline">초기화</a>
+ </form>
+
+ <div th:if="${files.isEmpty()}" class="empty-state">
+   <svg class="icon" style="width:40px;height:40px;opacity:.35" viewBox="0 0 24 24">
+     <path d="M14 2H6a2 2 0 0 0 2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2 2v8z"/>
+     <polyline points="14 2 14 8 20 8"/>
+   </svg>
+   <div class="empty-state-text">조건에 맞는 파일이 없습니다.</div>
+ </div>
+
+ <div th:if="${!files.isEmpty()}" class="table-card">
+   <table>
+     <thead>
+     <tr>

```

```

+       <th>문서종류</th>
+
+       <th>원본 파일명</th>
+
+       <th>기관</th>
+
+       <th>기관분류</th>
+
+       <th>연도 / 공고일</th>
+
+       <th>파싱</th>
+
+       <th>산출물</th>
+
+       <th>파싱 일시</th>
+
+       <th></th>
+
+     </tr>
+
+   </thead>
+
+   <tbody>
+
+     <tr th:each="f : ${files}">
+
+       <td style="font-size:12.5px" th:text="${f.docTypeLabel}"></td>
+
+       <td>
+
+         <span th:with="ext=${#strings.toLowerCase(f.originalName.substring(f.originalName.lastIndexOf('.') + 1))}"
+
+           th:class="'file-ext ext-' + ${ext}" th:text="${#strings.toUpperCase(ext)}"></span>
+
+         <span class="file-name" th:text="${f.originalName}"></span>
+
+       </td>
+
+       <td style="font-size:12.5px" th:text="${f.institutionName != null ? f.institutionName : '-'}"></td>
+
+       <td>
+
+         <span class="category-chip" th:text="${f.categoryLabel}"></span>
+
+       </td>
+
+       <td class="time-col" th:text="${f.docDate != null ? f.docDate : '-'}"></td>
+
+       <td>
+
+         <span th:if="${f.parseStatus == 'SUCCESS'}" class="badge badge-done">성공</span>
+
+         <span th:if="${f.parseStatus == 'FAILED'}" class="badge badge-pending">실패</span>
+
+         <span th:if="${f.parseStatus != 'SUCCESS' and f.parseStatus != 'FAILED'}"
+
+           class="badge badge-neutral" th:text="${f.parseStatus != null ? f.parseStatus : '-'}"></span>
+
+         <!-- ⚠ 실패 사유는 저장하지 않는다(스키마 무변경안). 서버 로그를 봐야 한다. -->
+
+         <div th:if="${f.parseStatus == 'FAILED'}" style="font-size:11.5px;color:var(--text-2);margin-top:3px">
+
+           사유는 서버 로그 확인
+
+         </div>
+
+       </td>
+
+     </tr>
+
+   </tbody>

```

```

+         <a th:if="{f.outputPath != null}" th:href="@{/parse-status/{id}/output(id={f.id})}"
+             target="_blank" class="file-name" style="font-size:12.5px"
+             th:text="{f.outputFileName}"></a>
+         <span th:if="{f.outputPath == null}" style="color:var(--text-3)">-</span>
+     </td>
+     <td class="time-col"
+         th:text="{f.parsedAt != null ? #temporals.format(f.parsedAt, 'yyyy-MM-dd HH:mm') : '-'}"></td>
+     <td>
+         <div class="form-inline">
+             <a th:if="{f.outputPath != null}"
+                 th:href="@{/parse-status/{id}/output(id={f.id},download=true)}"
+                 class="btn btn-outline btn-sm">내려받기</a>
+             <form th:action="@{/parse-status/{id}/reparse(id={f.id})}" method="post">
+                 <button type="submit" class="btn btn-primary btn-sm"
+                     th:disabled="{f.docType == null}">재파싱</button>
+             </form>
+         </div>
+     </td>
+ </tr>
+ </tbody>
+ </table>
+ </div>
+
+ <p style="font-size:12.5px;color:var(--text-2);margin-top:12px">
+     기관을 새로 등록한 뒤 <b>재파싱</b>을 누르면 기관분류가 반영되고 산출물 파일명도 새로 만들어집니다.
+ </p>
+
+ </section>
+ </body>
+ </html>

```

수정 templates/upload.html

@@ -6,77 +6,82 @@

<section>

```

<div class="section-header" style="margin-bottom:20px">
  <span class="section-title" style="font-size:18px">파일 업로드</span>
+   <span class="section-sub">업로드하면 문서 내용을 읽어 산출물을 만듭니다. 기관·연도는 본문에서 찾습니다.</span>

</div>

-   <!-- 업로드 카드 -->

-   <div class="card" style="max-width:600px;margin-bottom:24px">

-       <div class="card-header">파일 선택</div>

-       <div class="card-body">

-           <form method="post" th:action="@{/upload}" enctype="multipart/form-data">

-               <div class="form-group">

-                   <label class="form-label">파일 선택 (pdf, hwp, md – 복수 선택 가능)</label>

-                   <input type="file" name="files" multiple accept=".pdf,.hwp,.md" class="form-control">

-               </div>

+       <div th:if="${errorMessage}"

+           style="margin-bottom:14px;padding:10px 14px;background:#fff3f3;border:1px solid #f5c6c6;border-radius:6px;color:#c0392b;font-size:13.5px"

+           th:text="${errorMessage}"></div>

-       <!-- 수동 입력 (파일명 규칙 불일치 시) -->

-       <div style="margin-top:16px;padding:12px 14px;background:var(--surface-2,#f8f9fa);border-radius:6px;border:1px solid var(--border,#e0e0e0)">

-           <div style="color:var(--text-2,#666);margin-bottom:10px;font-weight:600">

-               파일명 규칙 불일치 시 직접 입력 <span style="font-weight:400">(선택사항 – 파일명이 <code>년도_기관명_설명.확장자</code> 형식이 아닐 때 사용)</span>

+       <div style="display:flex;gap:16px;flex-wrap:wrap;margin-bottom:24px">

+       <!-- 입찰제안서 -->

+       <div class="card" style="flex:1;min-width:320px">

+           <div class="card-header">입찰제안서</div>

+           <div class="card-body">

+               <form method="post" th:action="@{/upload/proposal}" enctype="multipart/form-data">

+                   <div class="form-group">

+                       <label class="form-label">ppt, pptx – 복수 선택 가능</label>

+                       <input type="file" name="files" multiple accept=".ppt,.pptx" class="form-control">

+                   </div>

-                   <div style="display:flex;gap:10px;flex-wrap:wrap">

-                       <div style="flex:1;min-width:140px">

-                           <label class="form-label">기관명</label>

-                           <input type="text" name="institution" class="form-control form-control-sm" placeholder="예: 서울에너지공사">

```

```

-         </div>
-         <div style="width:100px">
-             <label class="form-label">연도</label>
-             <input type="text" name="year" class="form-control form-control-sm" placeholder="예: 2024" maxlength="4">
-         </div>
-         <div style="flex:1;min-width:140px">
-             <label class="form-label">카테고리</label>
-             <select name="category" class="form-control form-control-sm">
-                 <option value="">선택 (선택사항)</option>
-                 <option th:each="cat : ${categories}" th:value="${cat}" th:text="${cat}"></option>
-             </select>
-         </div>
+         <div style="color:var(--text-2,#666);font-size:12.5px;margin-top:8px">
+             슬라이드를 읽어 <code>{연도}_{기관분류}_{기관명}_{파싱일}_bid_proposal.json</code> 을 만듭니다.
+         </div>
-     </div>
+     <button type="submit" class="btn btn-primary" style="margin-top:14px">제안서 업로드</button>
+ </form>
+ </div>
+ </div>

-     <div th:if="${errorMessage}" style="margin-top:12px;padding:10px 14px;background:#fff3f3;border:1px solid #f5c6c6;border-radius:6px;color:#c0392b;font-size:13.5px" th:text="${errorMessage}"></div>
-     <button type="submit" class="btn btn-primary" style="margin-top:14px">업로드</button>
- </form>
+ <!-- 임찰공고문 RFP -->
+ <div class="card" style="flex:1;min-width:320px">
+     <div class="card-header">임찰공고문 (RFP)</div>
+     <div class="card-body">
+         <form method="post" th:action="@{/upload/rfp}" enctype="multipart/form-data">
+             <div class="form-group">
+                 <label class="form-label">pdf, hwp, hwpX - 복수 선택 가능</label>
+                 <input type="file" name="files" multiple accept=".pdf,.hwp,.hwpX" class="form-control">
+             </div>
+             <div style="color:var(--text-2,#666);font-size:12.5px;margin-top:8px">
+                 본문을 요약해 <code>{공고일}_{기관분류}_{기관명}_{파싱일}.md</code> 를 만듭니다.

```

```

+         </div>
+         <button type="submit" class="btn btn-primary" style="margin-top:14px">공고문 업로드</button>
+     </form>
+ </div>
+
+ </div>
+
+ </div>
+
+ <!-- 업로드 결과 -->
+ <div th:if="${results != null and !results.isEmpty()}">
+     <div class="section-header">
+         <span class="section-title">업로드 결과</span>
+         <span th:if="${uploadedType}" class="section-sub" th:text="${uploadedType}"></span>
+     </div>
+     <div class="table-card">
+         <table>
+             <thead>
+                 <tr>
+                     <th>파일명</th><th>카테고리</th><th>결과</th><th>메시지</th>
+                     <th>파일명</th><th>기관분류</th><th>파싱</th><th>메시지</th>
+                 </tr>
+             </thead>
+             <tbody>
+                 <tr th:each="r : ${results}">
+                     <td>
+                         <span th:with="ext=${#strings.toLowerCase(r.filename.substring(r.filename.lastIndexOf('.') + 1))}"
+                             th:class="${'file-ext' + (ext == 'pdf' ? ' ext-pdf' : (ext == 'hwp' ? ' ext-hwp' : (ext == 'md' ? ' ext-md' : '')))}"
+                             th:text="${#strings.toUpperCase(ext)}"></span>
+                         <!-- ⚠ #strings.substringAfterLast 는 이 Thymeleaf 버전에 없다. 쓰면 렌더링이 끊긴다 -->
+                         <span th:with="ext=${#strings.toLowerCase(r.filename.substring(r.filename.lastIndexOf('.') + 1))}"
+                             th:class="${'file-ext' + (ext == 'pdf' ? ' ext-pdf' : (ext == 'hwp' ? ' ext-hwp' : (ext == 'md' ? ' ext-md' : '')))}"
+                             th:text="${#strings.toUpperCase(ext)}"></span>
+
+                         <span class="file-name" th:text="${r.filename}"></span>
+                     </td>
+                     <td>
+                         <span th:if="${r.category != null}" class="category-chip" th:text="${r.category}"></span>
+                         <span th:if="${r.category == null}" style="color:var(--text-3)"></span>
+                     </td>
+                     <td>
+                         <span th:class="${'badge ' + (r.classified ? 'badge-done' : 'badge-pending')}"
+                         th:text="${r.classified ? '분류 완료' : '미분류'}"></span>
+                         <span th:class="${'badge ' + (r.classified ? 'badge-done' : 'badge-pending')}"
+                         th:text="${r.classified ? '성공' : '실패'}"></span>
+                     </td>
+                     <td style="color:var(--text-2);font-size:12.5px" th:text="${r.message}"></td>
+                 </tr>
+             </tbody>

```

수정 test/ApplicationContextTest.java

```
@@ -1,9 +1,8 @@

package com.kb.uploader;

import com.kb.uploader.job.BackgroundScheduler;
import com.kb.uploader.job.LocalScheduler;
- import com.kb.uploader.job.ReclassificationTrigger;

import org.junit.Test;
import org.junit.runner.RunWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
@@ -47,12 +46,8 @@ public class ApplicationContextTest {

    assertTrue("폴백이 아니라 " + scheduler.describe(),
               scheduler instanceof LocalScheduler);
}

- @Test
- public void 재분류_트리거가_빈으로_있다() {
-     assertNotNull(context.getBean(ReclassificationTrigger.class));
- }

@Test
public void Spring_자체_스케줄러는_더_이상_만들어지지_않는다() {
    // @EnableScheduling 이 남아 있으면 ScheduledAnnotationBeanPostProcessor 가
```

수정 test/config/ConfigEnvsTest.java

```
@@ -36,9 +36,14 @@ public class ConfigEnvsTest {

    "mybatis.mapper-locations",
    "mybatis.type-aliases-package",
    "mybatis.configuration.map-underscore-to-camel-case",
    "upload.base-dir",
-    "reclassification.cron");
+
+    // 2026-09-23 파싱 전환: reclassification.cron(재분류 배치)이 빠지고
+    // 원본 보관·산출물 경로 3개가 들어왔다. 한 환경이라도 빠지면 그 환경에서
+    // FileStorageService 가 @Value 를 못 채워 기동이 죽는다.
+    "upload.userdata-dir",
+    "upload.proposal-json-dir",
+    "upload.rfp-md-dir");

/**
 * DataSource 계열은 환경마다 방식이 갈려서(내부망 prod 는 JNDI, 나머지는 직접 접속)
 * 여기서 요구하지 않는다 - 대신 <b>둘 중 하나는</b> 있어야 한다.
```

삭제 test/controller/UploadControllerTest.java

```
deleted file mode 100644
@@ -1,62 +0,0 @@
```

```

-package com.kb.uploader.controller;

-

-import com.kb.uploader.dto.UploadResultItem;

-import com.kb.uploader.mapper.InstitutionMapper;

-import com.kb.uploader.mapper.UploadedFileMapper;

-import com.kb.uploader.service.FileUploadService;

-import org.junit.Test;

-import org.junit.runner.RunWith;

-import org.springframework.beans.factory.annotation.Autowired;

-import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;

-import org.springframework.boot.test.mock.mockito.MockBean;

-import org.springframework.mock.web.MockMultipartFile;

-import org.springframework.test.context.junit4.SpringRunner;

-import org.springframework.test.web.servlet.MockMvc;

-import java.util.Arrays;

-import static org.mockito.Mockito.*;

-import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;

-import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

-

-@RunWith(SpringRunner.class)

-@WebMvcTest(controllers = {KGI1100$UploadView.class, KGI1100$UploadAction.class})

-public class UploadControllerTest {

-

-    @Autowired

-    private MockMvc mockMvc;

-

-    @MockBean

-    private FileUploadService uploadService;

-

-    @MockBean

-    private UploadedFileMapper fileMapper;

-

-    // KGI1100$UploadAction 이 생성자 3번째 인자로 받는다(업로드 품의 카테고리 목록을

-    // populateCategories()에서 채운다). 이게 없으면 @WebMvcTest 컨텍스트 자체가 못 뜬다.

-    @MockBean

```

```

- private InstitutionMapper instMapper;
-
-
- @Test
- public void GET_upload_200반환() throws Exception {
-     when(fileMapper.countByStatus("UNCLASSIFIED")).thenReturn(0L);
-     mockMvc.perform(get("/upload"))
-         .andExpect(status().isOk())
-         .andExpect(view().name("upload"));
- }
-
-
- @Test
- public void POST_upload_결과_모델포함() throws Exception {
-     MockMultipartFile file = new MockMultipartFile(
-         "files", "2024_서울대_보고서.pdf",
-         "application/pdf", "내용".getBytes());
-
-     // upload()는 오버로드다 - 컨트롤러가 부르는 것은 4인자 쪽이다.
-     // 1인자 스텝은 매칭되지 않아 results 가 null 로 들어간다.
-     when(uploadService.upload(any(), any(), any(), any())).thenReturn(Arrays.asList(
-         new UploadResultItem("2024_서울대_보고서.pdf", true, "대학교", "분류 완료")));
-     when(fileMapper.countByStatus("UNCLASSIFIED")).thenReturn(0L);
-
-     mockMvc.perform(multipart("/upload").file(file))
-         .andExpect(status().isOk())
-         .andExpect(view().name("upload"))
-         .andExpect(model().attributeExists("results"));
- }
-}

```

삭제 test/job/ReclassificationJobTest.java

```

deleted file mode 100644
@@ -1,58 +0,0 @@
- package com.kb.uploader.job;
-
-
- import com.kb.uploader.domain.UploadedFile;
- import com.kb.uploader.mapper.UploadedFileMapper;

```

```
-import com.kb.uploader.service.ClassificationService;

-import org.junit.Test;

-import org.junit.runner.RunWith;

-import org.mockito.InjectMocks;

-import org.mockito.Mock;

-import org.mockito.junit.MockitoJUnitRunner;

-

-import java.util.Arrays;

-import java.util.Collections;

-

-import static org.mockito.ArgumentMatchers.any;

-import static org.mockito.Mockito.*;

-

-@RunWith(MockitoJUnitRunner.class)

-public class ReclassificationJobTest {

-

-    @Mock private UploadedFileMapper uploadedFileMapper;

-    @Mock private ClassificationService classificationService;

-    @InjectMocks private ReclassificationJob reclassificationJob;

-

-    @Test

-    public void reclassify_미분류파일있음_분류시도() {

-        UploadedFile file = new UploadedFile("2024_서울시_보고서.pdf", "/path/file.pdf", "2024", "서울시");

-        when(uploadedFileMapper.findByStatus("UNCLASSIFIED")).thenReturn(Collections.singletonList(file));

-        when(classificationService.classify(file)).thenReturn(true);

-

-        reclassificationJob.reclassify();

-

-        verify(classificationService, times(1)).classify(file);

-    }

-

-    @Test

-    public void reclassify_미분류파일없음_호출안함() {

-        when(uploadedFileMapper.findByStatus("UNCLASSIFIED")).thenReturn(Collections.emptyList());

-    }

-}
```



```

-import static org.mockito.Mockito.mock;

-import static org.mockito.Mockito.when;

-

-/**
- * "한 번 돌고 다음을 다시 예약"하는 반복 규약을 본다 - 시계를 기다리지 않고 검증한다.
- *
- * <p>여기서 지키려는 것은 하나다: <b>한 번 실패했다고 반복이 멈추면 안 된다.</b>
- * 그러면 증상이 "미분류 파일이 계속 쌓인다"로만 나타나 원인을 찾기 어렵다.
- */
-public class ReclassificationTriggerTest {
-
-    /** 예약을 받아 두기만 하고 돌리지는 않는 스케줄러. */
-    private static final class RecordingScheduler implements BackgroundScheduler {
-
-        final List<Long> delays = new ArrayList<Long>();
-
-        final List<Runnable> works = new ArrayList<Runnable>();
-
-
-        @Override
-
-        public void scheduleOnce(String name, long delayMillis, Runnable work) {
-
-            delays.add(Long.valueOf(delayMillis));
-
-            works.add(work);
-
-        }
-
-
-        @Override
-
-        public String describe() {
-
-            return "recording";
-
-        }
-
-    }
-
-
-    private static ReclassificationJob jobThatWorks() {
-
-        UploadedFileMapper mapper = mock(UploadedFileMapper.class);
-
-        when(mapper.findByStatus("UNCLASSIFIED")).thenReturn(Collections.emptyList());
-
-        return new ReclassificationJob(mapper, mock(ClassificationService.class));
-
-    }
-
-
-    private static ReclassificationJob jobThatThrows() {

```

```

-     UploadedFileMapper mapper = mock(UploadedFileMapper.class);
-
-     when(mapper.findByStatus("UNCLASSIFIED"))
-
-         .thenThrow(new IllegalStateException("DB 가 죽었다"));
-
-     return new ReclassificationJob(mapper, mock(ClassificationService.class));
- }
-
-
- @Test
- public void 기동하면_다음_cron_시각으로_한_번_예약한다() {
-
-     RecordingScheduler scheduler = new RecordingScheduler();
-
-     new ReclassificationTrigger(scheduler, jobThatWorks(), "0 */5 * * *").start();
-
-
-     assertEquals("기동 직후 예약이 하나 있어야 한다", 1, scheduler.works.size());
-
-     long delay = scheduler.delays.get(0).longValue();
-
-     assertTrue("지연이 음수다: " + delay, delay >= 0);
-
-     assertTrue("5분 주기인데 지연이 너무 크다: " + delay, delay <= 5 * 60 * 1000L);
-
- }
-
-
- @Test
- public void 한_번_돌면_다음을_다시_예약한다() {
-
-     RecordingScheduler scheduler = new RecordingScheduler();
-
-     new ReclassificationTrigger(scheduler, jobThatWorks(), "0 */5 * * *").start();
-
-
-     scheduler.works.get(0).run();
-
-
-     assertEquals("실행 뒤 다음 예약이 없다 - 반복이 한 번에 끝난다",
-
-         2, scheduler.works.size());
-
- }
-
-
- @Test
- public void 작업이_실패해도_다음을_예약한다() {
-
-     RecordingScheduler scheduler = new RecordingScheduler();
-
-     new ReclassificationTrigger(scheduler, jobThatThrows(), "0 */5 * * *").start();
-
-
-     scheduler.works.get(0).run();
-
-

```

```

- assertEquals("실패했다고 반복이 멈추면 미분류가 영영 쌓인다",
-             2, scheduler.works.size());
- }
-
- @Test
- public void 종료된_뒤에는_다시_예약하지_않는다() {
-     RecordingScheduler scheduler = new RecordingScheduler();
-     ReclassificationTrigger trigger =
-         new ReclassificationTrigger(scheduler, jobThatWorks(), "0 */5 * * * *");
-     trigger.start();
-     trigger.stop();
-
-     scheduler.works.get(0).run();
-
-     assertEquals("컨텍스트가 달렸는데 예약이 늘었다", 1, scheduler.works.size());
- }
-
- @Test
- public void cron_이_비면_아무것도_예약하지_않는다() {
-     RecordingScheduler scheduler = new RecordingScheduler();
-     new ReclassificationTrigger(scheduler, jobThatWorks(), " ").start();
-
-     assertTrue("끄기로 했는데 예약이 걸렸다", scheduler.works.isEmpty());
- }
-
- @Test
- public void 잘못된_cron_은_기동_때_죽는다() {
-     try {
-         new ReclassificationTrigger(new RecordingScheduler(), jobThatWorks(), "매 5분").start();
-         fail("해석 불가능한 cron 인데 예외가 안 났다");
-     } catch (IllegalStateException expected) {
-         assertTrue(expected.getMessage().contains("매 5분"));
-     }
- }

```

```
-}
```

삭제 test/service/ClassificationServiceTest.java

```
deleted file mode 100644
```

```
@@ -1,64 +0,0 @@
```

```
-package com.kb.uploader.service;
```

```
-
```

```
-import com.kb.uploader.domain.Institution;
```

```
-import com.kb.uploader.domain.UploadedFile;
```

```
-import com.kb.uploader.mapper.InstitutionMapper;
```

```
-import com.kb.uploader.mapper.UploadedFileMapper;
```

```
-import org.junit.Before;
```

```
-import org.junit.Test;
```

```
-
```

```
-import java.nio.file.Paths;
```

```
-import java.util.Optional;
```

```
-
```

```
-import static org.junit.Assert.*;
```

```
-import static org.mockito.Mockito.*;
```

```
-
```

```
-public class ClassificationServiceTest {
```

```
-
```

```
-    private InstitutionMapper institutionMapper;
```

```
-    private UploadedFileMapper fileMapper;
```

```
-    private FileStorageService storageService;
```

```
-    private ClassificationService sut;
```

```
-
```

```
-    @Before
```

```
-    public void setUp() {
```

```
-        institutionMapper = mock(InstitutionMapper.class);
```

```
-        fileMapper = mock(UploadedFileMapper.class);
```

```
-        storageService = mock(FileStorageService.class);
```

```
-        sut = new ClassificationService(institutionMapper, fileMapper, storageService);
```

```
-    }
```

```
-
```

```
-    @Test
```

```

- public void 기관명_매핑_성공시_CLASSIFIED() throws Exception {
-
-     UploadedFile file = new UploadedFile(
-
-         "2024_서울대학교_보고서.pdf",
-
-         "/tmp/unclassified/2024_서울대학교_보고서.pdf",
-
-         "2024", "서울대학교");
-
-     when(institutionMapper.findByName("서울대학교"))
-
-         .thenReturn(Optional.of(new Institution("서울대학교", "대학교")));
-
-     when(storageService.moveToClassified(any(), eq("대학교"), eq("2024"), eq("서울대학교")))
-
-         .thenReturn(Paths.get("/tmp/classified/대학교/2024/서울대학교/2024_서울대학교_보고서.pdf"));
-
-
-
-     boolean result = sut.classify(file);
-
-
-
-     assertTrue(result);
-
-     assertEquals("CLASSIFIED", file.getStatus());
-
-     assertEquals("대학교", file.getCategory());
-
-     verify(fileMapper).update(file);
-
-     }
-
-
-
- @Test
-
- public void 기관명_미등록시_UNCLASSIFIED_유지() throws Exception {
-
-     UploadedFile file = new UploadedFile(
-
-         "2024_미지기관_보고서.pdf",
-
-         "/tmp/unclassified/2024_미지기관_보고서.pdf",
-
-         "2024", "미지기관");
-
-     when(institutionMapper.findByName("미지기관")).thenReturn(Optional.empty());
-
-
-
-     boolean result = sut.classify(file);
-
-
-
-     assertFalse(result);
-
-     assertEquals("UNCLASSIFIED", file.getStatus());
-
-     verify(fileMapper, never()).update(any());
-
-     }
-
- }

```

deleted file mode 100644

@@ -1,53 +0,0 @@

```
-package com.kb.uploader.service;
-
-
-import com.kb.uploader.dto.ParsedFileName;
-import org.junit.Test;
-import java.util.Optional;
-import static org.junit.Assert.*;
-
-public class FileParserServiceTest {
-
-    private final FileParserService sut = new FileParserService();
-
-    @Test
-    public void 정상_파일명_파싱() {
-        Optional<ParsedFileName> result = sut.parse("2024_서울대학교_보고서.pdf");
-        assertTrue(result.isPresent());
-        assertEquals("2024", result.get().getYear());
-        assertEquals("서울대학교", result.get().getInstitutionName());
-        assertEquals("보고서", result.get().getDescription());
-        assertEquals("pdf", result.get().getExtension());
-    }
-
-    @Test
-    public void 설명에_언더스코어_포함시_기관명만_두번째_토큰() {
-        Optional<ParsedFileName> result = sut.parse("2024_서울대학교_2024년_연간보고서.pdf");
-        assertTrue(result.isPresent());
-        assertEquals("서울대학교", result.get().getInstitutionName());
-        assertEquals("2024년_연간보고서", result.get().getDescription());
-    }
-
-    @Test
-    public void 년도가_4자리_숫자_아니면_빈_Optional() {
-        assertFalse(sut.parse("년도없는_서울대학교_보고서.pdf").isPresent());
-    }
-}
```

```

-     assertFalse(sut.parse("24_서울대학교_보고서.pdf").isPresent());
-     assertFalse(sut.parse("abcd_서울대학교_보고서.pdf").isPresent());
- }
-
- @Test
- public void 구분자_2개_미만이면_빈_Optional() {
-     assertFalse(sut.parse("2024_서울대학교.pdf").isPresent());
-     assertFalse(sut.parse("2024.pdf").isPresent());
- }
-
- @Test
- public void 허용되지_않은_확장자는_빈_Optional() {
-     assertFalse(sut.parse("2024_서울대학교_보고서.xlsx").isPresent());
-     assertFalse(sut.parse("2024_서울대학교_보고서.txt").isPresent());
- }
-
- @Test
- public void 파일명이_null이면_빈_Optional() {
-     assertFalse(sut.parse(null).isPresent());
- }
-}

```

수정 test/service/FileStorageServiceTest.java

```

@@ -17,9 +17,11 @@ public class FileStorageServiceTest {
    private FileStorageService sut;

    @Before
    public void setUp() {
-        sut = new FileStorageService(tempFolder.getRoot().getAbsolutePath());
+        String root = tempFolder.getRoot().getAbsolutePath();
+        sut = new FileStorageService(root, root + "/userdata",
+            root + "/out-json", root + "/out-md");
    }

    @Test
    public void unclassified_저장_파일생성() throws Exception {
@@ -57,5 +59,36 @@ public class FileStorageServiceTest {

        assertTrue(moved.toString().contains("2024"));
        assertTrue(moved.toString().contains("서울대학교"));
    }
}

```

```

        assertFalse(Files.exists(source));
    }
+
+
+ // — 2026-09-23 파싱 전환으로 추가된 API —
+
+
+ @Test
+ public void 원본은_userdata에_보관한다() throws Exception {
+     MockMultipartFile file = new MockMultipartFile(
+         "file", "제안서.pptx", "application/octet-stream", "내용".getBytes());
+
+
+     Path saved = sut.saveOriginal(file, "제안서.pptx");
+
+
+     assertTrue(Files.exists(saved));
+     assertTrue(saved.toString().contains("userdata"));
+ }
+
+
+ @Test
+ public void 산출물_중복이름은_숫자를_붙인다() throws Exception {
+     Path dir = sut.getProposalJsonDir();
+
+
+     Path first = sut.writeOutput(dir, "out.json", "{}".getBytes());
+     Path second = sut.writeOutput(dir, "out.json", "{}".getBytes());
+
+
+     assertEquals("out.json", first.getFileName().toString());
+     assertEquals("out_2.json", second.getFileName().toString());
+ }
+
+
+ @Test
+ public void 파일명에서_경로구분자를_떼어낸다() {
+     assertEquals("보고서.pdf", FileStorageService.safeFileName("C:/temp/보고서.pdf"));
+     assertEquals("보고서.pdf", FileStorageService.safeFileName("../..//보고서.pdf"));
+     assertEquals("unnamed", FileStorageService.safeFileName(null));
+ }
+
+ }

```

deleted file mode 100644

@@ -1,85 +0,0 @@

```
-package com.kb.uploader.service;
-
-
-import com.kb.uploader.domain.UploadedFile;
-import com.kb.uploader.dto.ParsedFileName;
-import com.kb.uploader.dto.UploadResultItem;
-import com.kb.uploader.mapper.UploadedFileMapper;
-import org.junit.Test;
-import org.junit.runner.RunWith;
-import org.mockito.InjectMocks;
-import org.mockito.Mock;
-import org.mockito.junit.MockitoJUnitRunner;
-import org.springframework.mock.web.MockMultipartFile;
-
-
-import java.nio.file.Path;
-import java.nio.file.Paths;
-import java.util.Arrays;
-import java.util.Collections;
-import java.util.List;
-import java.util.Optional;
-
-
-import static org.junit.Assert.*;
-import static org.mockito.ArgumentMatchers.*;
-import static org.mockito.Mockito.when;
-
-@RunWith(MockitoJUnitRunner.class)
-public class FileUploadServiceTest {
-
-
-    @Mock private FileParserService fileParserService;
-
-    @Mock private FileStorageService fileStorageService;
-
-    @Mock private ClassificationService classificationService;
-
-    @Mock private UploadedFileMapper uploadedFileMapper;
-
-    @InjectMocks private FileUploadService fileUploadService;
```

```

-
-
- @Test
-
- public void upload_파싱성공_분류성공() throws Exception {
-
-     // given
-
-     MockMultipartFile file = new MockMultipartFile("file", "2024_서울시_보고서.pdf",
-
-         "application/pdf", "content".getBytes());
-
-     ParsedFileName parsed = new ParsedFileName("2024", "서울시", "보고서", "pdf");
-
-     when(fileParserService.parse("2024_서울시_보고서.pdf")).thenReturn(Optional.of(parsed));
-
-     Path storedPath = Paths.get("/tmp/uploader-test/unclassified/2024_서울시_보고서.pdf");
-
-     when(fileStorageService.saveToUnclassified(file, "2024_서울시_보고서.pdf")).thenReturn(storedPath);
-
-     when(classificationService.classify(any(UploadedFile.class))).thenReturn(true);
-
-
-
-     // when
-
-     List<UploadResultItem> results = fileUploadService.upload(Collections.singletonList(file));
-
-
-
-     // then
-
-     assertEquals(1, results.size());
-
-     assertTrue(results.get(0).isClassified());
-
- }
-
-
- @Test
-
- public void upload_파싱실패_미분류() throws Exception {
-
-     MockMultipartFile file = new MockMultipartFile("file", "잘못된파일명.pdf",
-
-         "application/pdf", "content".getBytes());
-
-     when(fileParserService.parse("잘못된파일명.pdf")).thenReturn(Optional.empty());
-
-     Path storedPath = Paths.get("/tmp/uploader-test/unclassified/잘못된파일명.pdf");
-
-     when(fileStorageService.saveToUnclassified(file, "잘못된파일명.pdf")).thenReturn(storedPath);
-
-
-
-     List<UploadResultItem> results = fileUploadService.upload(Collections.singletonList(file));
-
-
-
-     assertEquals(1, results.size());
-
-     assertFalse(results.get(0).isClassified());
-
- }
-
-
- @Test

```

```

- public void upload_예외발생_다른파일계속처리() throws Exception {
-     MockMultipartFile file1 = new MockMultipartFile("file", "오류파일.pdf",
-         "application/pdf", "content".getBytes());
-     MockMultipartFile file2 = new MockMultipartFile("file", "2024_서울시_보고서.pdf",
-         "application/pdf", "content".getBytes());
-     when(fileParserService.parse("오류파일.pdf")).thenReturn(Optional.empty());
-     when(fileStorageService.saveToUnclassified(eq(file1), anyString())).thenThrow(new RuntimeException("저장 오류"));
-     ParsedFileName parsed = new ParsedFileName("2024", "서울시", "보고서", "pdf");
-     when(fileParserService.parse("2024_서울시_보고서.pdf")).thenReturn(Optional.of(parsed));
-     Path storedPath = Paths.get("/tmp/uploader-test/unclassified/2024_서울시_보고서.pdf");
-     when(fileStorageService.saveToUnclassified(eq(file2), anyString())).thenReturn(storedPath);
-     when(classificationService.classify(any(UploadedFile.class))).thenReturn(true);
-
-     List<UploadResultItem> results = fileUploadService.upload(Arrays.asList(file1, file2));
-
-     assertEquals(2, results.size()); // 두 파일 모두 결과에 포함
- }
-}

```

수정 test/service/InstitutionServiceTest.java

```

@@ -17,19 +17,17 @@ import static org.mockito.Mockito.*;

public class InstitutionServiceTest {

    private InstitutionMapper institutionMapper;
-    private ClassificationService classificationService;

    private UploadedFileMapper uploadedFileMapper;
    private InstitutionService sut;

    @Before
    public void setUp() {
        institutionMapper = mock(InstitutionMapper.class);
-        classificationService = mock(ClassificationService.class);

        uploadedFileMapper = mock(UploadedFileMapper.class);
        when(uploadedFileMapper.findByStatus("UNCLASSIFIED")).thenReturn(Collections.emptyList());
-        sut = new InstitutionService(institutionMapper, classificationService, uploadedFileMapper);
+        sut = new InstitutionService(institutionMapper, uploadedFileMapper);
    }

    @Test
    public void findAll_저장소_목록_위임() {

```

```
@@ -23,5 +23,19 @@ mybatis.mapper-locations=classpath:mapper/*.xml
```

```
mybatis.type-aliases-package=com.kb.uploader.domain  
mybatis.configuration.map-underscore-to-camel-case=true
```

```
upload.base-dir=/tmp/uploader-test
```

```
-reclassification.cron=0 */5 * * * *
```

```
+# (2026-09-23 배치 폐지) reclassification.cron=0 */5 * * * *
```

```
+
```

```
+# ---- 2026-09-23 파싱 전환: 원본 보관 / 산출물 경로 ----
```

```
+# .properties 는 ISO-8859-1 로 읽히므로 한글 경로는 \uXXXX 로 적는다.
```

```
+# \uc6d0\ubcf8/\... = 원본/전처리/입찰제안서_PPT
```

```
+# \uc804\ucc98\uub9ac/RFP/\... = 전처리/RFP/파싱후
```

```
+upload.userdata-dir=${upload.base-dir}/userdata
```

```
+upload.proposal-json-dir=${upload.base-dir}/\uc6d0\ubcf8/\uc804\ucc98\uub9ac/\uc785\ucc30\ucc81c\ucc548\ucc11c_PPT
```

```
+upload.rfp-md-dir=${upload.base-dir}/\uc804\ucc98\uub9ac/RFP/\ud30c\uc2f1\uud6c4
```

```
+
```

```
+# 한글 품값·파일명 인코딩 고정
```

```
+server.servlet.encoding.charset=UTF-8
```

```
+server.servlet.encoding.enabled=true
```

```
+server.servlet.encoding.force=true
```

```
+server.tomcat.uri-encoding=UTF-8
```

4. 검증

1. `mvn -B clean test` — `MapperDialectTest` 가 두 방언 XML 을 모두 검사한다. 실제로 이 작업 중 이 테스트가 `findRecent` 누락을 잡아냈다.
2. 기동 후 `GET /` 의 KPI 4개가 실제 건수와 맞는지.
3. RFP(hwp) 1건 업로드 → 전처리/RFP/파싱후/ 에 md 생성 → 그 행의 **저장경로내용이 .md 로 바뀌었는지**.
4. `GET /parse-status` 에서 성공/실패 필터가 도는지. 실패 건은 저장경로가 원본 그대로여야 한다.
5. `GET /api/rfps` 목록과 `?format=text` 원문.
6. `GET /file-status/classified` 에서 기관 지정 → 등록 + 재파싱까지.

외부망에서 확인한 것(참고). TSKGIAF01 의 컬럼 7개를 실제로 DROP 해 원래 11개로 되돌린 뒤 기동했다. 화면 .API 전부 200, 실제 HWP 업로드가 파싱 성공(저장경로 .md, 분류일시 = 파싱 시각, 기관 02 매칭). 테스트 64건 통과.

5. 함정 모음

일자	증상	원인과 교훈
09-23	기존 SQL 이 통째로 사라짐	매퍼를 한꺼번에 바꾸다 <code>findRecent</code> · <code>search</code> 등을 같이 지웠다. MapperDialectTest 가 잡아냈다 — 매퍼를 손대면 이 테스트를 꼭 돌릴 것
09-09	DB 에는 값이 있는데 화면은 빈칸	<code>resultMap</code> 누락. 읽어서 그대로 UPDATE 하는 경로에서 <code>null</code> 이 되써진다
09-08	한쪽 DB 에서만 SQL 이 낡음	<code>databaseId</code> 분기를 한쪽만 수정. 항상 두 벌을 같이
상시	없는 클래스를 참조한다는 오류	<code>target/</code> 의 옛 <code>.class</code> 클래스를 지운 뒤에는 <code>mvn clean</code>
상시	한글 폼값·파일명이 깨짐	<code>server.servlet.encoding.*</code> · <code>server.tomcat.uri-encoding</code> 확인

작성: Claude Code 세션 (2026-09-23) · 목표 `9c42b28` · v1(컬럼 7개판)은 내부망작업지시_2026-09-23_파싱전환이식_v1_컬럼7개판.pdf 로 남겨 두었다 · 미해소 항목은 `GGFileserver/uploader/TODO/TODO_modify.md`